MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

ÍNDICE DE CONTENIDOS

0. ¿CUÁL ES EL PROPÓSITO DE ESTE MÓDULO? 5

1. BIOS LEGACY Y BIOS UEFI 6
   1.1. Introducción 7
   1.2. Rutina de inicio 8
      1.2.1. BIOS LEGACY 9
      1.2.2. BIOS UEFI 10

2. TRANSICIÓN HACIA EL KERNEL 13
   2.1. Proceso de arranque en sistemas BIOS Legacy 14
      2.1.1. Master Boot Record (MBR) 14
      2.1.2. Volume Boot Record (VBR) 15
      2.1.3. Bootmgr.exe y Winload.exe 17
   2.2. Proceso de arranque en sistemas BIOS UEFI 19
      2.2.1. Bootmgfw.efi y Winload.efi 19

3. BOOTKITS 21
   3.1. Introducción a los Bootkits 22
      3.1.1. Fundamentos y mecanismo de operación 22
      3.1.2. Proceso de infección del arranque 24
   3.2. Bootkits in the Wild 26
      3.2.1. EfiGuard 27
      3.2.2. FinSpy 28
      3.2.3. ESPecter 29
      3.2.4. BlackLotus 30
   3.3. Desarrollo de Malware (Base) 32
      3.3.1. Entorno de desarrollo (EDK2) 32
      3.3.2. Mi primera Aplicación UEFI 40
      3.3.3. Diseño de Bootkits modernos 44

4. MODO USUARIO Y MODO KERNEL 47
   4.1. Arquitectura general de un sistema Windows 48
      4.1.1. Modo Usuario 50
         4.1.1.1. Subsistema 50
         4.1.1.2. Procesos del sistema 51
         4.1.1.3. Servicios 52
         4.1.1.4. Procesos de usuario 53
         4.1.1.5. NTDLL 53
      4.1.2. Modo Kernel 54
         4.1.2.1. Executive 54
         4.1.2.2. Kernel 56
         4.1.2.3. Drivers 56
         4.1.2.4. WIN32K 56
         4.1.2.5. HAL 56
   4.2. Diferencias entre programación en modo usuario y modo kernel 57

4.2.1. Gestión de la memoria 57
4.2.2. Gestión de errores 58
4.2.3. Interrupt Request Level (IRQL) 59
4.3. Estructuras habituales en modo kernel 60
4.3.1. UNICODE_STRING 60
4.3.2. DRIVER_OBJECT 61
4.3.3. LIST_ENTRY 62
4.3.4. Estructuras opacas del kernel 63
4.3.4.1. Recursos en internet 64
4.3.4.2. WinDbg 66
4.3.5. Prefijos comunes 67

5. MECANISMOS DE PROTECCIÓN DEL SISTEMA OPERATIVO 68
5.1. Driver Signature Enforcement (DSE) 69
5.2. Procesos Protegidos 72
5.3. Virtualization Based Security (VBS) 74
5.4. Early Launch Anti-Malware (ELAM) 76
5.5. Kernel Patch Protection (KPP) 77
5.6. Control Flow Guard (CFG) 78

6. DEBUG Y ANÁLISIS DEL SISTEMA OPERATIVO 80
6.1. Local Kernel Debugging 81
6.2. Remote Kernel Debugging 85
6.3. Comandos de depuración 91
6.3.1. Símbolos 92
6.3.2. Procesos e Hilos 94
6.3.3. Memoria 95
6.3.3.1. Operaciones de lectura 95
6.3.3.2. Operaciones de escritura 97
6.3.4. Flujo 98
6.3.5. Otros 101
6.4. Scripting 104
6.4.1. Scripts clásicos 104
6.4.2. Javascript 105
6.4.3. Python 106

7. ROOTKITS 107
7.1. Introducción a los Rootkits 108
7.2. Desarrollo de Malware (Base) 109
7.2.1. Entorno de desarrollo (WDK) 109
7.2.2. Mi primer Driver de modo Kernel 112
7.3. Técnicas aplicadas por el malware 116
7.3.1. Interrupt Descriptor Table (IDT) Hooking 117
7.3.2. Direct Kernel Object Manipulation (DKOM) 120
7.3.3. System Service Descriptor Table (SSDT) Hooking 123
7.4. Desarrollo de Malware (Avanzado) 127
7.4.1. Pruebas de concepto Malware 127

7.4.2. Gestión de estructuras 128
7.4.3. Comunicación entre modo usuario y modo kernel 129
7.4.3.1. Input/Output Control Code 129
7.4.3.2. Dispatch Routines 130

8. BIBLIOGRAFÍA 131

9. ANEXOS 132
9.1. Anexo A: Extensiones del estudio del malware 132
9.1.1. Ofuscación de código fuente 132
9.1.2. Explotación de drivers vulnerables 133
9.2. Anexo B: Documentos complementarios 134

0. ¿CUÁL ES EL PROPÓSITO DE ESTE MÓDULO?

En la actualidad, buena parte del malware desarrollado, tiene como objetivo la infección de sistemas operativos Windows, debido al empleo tan extendido de este S.O. para uso personal y empresarial.

En este módulo, como cabría esperar, nos centraremos en este sistema operativo, incidiendo en el kernel del mismo, para entender de una mejor forma su funcionamiento y en consecuencia el comportamiento del malware dirigido a Windows.

Comenzaremos desde el principio: veremos cómo es el proceso de arranque de nuestro sistema, la infección de dicho proceso mediante la implantación de bootkits, el modo usuario y el modo kernel de Windows, nos adentraremos en el desarrollo de drivers maliciosos y despliegue de técnicas de persistencia realizadas por rootkits, llegando finalmente a hacer ingeniería inversa del propio S.O., con el objetivo último de detectar malware que esté corriendo en nuestro equipo y comprender el comportamiento interno de Windows, conocimiento del que tanto los atacantes como nosotros nos aprovechamos.

Espero que este módulo, el último del curso, os resulte emocionante y a la vez os suponga un desafío.

Bienvenidos a todos :)

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

1. BIOS LEGACY Y BIOS UEFI

Empezaremos por conocer la BIOS, el primer tema de varios que nos llevarán a adquirir el conocimiento de operaciones previas a tener nuestra pantalla de inicio de sesión en Windows, lo que son las tareas más fundamentales que ocurren antes de cargar el S.O., donde los componentes hardware y sobre todo firmware se ve son imprescindibles para iniciar nuestro equipo.

1.1. INTRODUCCIÓN

¿Qué entendemos por BIOS? Podríamos definir la BIOS (Basic Input/Output System) como el firmware, software integrado en un dispositivo o componente hardware, que inicializa y gestiona tareas fundamentales durante el proceso de arranque, en nuestro equipo.

La BIOS original, creada en los años 70, consiste en un chip integrado a la placa base que se encarga de realizar una serie de tests para comprobar el correcto funcionamiento de los componentes hardware del equipo, previo a cargar el sistema operativo, proporcionando, posteriormente, un conjunto de rutinas de bajo nivel actuando a modo de "puente", para que el S.O. pueda interactuar con la pantalla, el teclado, el ratón y demás componentes hardware.

El uso de la BIOS tradicional, conocida como BIOS Legacy, es algo singular en la actualidad. Poco a poco, el empleo de esta fue cada vez menor y a principios de los años 2000 surgió la evolución/alternativa de este sistema, UEFI.

UEFI (Unified Extensible Firmware Interface) fue desarrollado por un compendio de empresas para tratar de estandarizar (ya que, con BIOS Legacy cada fabricante hacia lo que consideraba) este proceso de arranque previo a cargar el sistema operativo, creando una interfaz más moderna, fácil de usar y añadiendo funcionalidades centradas en la seguridad, cosa inexistente en BIOS Legacy.

Esta interfaz de firmware, UEFI, lo que nos proporciona es una forma estandarizada para que el hardware y el software de nuestro equipo interaccionen durante el proceso de arranque. En otras palabras, UEFI es una especificación para plataformas x86, x86-64, ARM e Itanium que define una interfaz de software entre el S.O. y el firmware del sistema, que en el contexto moderno reemplaza a la BIOS tradicional.

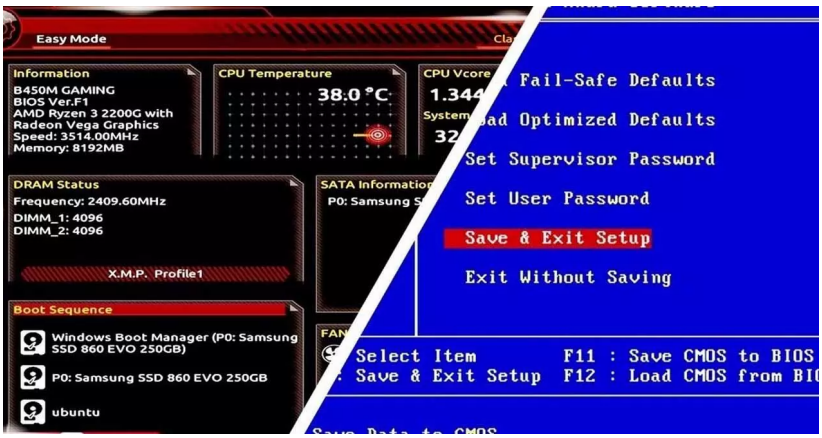


Figure 1: BIOS UEFI y BIOS Legacy

1.2. RUTINA DE INICIO

¿Os habéis preguntado alguna vez que pasa cuando pulsamos el botón de encendido? Cuando esto sucede, la CPU ejecuta automáticamente la instrucción "reset", un comando de bajo nivel que indica a la CPU el inicio del proceso de arranque.

Seguidamente el procesador busca una dirección fija de memoria (0XFFFF0 en procesadores x86 de 16 bits y 0XFFFFFFF0 en procesadores x86 de 32 y 64 bits, dirección conocida como vector de reinicio), que apunta, es decir, es una instrucción de salto, a la primera instrucción de la BIOS o UEFI.

Durante esta fase inicial, nuestro equipo trabaja en "modo real", un modo necesario por razones de compatibilidad con sistemas heredados, mediante el cual, por ejemplo nuestro procesador i9 de última generación ve limitadas sus capacidades, teniendo que utilizar un esquema de direccionamiento de memoria de 16 bits y pudiendo acceder solamente a 1 MB de memoria física.

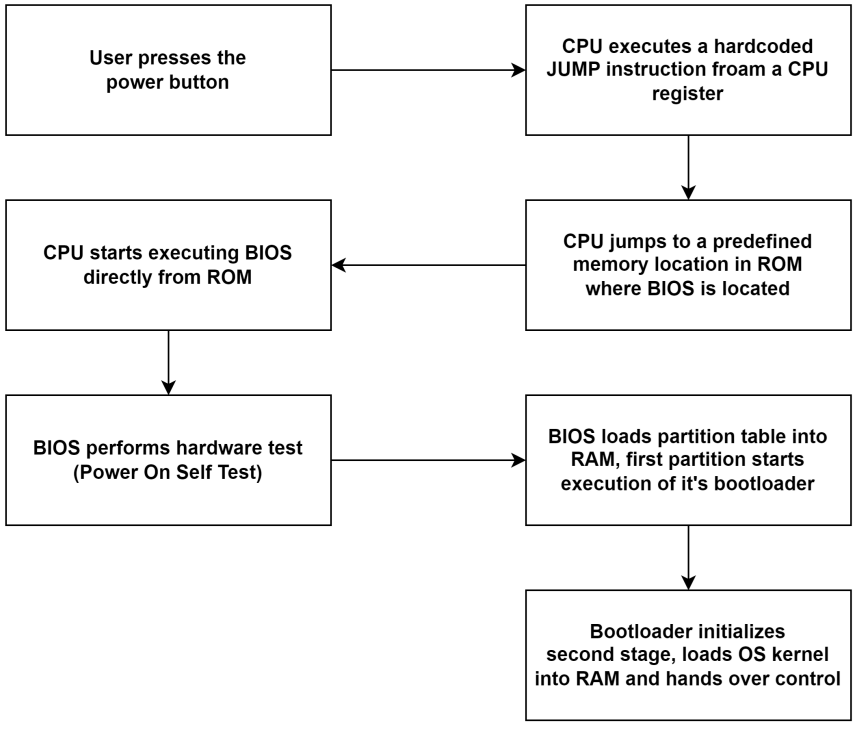


Figure 2: Secuencia de encendido de una máquina

1.2.1. BIOS LEGACY

En las primeras etapas de la ejecución de la BIOS, se lleva a cabo, como comentamos anteriormente, la inicialización del hardware de nuestro equipo y esos tests básicos de diagnóstico para comprobar el correcto funcionamiento del mismo (RAM, disco duro, tarjeta gráfica, …). Esta primera fase se denomina POST (Power-on self-test).

Tras la fase POST, y si no hay errores, ya que en caso contrario se emitirán pitidos de advertencia, ocurre la inicialización de más elementos hardware (que pueden contener "mini bios" ejecutadas también a su vez para inicializarse) como la placa de vídeo o los discos.

La BIOS configura los parámetros del hardware como la cantidad de memoria RAM instalada o la fecha y hora del sistema, entre otros ajustes, almacenando estos valores en una memoria no volátil conocida como CMOS (Complementary Metal-Oxide-Semiconductor).

Acto seguido, busca los dispositivos bootables que se encuentren en nuestro equipo y traslada el control al MBR (Master Boot Record o Registro primario de arranque) del primer dispositivo, iniciando la carga del sistema operativo instalado en dicho dispositivo para continuar con el proceso de arranque.

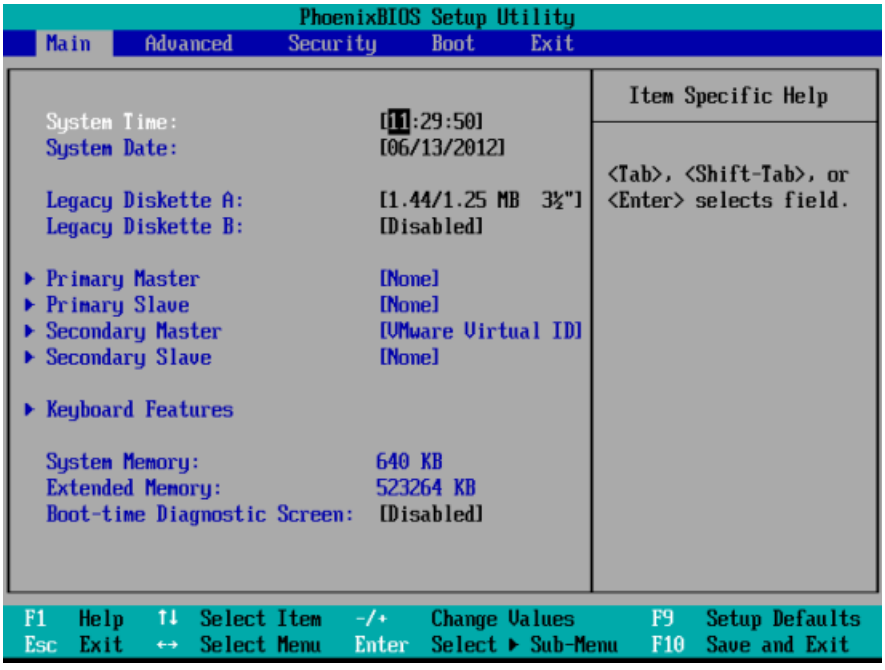


Figure 3: Configuración del arranque en BIOS Legacy

Como ya hemos comentado, BIOS Legacy no era un estándar "oficial", es decir, cada fabricante implementaba o no los pasos que hemos visto, y debido al ecosistema tan variado en torno a las BIOS y a que no se diseñó tomando en consideración la seguridad, los fabricantes decidieron adoptar UEFI.

1.2.2. BIOS UEFI

El uso de la BIOS traía consigo diferentes limitaciones que se debían dejar atrás, por ejemplo, en torno a la seguridad, la única implementación era el propio chip (Security Through Obscurity), de ahí que aplicando ingeniería inversa se llegasen a clonar las primeras BIOS.

En este momento, aunque durante los próximos apartados os generaréis una idea más cercana a la realidad, podéis ver a UEFI como una evolución de la BIOS, cuya adopción proporciona muchas mejoras:

- Se introduce el **estándar PI** (Platform Initialization), que contiene las especificaciones y recomendaciones que los fabricantes deben seguir para desarrollar firmware compatible con UEFI.
- Se mejora la interfaz gráfica de usuario (**GUI**) facilitando la navegación y la configuración del sistema.
- Se estandariza el uso del hardware al incluir el denominado **HAL** (Hardware Abstraction Layer), una capa de abstracción, que utilizan los sistemas operativos modernos, entre el software y el hardware para eliminar la necesidad de conocer la complejidad del hardware subyacente.
- Se añade la capacidad de arrancar desde discos duros con un tamaño máximo de hasta 9.4 ZB (zettabytes), gracias al sistema de **particionado GPT**. De esta forma se logró aumentar el número de particiones primarias de los discos, que con BIOS se encontraba limitado a cuatro.
- Se cambia el modo de funcionamiento, a **modo protegido** x86/x64 en las primeras fases del arranque.
- Se define la tecnología "**Secure boot**", teniendo en cuenta que el proceso de arranque del sistema operativo puede ser un objetivo de ataque, verificando así la integridad de los controladores y del firmware del sistema antes de arrancar.

Aparte de los anteriores puntos, también debemos hacer hincapié en la palabra "estándar", ya que con UEFI, se puede decir que aparece una manera única y consensuada de hacer las cosas, en este caso para definir y llevar a cabo diferentes fases del proceso de arranque en nuestro equipo.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

A continuación, vamos a introducir la arquitectura general de UEFI, fases y elementos que la componen, para entender mejor todo el proceso de inicio en una máquina:

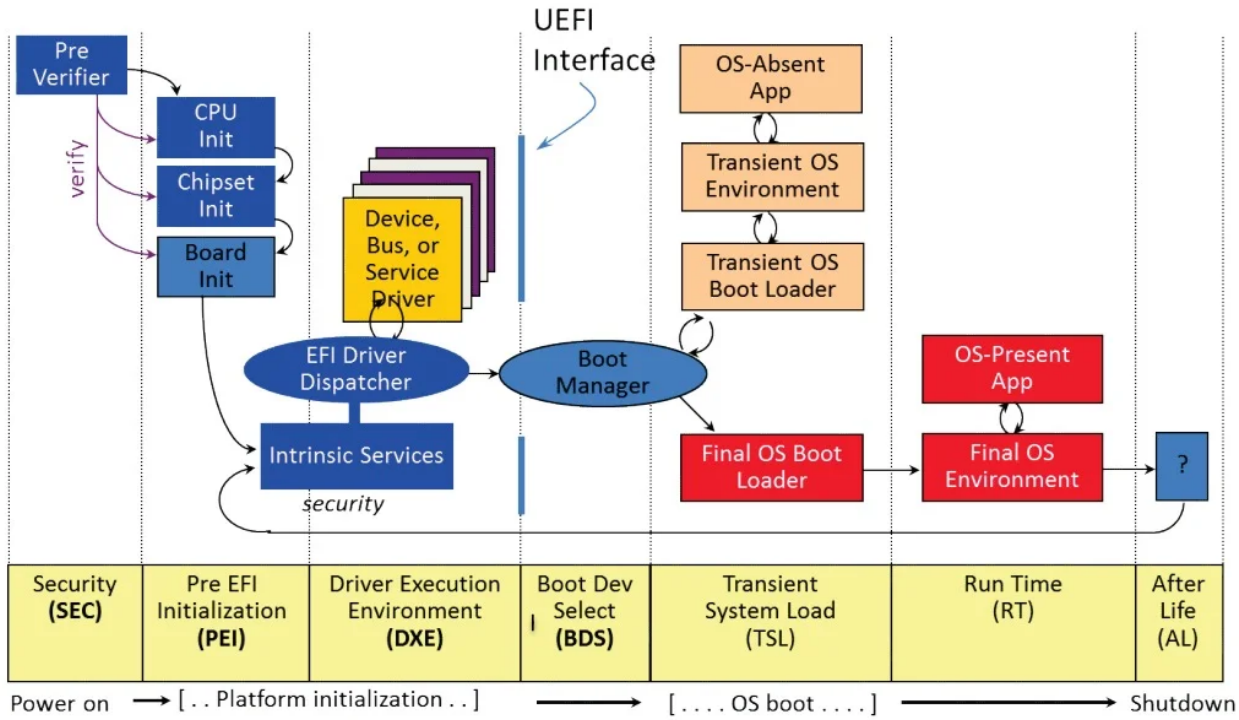


Figure 4: Arquitectura general de UEFI/PI

En la imagen anterior, como podéis observar, aparecen las diferentes fases del arranque en un sistema UEFI. Con estas fases, se definen todos los pasos a seguir desde que se enciende el sistema hasta que se carga el S.O., pero antes de entrar en materia y ver cada una de ellas, nos abstraeremos un poco para verlo de forma sencilla:

- El estándar define una serie de pasos para que estos puedan ser seguidos por los fabricantes y así desarrollar sus propias implementaciones de forma ordenada.
- En cada fase se llevan a cabo esa serie de pasos/acciones definidas previamente.
- Las fases anteriores proveen de información a la fase posterior, en cuanto a qué acción o acciones se han llevado a cabo.
- Por orden de ejecución, en este punto del arranque se dan las fases SEC, PEI, DXE y BDS, siguiendo posteriormente las etapas de TSL, RT y AL.

Fase SEC (Security): Es la primera fase en el proceso de arranque, en donde se realizan comprobaciones sobre el estado de los componentes hardware, es decir, se comprueba la integridad de todos los módulos que se vayan a ejecutar.

En contraposición con BIOS Legacy, además de estas comprobaciones básicas, en esta fase se cambia el funcionamiento del procesador a modo protegido, para poder acceder a una mayor cantidad de memoria física y habilitar la protección de memoria y el control de acceso.

Fase PEI (Pre EFI Initialization): Es la segunda fase del proceso de arranque, y es en esta fase en donde se inicializa la memoria principal (RAM), en un proceso conocido como CAR (Cache as RAM). Durante este proceso, se configura la caché del procesador para que actúe como una memoria RAM temporal, lo que permite una inicialización más rápida y eficiente.

Otra tarea importante es la ejecución de los módulos PEI (PEI modules), módulos que forman parte del firmware, necesarios para realizar tareas más avanzadas como detectar y configurar controladores de hardware adicionales.

Fase DXE (Driver Execution Environment): En esta fase se termina de inicializar el hardware del equipo y se cargan/ejecutan los drivers DXE y los drivers DXE Runtime, de los dispositivos compatibles con UEFI, fijando servicios necesarios para el sistema.

Fase BDS (Boot Device Selection): Es la última fase que trataremos en este punto, fase en donde se buscan dispositivos de almacenamiento conectados al equipo (discos duros, USBs, …), los cuales contengan un archivo de arranque compatible con UEFI, que se encuentra generalmente en la carpeta "\EFI\Microsoft\Boot\".

Llegado este punto, vistas ya parte de las fases de arranque, de forma muy resumida, podemos definir UEFI, como un firmware que reemplaza a la BIOS, considerado un estándar que proporciona una interfaz entre el sistema operativo y el hardware del sistema, el cual además se encarga de cargar drivers, ejecutar aplicaciones y proporcionar servicios entre otras funcionalidades, llegando a actuar como un sistema operativo en sí mismo.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

2. TRANSICIÓN HACIA EL KERNEL

Ahora ya conocido el concepto de BIOS y las operaciones de lo que hemos descrito como primeras rutinas del inicio, nos queda terminar de comprender el proceso de arranque, un proceso mediante el cual se carga el sistema operativo que está instalado en nuestro equipo. Y eso precisamente es lo que vamos a analizar en este punto, todos los elementos que intervienen en la operativa de cargar Windows y el kernel de este.

2.1. PROCESO DE ARRANQUE EN SISTEMAS BIOS LEGACY

2.1.1. MASTER BOOT RECORD (MBR)

Anteriormente, en relación a la BIOS, nos quedamos en la última fase del arranque del sistema, en la que se trasladaba el control del equipo al MBR o Master Boot Record. Este, consta de 512 bytes de datos, distribuidos de la siguiente manera:

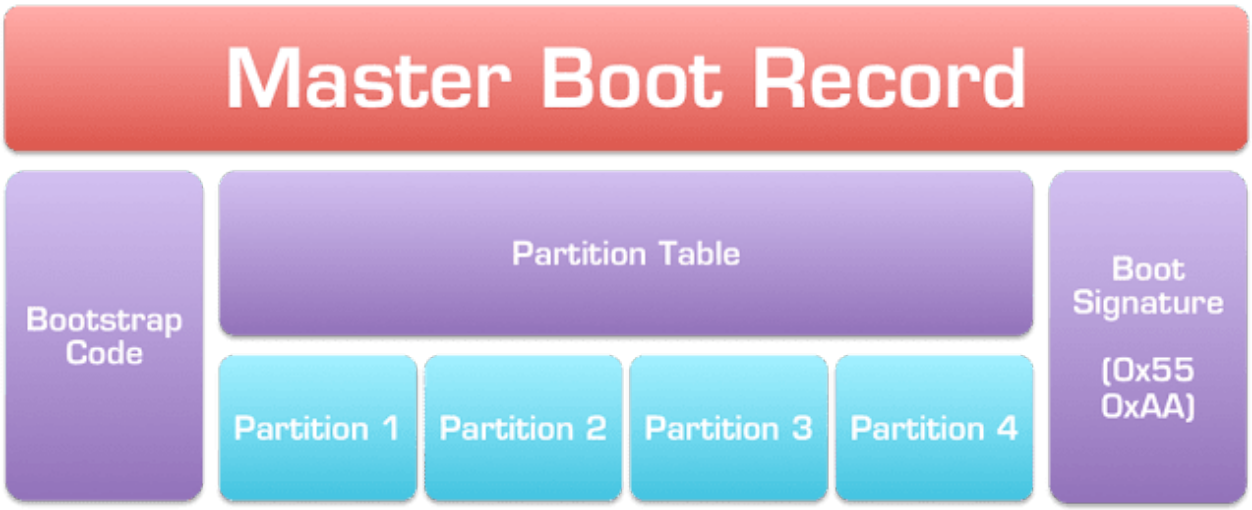


Figure 5: Master Boot Record

- **Código bootstrap (446 bytes)**: Es el código ejecutable del MBR, con el que iniciamos el proceso de carga del sistema operativo. Este buscará la partición activa, que contiene el S.O., y saltará al código del VBR de esa partición.
- **Tabla de particiones (64 bytes)**: En estos bytes se almacena la tabla de particiones, donde se define la ubicación y el tamaño de las particiones del disco.
- **Firma de arranque (2 bytes)**: Los últimos dos bytes del MBR contienen la denominada "firma de arranque", que simplemente le indica a la BIOS que el MBR es válido y que el disco es "arrancable".

2.1.2. VOLUME BOOT RECORD (VBR)

El VBR es similar al MBR, pero este a diferencia, se localiza en el primer sector de cada partición, y contiene la información necesaria para arrancar el sistema operativo que se encuentre en ella. Como ejemplo gráfico, podéis ver la muestra de una extracción del VBR en un equipo Windows.

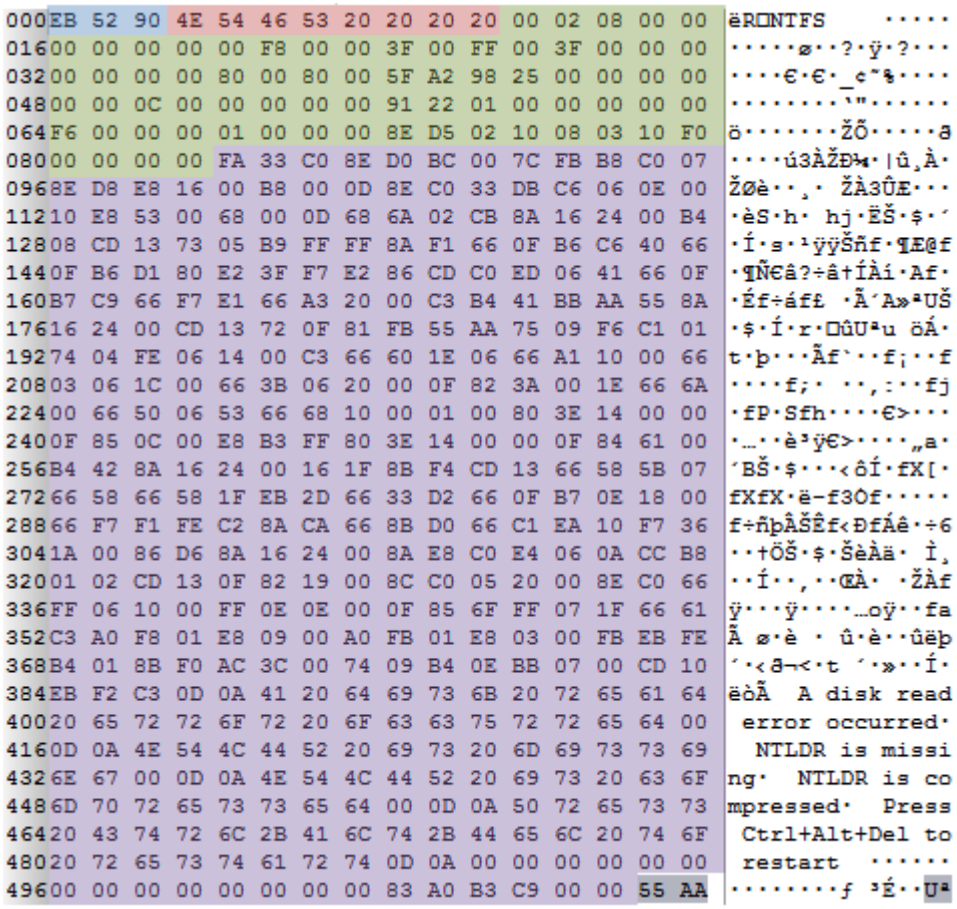


Figure 6: Volume Boot Record

En la imagen, separados por colores, podemos reconocer los siguientes elementos:

- **Instrucción de salto**: Se corresponde con los tres primeros bytes (EB 52 90: JMP 52h + NOP), interpretados como: realizar un salto hacia la parte de color morado, el código de arranque.
- **OEM ID**: Es un campo de 8 caracteres que identifica al fabricante del disco.
- **BIOS Parameter Block (BPB)**: Es una estructura de datos en la que se almacena información sobre el volumen del almacenamiento, como el número de sectores existentes por cada pista.
- **Bootstrap code (código de arranque)**: Es un pequeño programa escrito en lenguaje ensamblador que copia el contenido del gestor de arranque y salta a él.
- **Mensajes de error**: Si durante la ejecución del VBR ocurre algún error, este se mostrará, como podéis comprobar en una parte de la sección del código morado, y se detendrá la carga del sistema operativo.
- **Firma de arranque**: Al igual que en el MBR, nos encontramos la firma de arranque "0x55 0xAA", indicando que el VBR es válido y se puede utilizar para el arranque.

Como resumen, y para diferenciar estos dos elementos, podríamos comentar que, el VBR es responsable de cargar el sistema operativo desde la partición en la que se encuentra, mientras que el MBR es responsable de encontrar la partición activa y cargar el VBR correspondiente.

2.1.3. BOOTMGR.EXE Y WINLOAD.EXE

Una vez que el VBR carga el programa denominado gestor de arranqe o Boot Manager (Bootmgr.exe), este se encarga de detectar los sistemas operativos instalados en el disco, siguiendo con el proceso de inicio del sistema.

Generalmente, el Boot Manager pasa desapercibido, pero en ocasiones podemos verlo en acción, cuando nos presenta una lista de opciones, como vemos en la imagen.

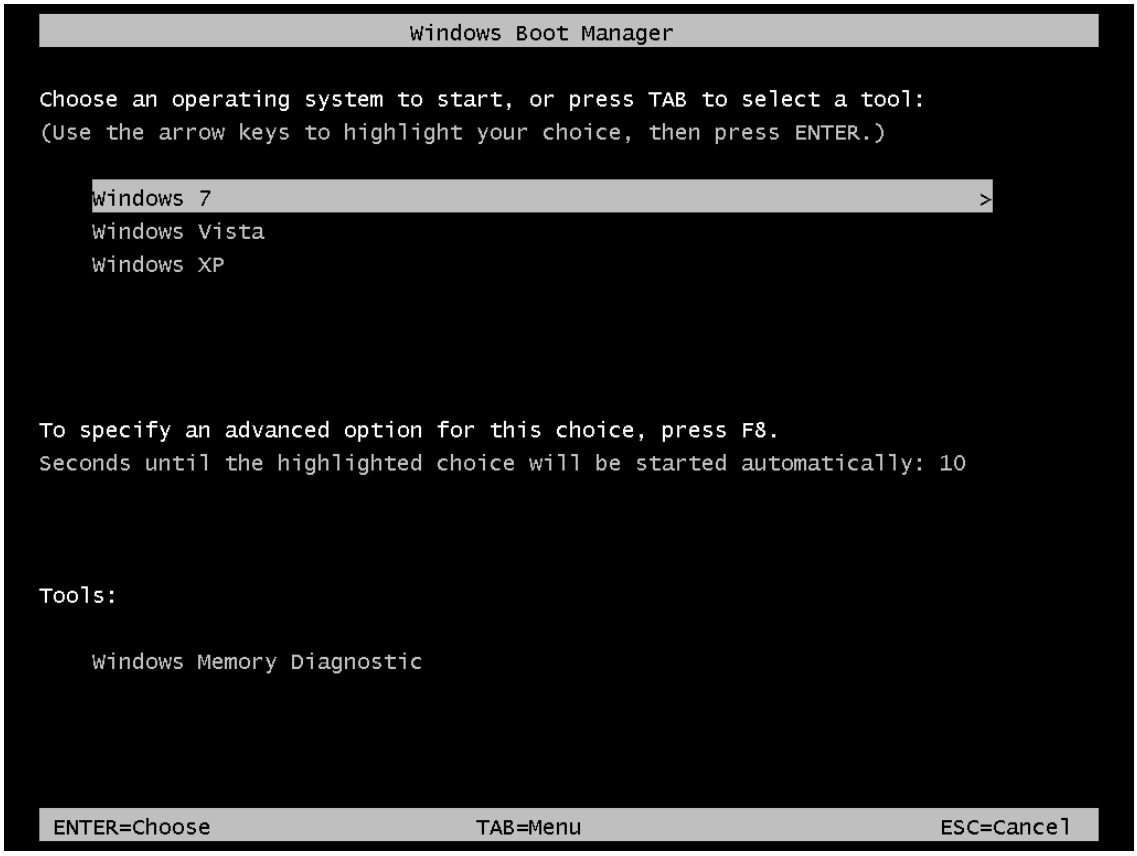


Figure 7: Windows Boot Manager

En este punto, en BIOS Legacy, tenemos que recordar que la ejecución aún sigue en modo real. Sin embargo, una vez se carga el Windows Boot Manager, se cambia el modo de operación del procesador a modo protegido.

Para terminar el proceso, se cede el control a uno de estos tres ejecutables, cada uno con un propósito específico, siendo el cargador de arranque (Winload.exe) el programa seleccionado en una secuencia de inicio común:

- **Memtest.exe**: Es una herramienta de diagnóstico de la memoria, utilizada comúnmente para detectar errores. Su uso es recomendable especialmente después de la instalación de un nuevo módulo de memoria o cuando se presentan problemas inexplicables de funcionamiento en el sistema.
- **Winresume.exe**: Un archivo utilizado para recuperar el estado del sistema si nuestro equipo se ha suspendido o hibernado, crucial para la restauración del equipo a su último punto funcional al asegurar que todas las aplicaciones y datos se reanudan sin corrupción.
- **Winload.exe**: Un binario de Windows que actúa como cargador de arranque, cargando, valga la redundancia, el propio kernel del sistema operativo así como los controladores necesarios. Este fichero, localizado en la ruta "\windows\system32\", es vital para la preparación del entorno del S.O., permitiendo que las subsiguientes capas del software se ejecuten correctamente.

2.2. PROCESO DE ARRANQUE EN SISTEMAS BIOS UEFI

2.2.1. BOOTMGFW.EFI Y WINLOAD.EFI

En BIOS UEFI, tras suceder la operación Power-On Self-Test o POST, ese proceso que verifica el hardware asegurándose que los componentes esenciales funcionan correctamente, se lanza el firmware UEFI, buscando las particiones y los archivos necesarios para el proceso de arranque del S.O.

Uno de esos archivos, fundamentales para efectivamente que se lleve a cabo la rutina de inicio, será el Windows Boot Manager (bootmgfw.efi), que se localiza en una partición especial del disco duro, la ESP (EFI System Partition) o partición del sistema EFI.

Este archivo, que se encarga de iniciar el proceso de arranque de Windows en sistemas UEFI, es el reemplazo natural del bootmgr.exe que vimos anteriormente, aunque existen varias diferencias entre ellos relacionadas con los sistemas sobre los que corren, como que ahora no se cambia el modo de ejecución a modo protegido porque ya se hizo esa modificación durante la fase SEC.

Este gestor carga el Windows OS Loader (winload.efi), que a su vez carga el sistema operativo en memoria y los controladores necesarios para el arranque de Windows.

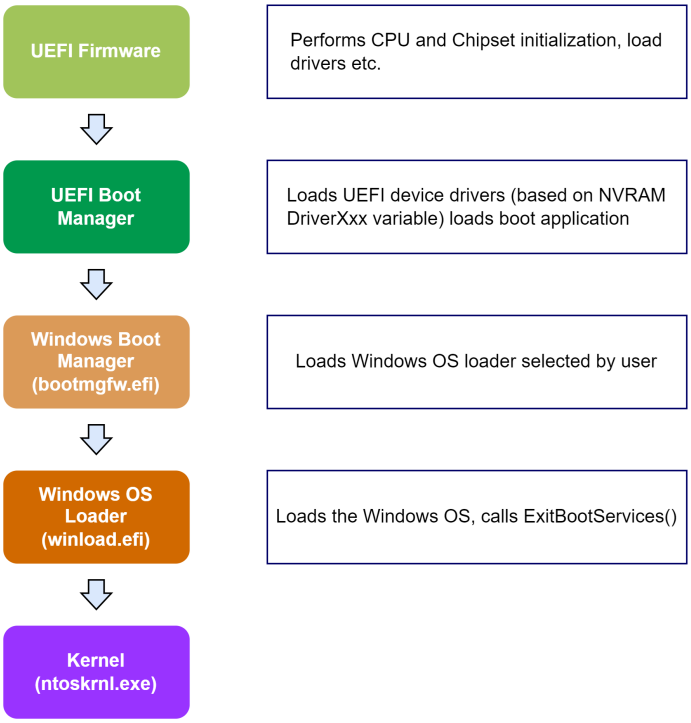


Figure 8: Proceso de arranque de Windows en sistemas UEFI

Para verlo de una forma más gráfica y completa, en la siguiente imagen se muestra la comparativa entre el proceso de arranque en sistemas BIOS Legacy y BIOS UEFI.

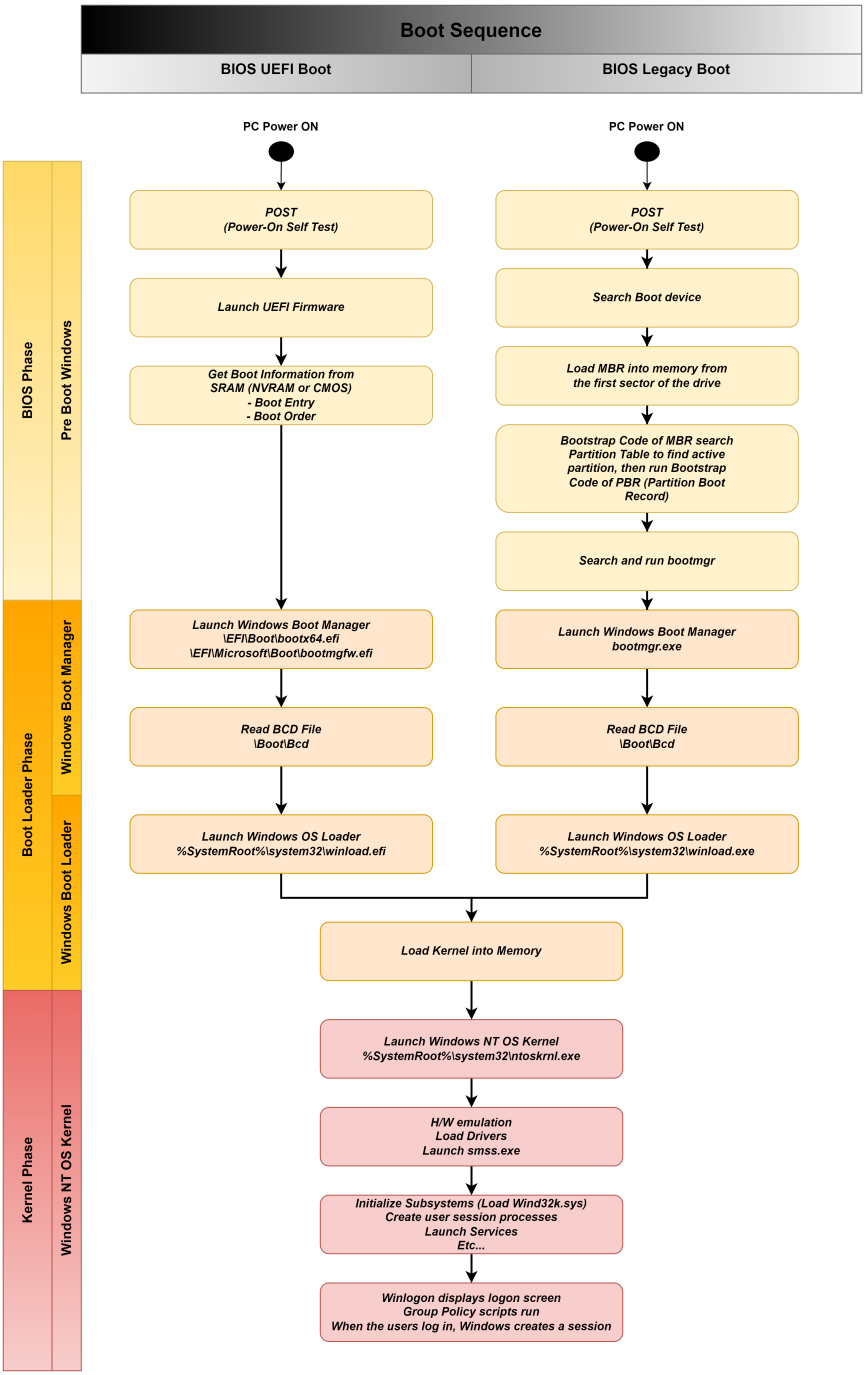


Figure 9: Comparativa del proceso de arranque entre sistemas BIOS y UEFI

3. BOOTKITS

Una vez comprendido el proceso de arranque del sistema operativo Windows, es necesario que hagamos la reflexión sobre que ese proceso es crítico para la funcionalidad y seguridad del equipo, siendo un punto de infección atractivo en muchos ciberataques.

Aquí es donde entran en juego los bootkits, una forma de malware avanzada que está diseñada para alterar la secuencia de arranque vista. Un malware en el que profundizaremos, explorando cómo este manipula la rutina normal de encendido, ejecutando código malicioso a un nivel muy bajo, lo que le permite esquivar muchas de las medidas de seguridad implementadas en capas superiores del sistema.

3.1. INTRODUCCIÓN A LOS BOOTKITS

3.1.1. FUNDAMENTOS Y MECANISMO DE OPERACIÓN

Con el término 'bootkit' hacemos referencia a un tipo de malware avanzado cuyo objetivo es infectar el proceso de arranque de un equipo, proceso en el que, como hemos visto anteriormente, interviene directamente UEFI en el contexto de un sistema moderno. Este software malicioso se carga durante las primeras etapas de la rutina de encendido, antes de que se inicie el sistema operativo, lo que le permite eludir toda medida de seguridad y llevar a cabo operaciones críticas, sin oposición alguna.

La implantación de este malware tan complejo, sobre una máquina objetivo, lleva a obtener un control altamente persistente y de bajo nivel sobre la misma, permitiendo a los atacantes realizar una variedad de actividades maliciosas sin ser detectados. Estamos ante el malware más sofisticado, complejo, con mayor capacidad de control y cuyo desarrollo plenamente funcional solo está al alcance de ciertas naciones y APTs.

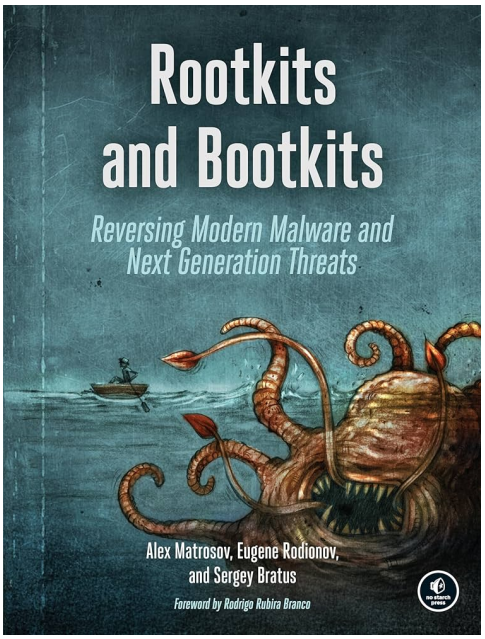


Figure 10: Volumen de referencia sobre Bootkits y amenazas avanzadas

Los bootkits poseen características que les permiten manipular el proceso de arranque y ocultar su presencia. Algunas de estas características incluyen la capacidad de alterar código de ficheros intervinientes en el arranque mediante técnicas de hooking y parcheo, la capacidad de modificar el código del kernel para ocultar su actividad maliciosa e impedir la detección por parte de soluciones antimalware, y la capacidad de, mientras la máquina se inicia, realizar operaciones de bajo nivel y emitir comunicaciones utilizando varios protocolos, entre otras acciones maliciosas.

En el contexto de los bootkits orientados a sistemas operativos modernos, esas tres operaciones clave mencionadas se despliegan con cierta sofisticación técnica de acuerdo a la especificación de UEFI y la versión del S.O. a infectar:

- **Modificación de archivos relacionados con el proceso de arranque**

  Técnicas de Hooking y Parcheo: Los bootkits UEFI pueden alterar directamente el firmware UEFI o los archivos que se cargan durante el proceso de arranque. Aplicando técnicas de hooking, un bootkit puede interceptar las llamadas de funciones del firmware, desviándolas hacia su propio código malicioso, lo que le permite ejecutar operaciones antes de que el sistema operativo se cargue, manteniendo el control sobre qué partes del firmware se ejecutan y cuáles se modifican o suprimen. Además, los bootkits implementan parches, es decir, modifican el contenido de archivos críticos para alterar el flujo de ejecución original y así ejecutar código arbitrario. Este nivel de control puede ser utilizado para deshabilitar funciones de seguridad o para cargar componentes maliciosos adicionales.

- **Modificación del código del kernel para ocultar actividades maliciosas**

  Artefactos a ejecutar en Ring 0: Una vez que el sistema operativo comienza a arrancar, el bootkit previamente ya ha establecido un entorno que le permite manipular el proceso de carga del kernel, pudiendo insertar su propio código o modificar el ya existente para ocultar su actividad y asegurarse la persistencia. Esto se lleva a cabo desplegando rootkits de kernel que manipulan las tablas de funciones del sistema operativo interceptando y modificando llamadas, lo que permite ocultar archivos, procesos y conexiones de red. Estas alteraciones hacen que el bootkit sea invisible no solo para los usuarios sino también para muchas soluciones antimalware que dependan de la integridad del kernel a fin de funcionar correctamente.

- **Comunicaciones de bajo nivel durante la rutina de encendido**

  Protocolos de red y envío de paquetes: Los bootkits UEFI tienen la capacidad única de ejecutar código y comunicarse utilizando protocolos estándar como HTTP, DNS, TCP, Bluetooth y WIFI, entre otros, desde el mismo momento en que la máquina se inicia. Esto les permite no solo recibir instrucciones de un servidor de command and control (C&C) o descargar otros componentes de malware antes de que se active cualquier software de seguridad, sino también interceptar o manipular el tráfico de red saliente e incluso bloquear o redireccionar ciertas comunicaciones. Esta capacidad de interactuar con la red desde tan temprano en el proceso de arranque es particularmente preocupante debido a la dificultad de detección y la amplitud de las potenciales actividades maliciosas que puede facilitar.

3.1.2. PROCESO DE INFECCIÓN DEL ARRANQUE

El proceso de infección del arranque es un procedimiento técnicamente avanzado que permite implantar código malicioso durante las primeras etapas de la rutina de inicio. Este proceso, como hemos comentado, se lleva a cabo mediante técnicas de hooking y parcheo en funciones críticas de los ficheros intervinientes en el arranque (bootmgfw.efi, winload.efi, ntoskrnl.exe). A continuación, se describen los pasos típicos que un bootkit lleva a cabo durante la infección:

- **Sustitución del Windows Boot Manager (bootmgfw.efi)**: El primer paso es reemplazar el archivo que referencia al Windows Boot Manager, conocido como bootmgfw.efi, que se encuentra en la partición ESP (EFI System Partition) en el directorio \EFI\Microsoft\Boot\bootmgfw.efi. Al sustituirlo con una versión maliciosa, el bootkit puede interceptar y controlar el flujo de arranque desde el principio.
- **Hooking de funciones de la especificación UEFI**: Después de la sustitución del bootmgfw.efi, el siguiente paso es realizar el hooking de funciones clave definidas en la especificación UEFI. Una función crítica para hookear aquí es gBS->LoadImage, que se utiliza para cargar imágenes EFI en la memoria. Al interceptar esta función, el bootkit se asegura que su propio código corra antes de que cualquier otro componente del sistema se ejecute.
- **Carga de bootmgfw.efi original**: Una vez que la función gBS->LoadImage ha sido hookeada, el bootkit carga el Windows Boot Manager (bootmgfw.efi) original, manteniendo el control sobre el flujo de ejecución y asegurando que las modificaciones y hookings adicionales se realizan correctamente.
- **Hooking de funciones clave en bootmgfw.efi**: Dentro del bootmgfw.efi, y esto lo podemos comprobar al utilizar un desensamblador como IDA, una función importante, por ser la responsable de iniciar el Windows OS Loader (winload.efi), es ImgArchStartBootApplication. Al hookear dicha función, el bootkit puede continuar ejecutándose incluso después de que el Windows OS Loader haya sido cargado, manteniendo así su control sobre el proceso de arranque.
- **Hooking de funciones clave en winload.efi**: El siguiente objetivo es el archivo winload.efi, y una función interesante de este es OslFwpKernelSetupPhase1, función que se encarga de las operaciones iniciales de configuración del kernel de Windows (ntoskrnl.exe). De forma parecida al paso anterior, hookeando OslFwpKernelSetupPhase1, el bootkit puede seguir ejecutándose y manipulando el flujo de ejecución a medida que el kernel de Windows se carga.
- **Deshabilitar protecciones del sistema**: Una vez que el kernel de Windows (ntoskrnl.exe) está cargado, completamente indefenso, el bootkit deshabilita varias protecciones del sistema para asegurar su persistencia. Esto incluye deshabilitar el mecanismo Driver Signature Enforcement (DSE) mediante el parcheo de la función SeCodeIntegrityQueryInformation que se encuentra en CI.dll.

- **Gestión de errores y manejo de condiciones**: Durante todo este proceso, es crucial gestionar correctamente los errores y las condiciones excepcionales para evitar la inestabilidad del sistema. Un manejo inadecuado de errores puede resultar en fallos críticos, como un pantallazo en negro o, con surte, la conocida pantalla azul de Windows (BSOD). Por lo tanto, este malware implementa mecanismos robustos de manejo de errores para asegurarse que no es detectado y que el sistema permanece operativo.
- **Explotación de vulnerabilidades de la BIOS y/o SecureBoot**: En algunos casos, el bootkit puede necesitar explotar vulnerabilidades en la BIOS o en el mecanismo de arranque seguro, SecureBoot, para evadir las verificaciones de integridad, permitiendo la carga de componentes no verificados durante el arranque. Esto es lo que permitiría al bootkit ejecutarse, incluso en sistemas que implementen las medidas de seguridad apropiadas.

En resumen, el proceso de infección del arranque, que realiza este malware, es un ataque sofisticado que involucra la sustitución y modificación de componentes críticos de la máquina desde las primeras etapas del arranque, asegurando así la persistencia y la capacidad de evadir medidas de seguridad a nivel de sistema operativo.

3.2. BOOTKITS IN THE WILD

Todos los bootkits, durante su inicio, funcionan de manera similar, asemejándose su operativa a la infección del arranque descrita anteriormente. Sin embargo, cada bootkit puede incluir curiosidades y mejoras específicas durante o a posteriori de ese proceso, diferenciándose de otros.

Para comprender mejor los conceptos y mecanismos discutidos sobre este malware, mientras recordamos que el mecanismo básico de infección es siempre el mismo, siendo la única diferencia que algunos bootkits hookean otras funciones en las que no hemos entrado, vamos a comentar algunos ejemplos que han sido utilizados en campañas de espionaje alrededor del mundo.

Estos ejemplos ilustran cómo los bootkits son implementados y desplegados por grupos de actores maliciosos y/o estados, cada cierto tiempo, en operaciones muy complejas, dirigidas y avanzadas.

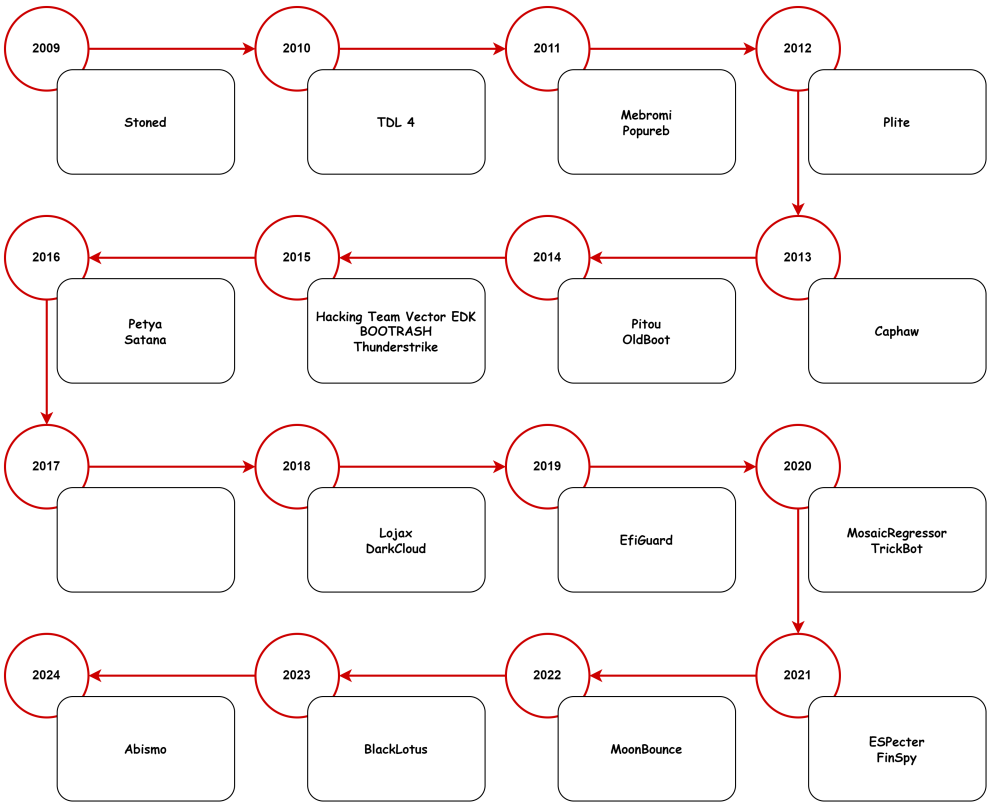


Figure 11: Cronología de la aparición de Bootkits modernos

3.2.1. EFIGUARD

EfiGuard es una prueba de concepto de bootkit UEFI portátil para arquitecturas x64 que, como todos durante el encendido, realiza parches en el gestor de arranque de Windows, el cargador de arranque y el kernel, siendo su finalidad principal el evadir mecanismos de seguridad específicos de Windows como Kernel Patch Protection (PatchGuard) y Driver Signature Enforcement (DSE) que veremos en posteriores temas.

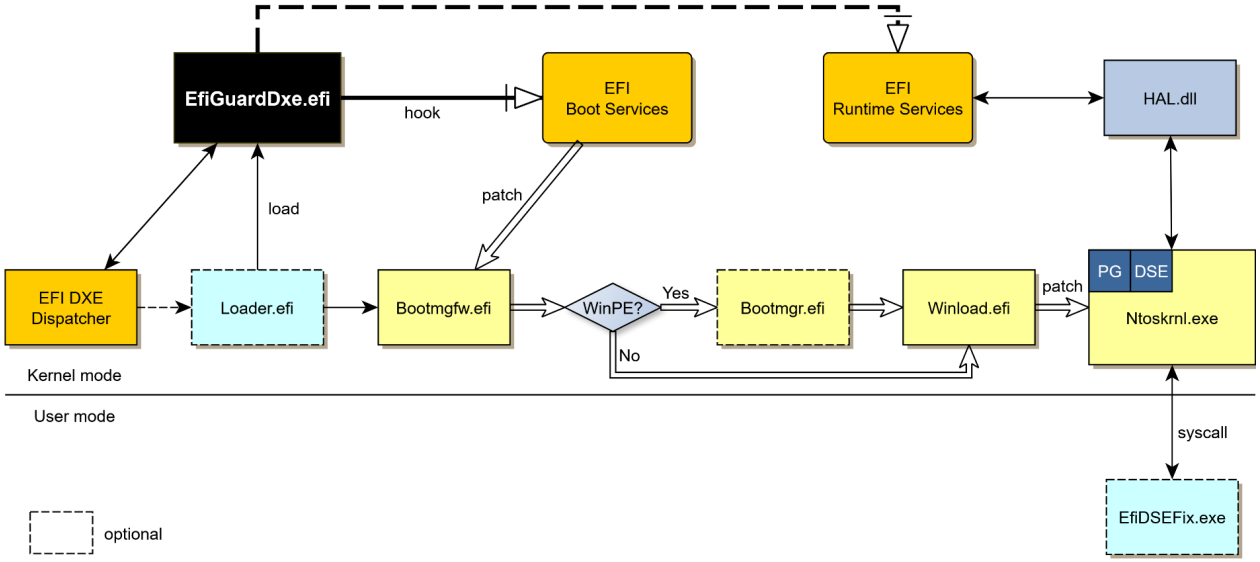


Figure 12: Proceso de infección de UEFI en el bootkit EfiGuard

3.2.2. FINSPY

FinSpy, también conocido como FinFisher o Wingbird, es un bootkit que despliega software de espionaje en espacio de usuario, permitiendo así llevar a cabo una vigilancia detallada de los sistemas infectados, recolectando información sensible y realizando comunicaciones con servidores de command and control. Además, FinSpy instala un troyano propio como software de control, pudiendo instalar y ejecutar componentes adicionales, lo que lo convierte en una herramienta maliciosa muy buena para el espionaje a nivel gubernamental y corporativo.

Figure 13: Proceso de infección de UEFI en el bootkit FinSpy

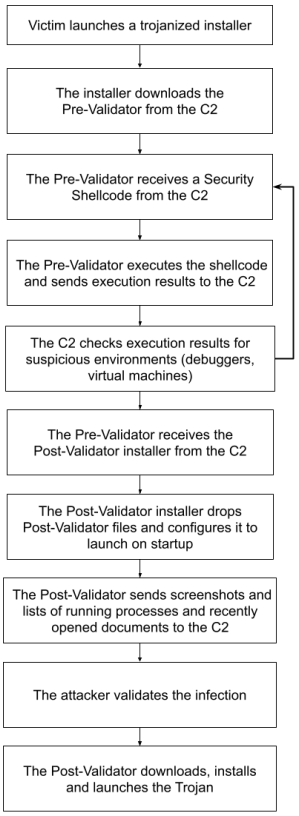


Figure 14: Proceso de instalación de software adicional en el bootkit FinSpy

3.2.3. ESPECTER

ESPecter es la representación de bootkit UEFI básico con capacidades avanzadas una vez se carga el kernel, ya que incluye un keylogger totalmente indetectable en ese nivel. Este keylogger captura todas las pulsaciones de teclas realizadas en el sistema, permitiendo robar información sensible como contraseñas y datos de inicio de sesión. La capacidad de operar a nivel del kernel hace que el keylogger de ESPecter sea extremadamente difícil de eliminar, lo que subraya la sofisticación y el peligro de este malware.

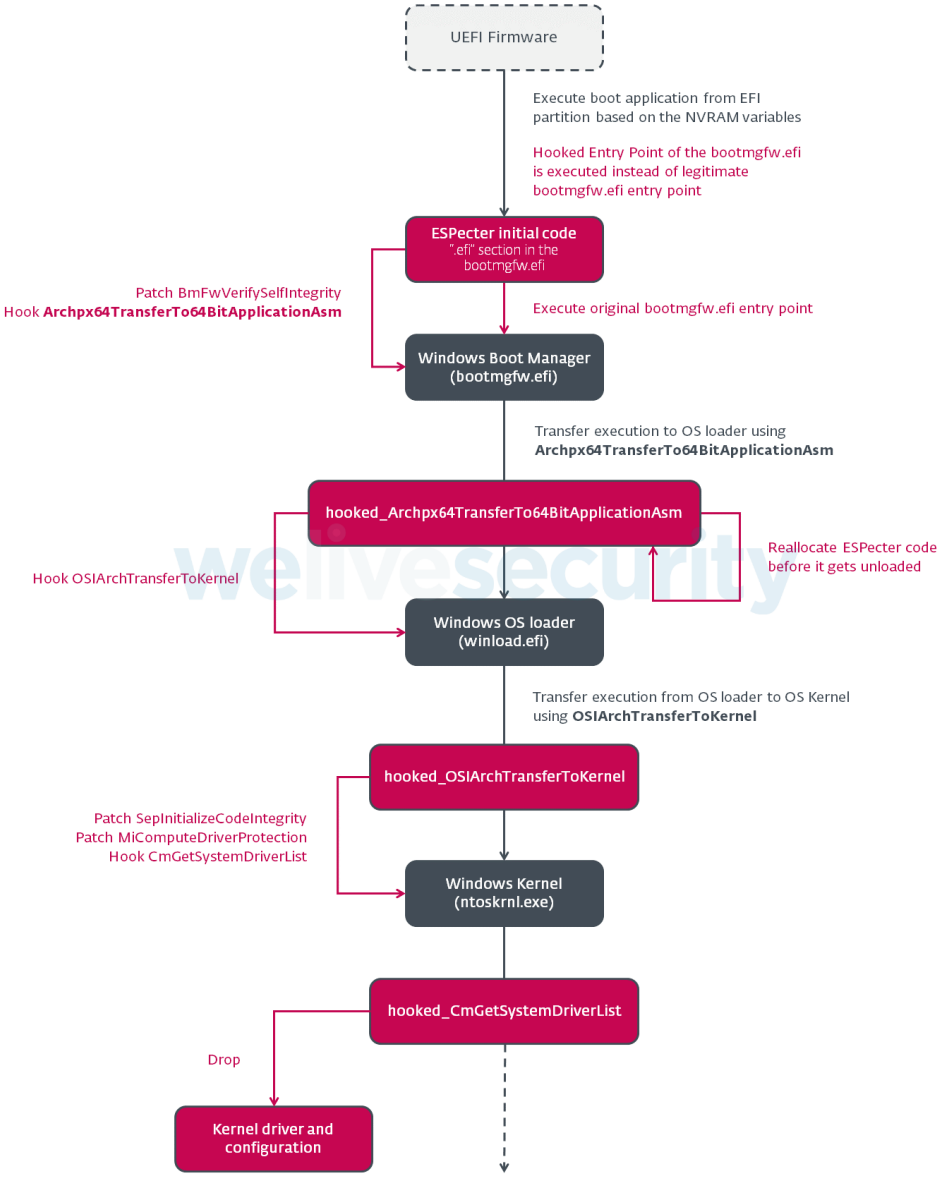


Figure 15: Proceso de infección de UEFI en el bootkit ESPecter

3.2.4. BLACKLOTUS

BlackLotus es el único bootkit conocido hasta la fecha que explota una vulnerabilidad en la BIOS para evadir la protección Secure Boot, una característica de seguridad crítica en los sistemas modernos que asegura que solo se ejecuten binarios confiables durante el arranque. Al verificar las firmas digitales de los componentes del arranque, Secure Boot previene que se ejecute un bootkit en la máquina, ya que esta no podrá iniciar al haberse modificado el Windows Boot Manager. Sin embargo, BlackLotus, explotando la vulnerabilidad Baton Drop, conocida por el CVE-2022-21894, consigue deshabilitar Secure Boot y así establecer su persistencia. Esta capacidad de evadir Secure Boot es especialmente preocupante porque permite que el malware se cargue incluso en sistemas actualizados y configurados correctamente.

BlackLotus, con sus operaciones, demuestra la importancia de estar atentos a las vulnerabilidades que se conocen sobre la BIOS, ya que es el componente más crítico de un equipo, y la importancia de aplicar parches de seguridad a la mayor brevedad posible.

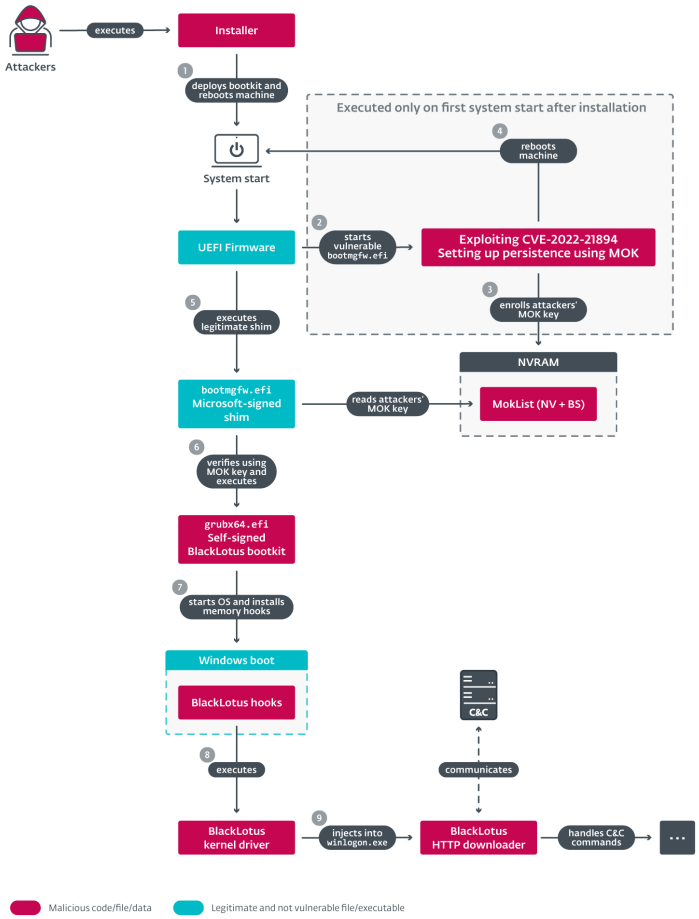


Figure 16: Proceso de infección de UEFI en el bootkit BlackLotus

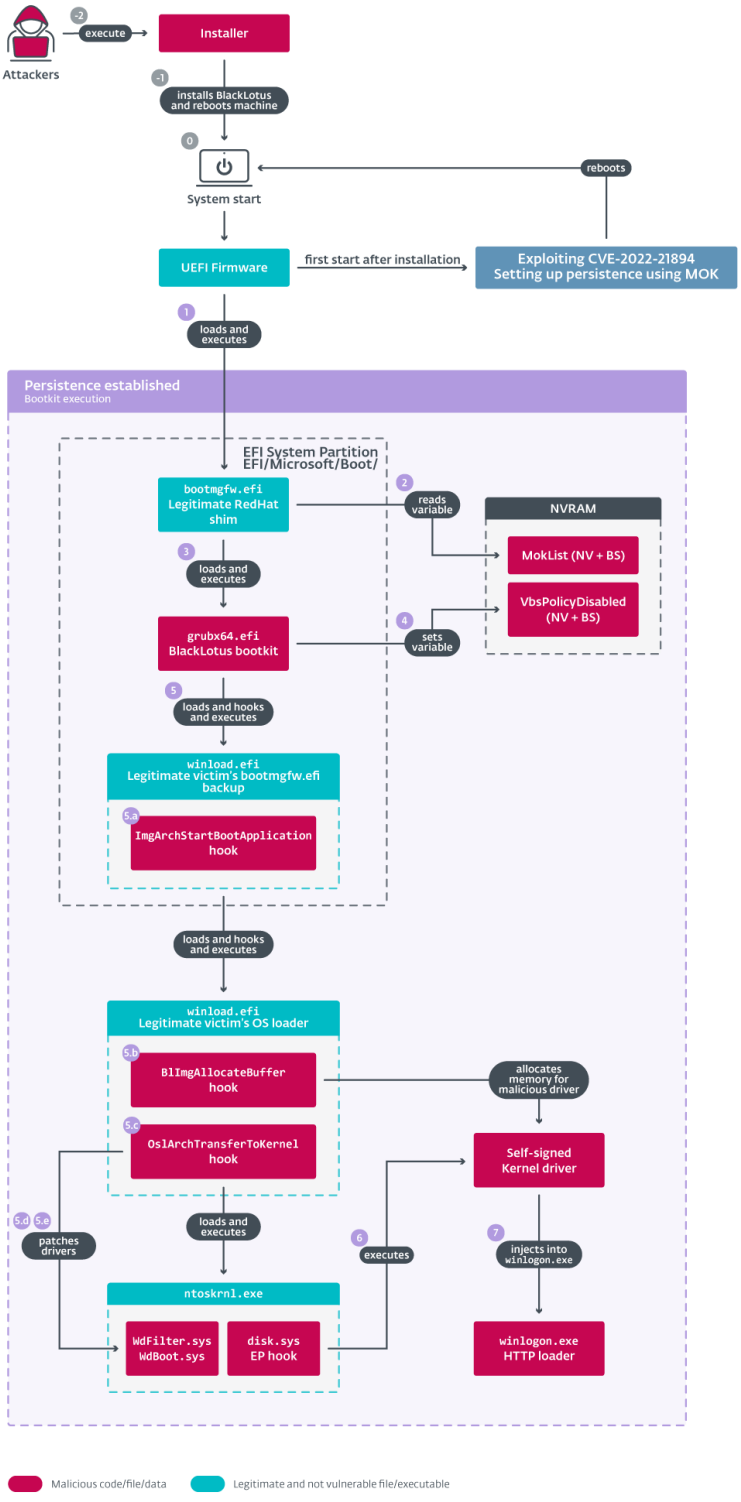


Figure 17: Proceso de evasión de SecureBoot en el bootkit BlackLotus

3.3. DESARROLLO DE MALWARE (BASE)

3.3.1. ENTORNO DE DESARROLLO (EDK2)

Tianocore EFI Development Kit 2 o EDK2 es el entorno de desarrollo oficial para aplicaciones/drivers UEFI y DXE, un proyecto de código abierto con contribuciones principales de empresas como Intel, HP y Microsoft, que nos ofrece la puerta de entrada para desarrollar Bootkits orientados a infectar sistemas modernos.

Este entorno es una implementación completa de la especificación UEFI, que incluye paquetes de firmware para facilitar a los desarrolladores de sistemas operativos, o de malware, la prueba de sus aplicaciones UEFI.

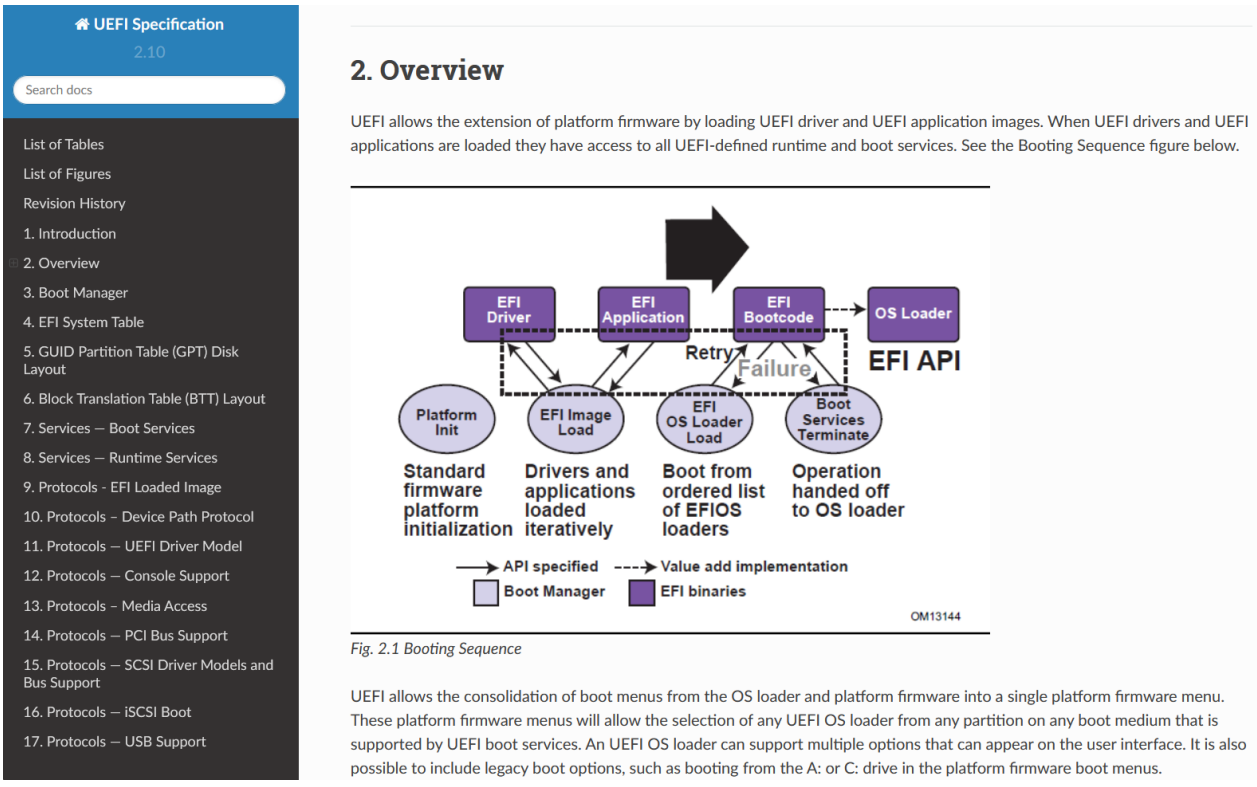


Figure 18: Especificación UEFI

Para comenzar a trabajar con EDK2, se debe configurar el entorno mediante algún script que automatice el proceso o de forma manual, y previamente a clonar el repositorio, es esencial instalar y configurar una serie de herramientas específicas, que son requisito para compilar el código que desarrollemos:

- **Git**: Git es esencial para la gestión de versiones y el control del código fuente que nos proporciona la librería de EDK2.

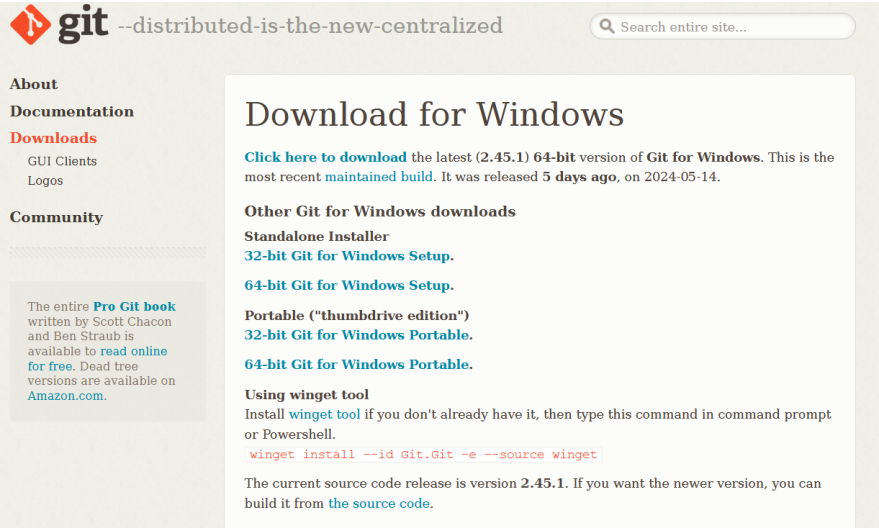


Figure 19: Descargar e instalar Git en Windows

- **Python**: La versión de Python 3.7 o superior es necesaria si queremos ejecutar herramientas de Python desde nuestro código fuente, teniendo que configurar la variable de entorno PYTHON_HOME para que apunte al directorio de instalación y así localizar e invocar el intérprete, set PYTHON_HOME=C:\Python37.

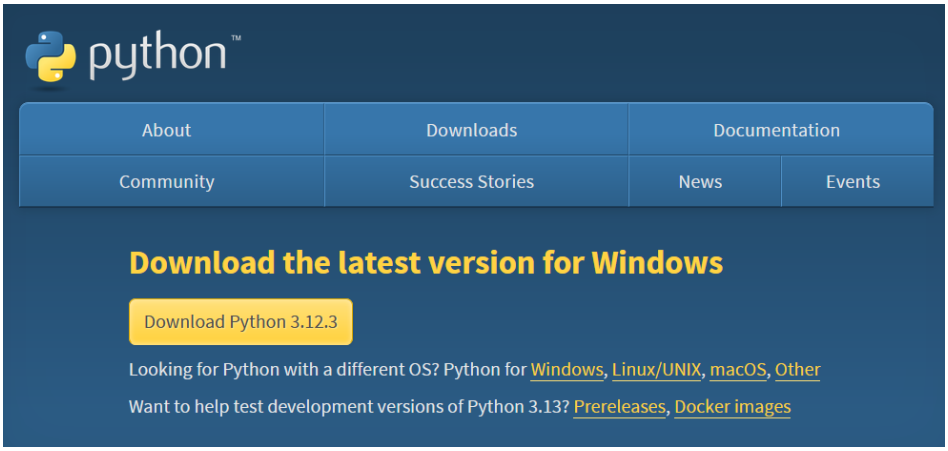


Figure 20: Descargar e instalar Python en Windows

- **Visual Studio Community**: Siguiendo la guía oficial de despliegue del entorno en un sistema Windows, se indica que la mejor opción es optar por el IDE predeterminado de Microsoft, Visual Studio, seleccionando el workload "Desarrollo de escritorio con C++" durante la instalación para obtener las herramientas y bibliotecas esenciales.

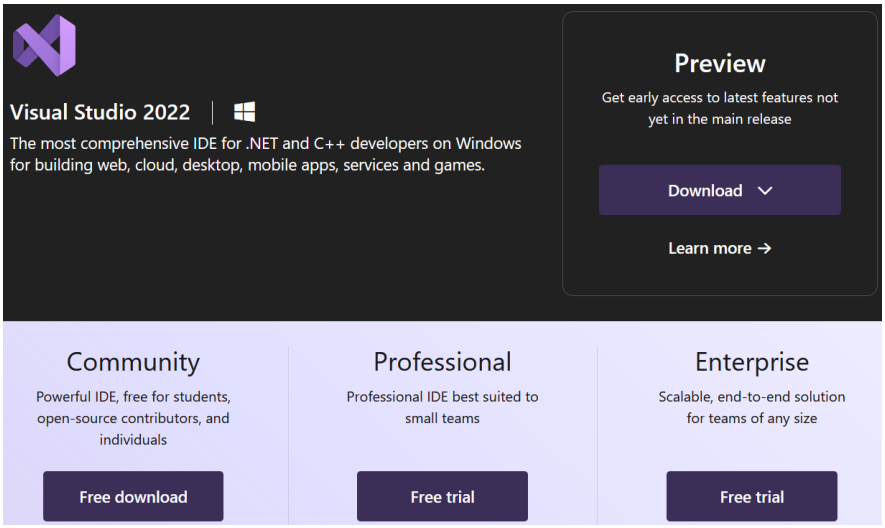


Figure 21: Descargar e instalar Visual Studio en Windows

- **NASM (Netwide Assembler)**: NASM 2.15.05 es la versión mínima recomendada para el desarrollo de firmware que incluya componentes de ensamblador, por sus mejoras en el soporte de mnemónicos de instrucciones de CPU y opciones para compilaciones deterministas. Para ejecutar esta utilidad debemos configurar la variable de entorno NASM_PREFIX, apuntando al directorio donde se instaló NASM, incluyendo la barra invertida final, NASM_PREFIX=C:\nasm\.

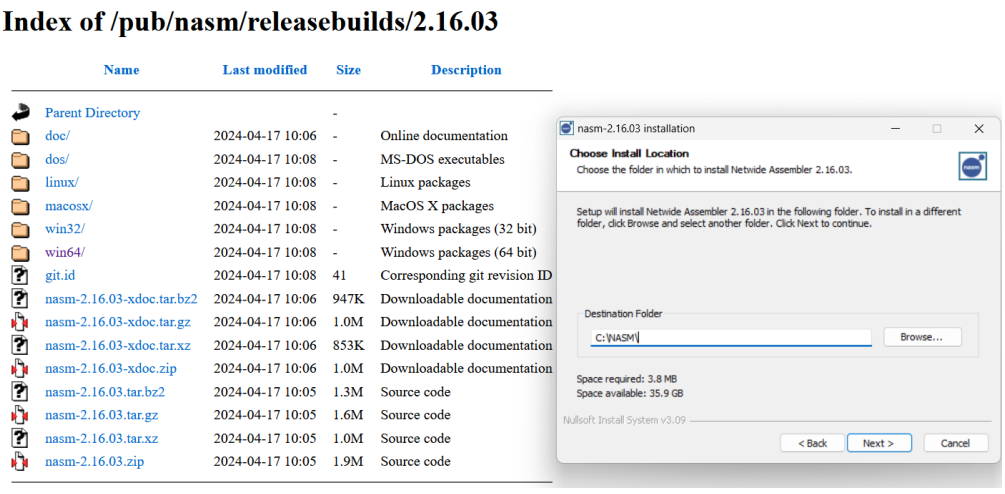


Figure 22: Descargar e instalar NASM en Windows

- **ACPI Source Language (ASL) Compiler**: EDK2 utiliza ACPI para definir la configuración de la plataforma y los métodos de gestión de energía en el sistema operativo. Estos métodos se describen utilizando ASL, requiriendo un compilador adicional (iasl.exe) que puede ser instalado descargando el paquete de herramientas binarias ACPICA de Windows. Para completar la instalación se debe colocar el contenido de las utilidades descomprimidas [iasl.exe, acpibin.exe, ...] en el directorio C:\ASL.

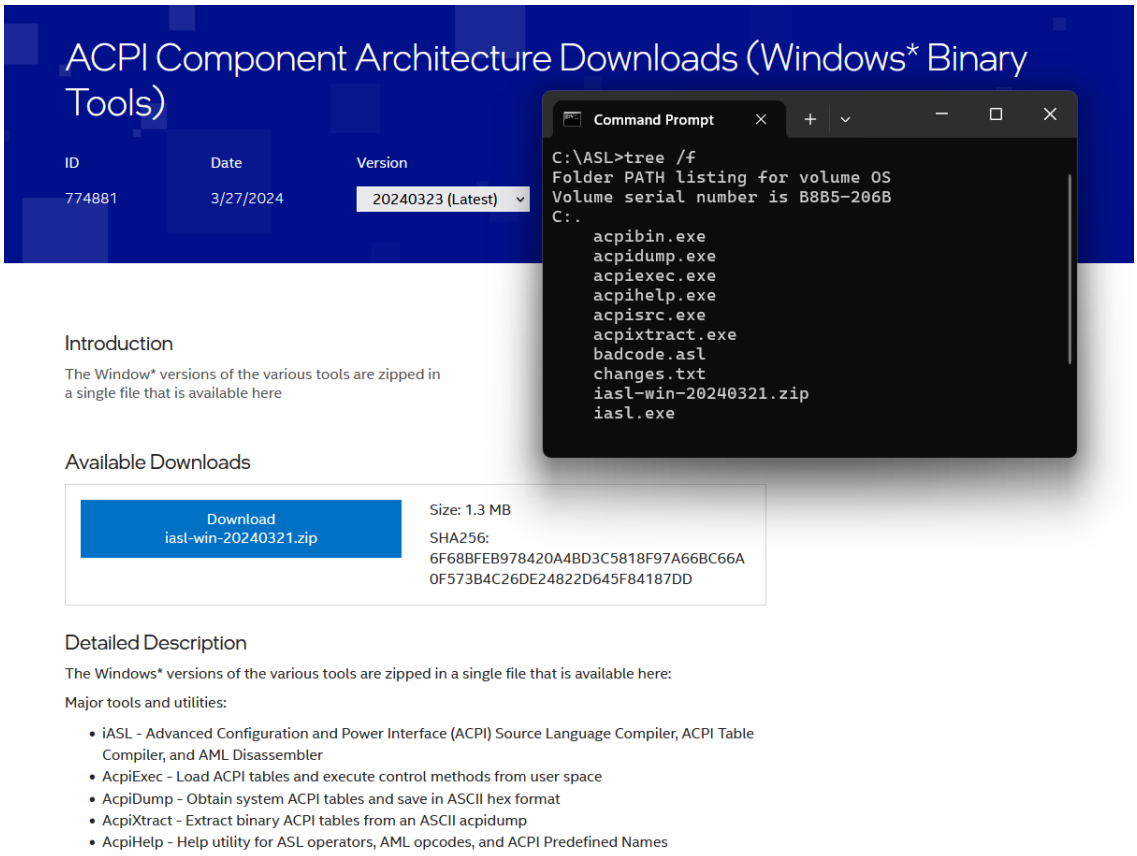


Figure 23: Descargar e instalar ASL en Windows

Con los requisitos previos ya instalados, es momento de proceder al verdadero despliegue del entorno de desarrollo, siendo el primer paso clonar el repositorio del proyecto EDK2, lo cual se puede llevar a cabo utilizando git en una línea de comandos. Para ello se debe emplear la opción de inicializar todos los submódulos necesarios, ya que en caso contrario se monstrarán errores cuando se estén ejecutando otras acciones posteriores, git clone https://github.com/tianocore/edk2 –recurse-submodules.

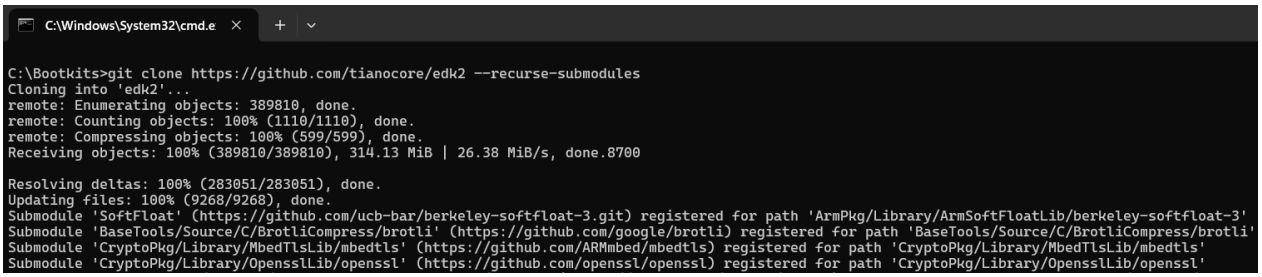


Figure 24: Clonar repositorio EDK2

Clonado el repositorio en nuestra máquina local, el siguiente paso es preparar y compilar las herramientas base, conocidas como BaseTools. Al ejecutar la siguiente instrucción "edksetup.bat Rebuild", en una línea de comandos o cmd sobre el directorio raíz de EDK2, se configura el entorno y reconstruyen las herramientas base, asegurando que todas las dependencias y módulos necesarios estén correctamente preparados para la compilación del firmware.

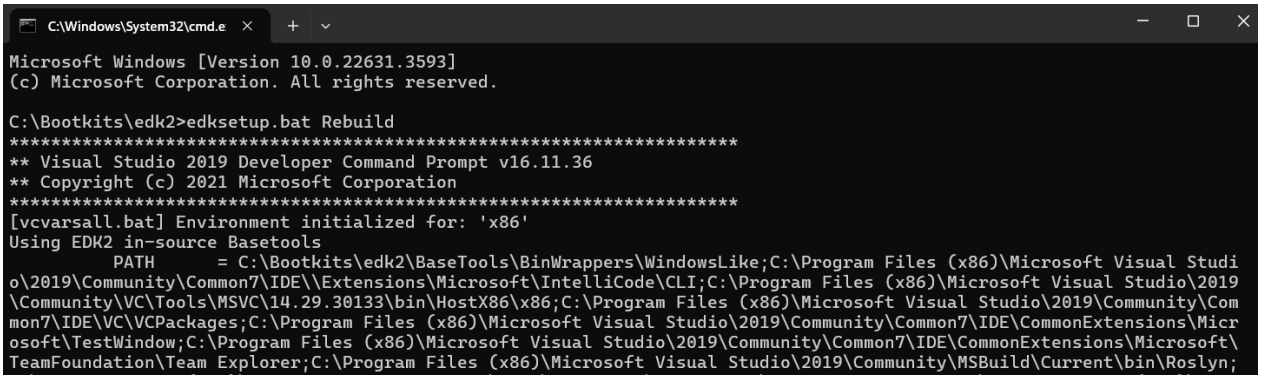


Figure 25: Configuración de herramientas Base de EDK2

En este punto ya podemos comenzar a compilar paquetes de librerías estándar y aplicaciones que ya nos ofrece EDK2, como el paquete MdeModulePkg, que contiene los módulos para cumplir con los estándares de la industria UEFI/PI y también las definiciones (incluyendo PPIs/PROTOCOLs/GUIDs y clases de biblioteca) e instancias de bibliotecas utilizadas para esos módulos. El último paso que debemos ejecutar, para comprobar que la compilación es exitosa, por facilidad de uso, es la modificación del fichero de configuraciones general, localizado en Conf\target.txt.

Dentro de este archivo, es esencial ajustar tres parámetros clave que influyen directamente en el proceso de compilación:

- **ACTIVE_PLATFORM**: Primero, hay que establecer la directiva ACTIVE_PLATFORM = MdeModulePkg/MdeModulePkg.dsc para dirigir el compilador específicamente hacia la compilación de ese paquete central, u otro de nuestro interés.
- **TOOL_CHAIN_TAG**: Seguidamente, es crucial ajustar TOOL_CHAIN_TAG según la cadena de herramientas instalada en el sistema, el IDE y versión, como VS2019 para Visual Studio 2019, garantizando así la compatibilidad y efectividad del proceso de compilación.
- **TARGET_ARCH**: Por último, la configuración de TARGET_ARCH debe reflejar la arquitectura del procesador para el que se desarrolla, como X64 o IA32, asegurando que el firmware compilado sea operativo en la máquina objetivo.

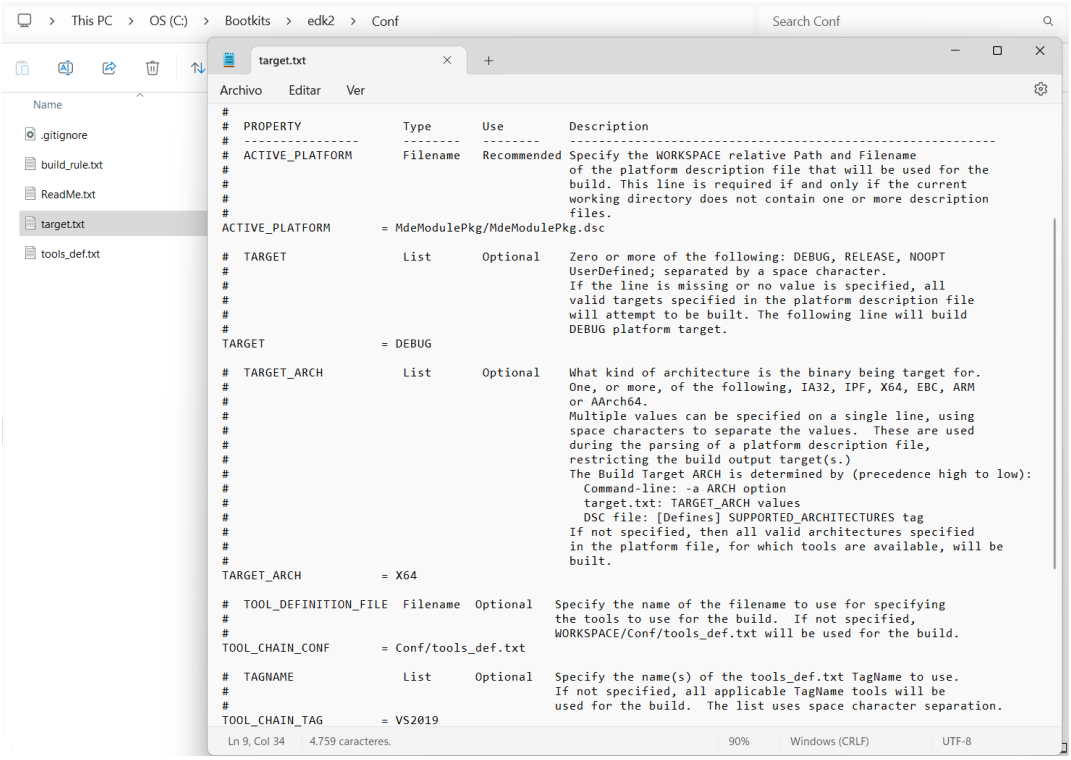


Figure 26: Modificar archivo de configuración de EDK2

Tras realizar esos ajustes, ejecutamos el comando 'build' en el directorio raíz de EDK2, compilando el paquete MdeModulePkg, resultando en la creación de varias aplicaciones básicas, como 'UEFI HelloWorld'.

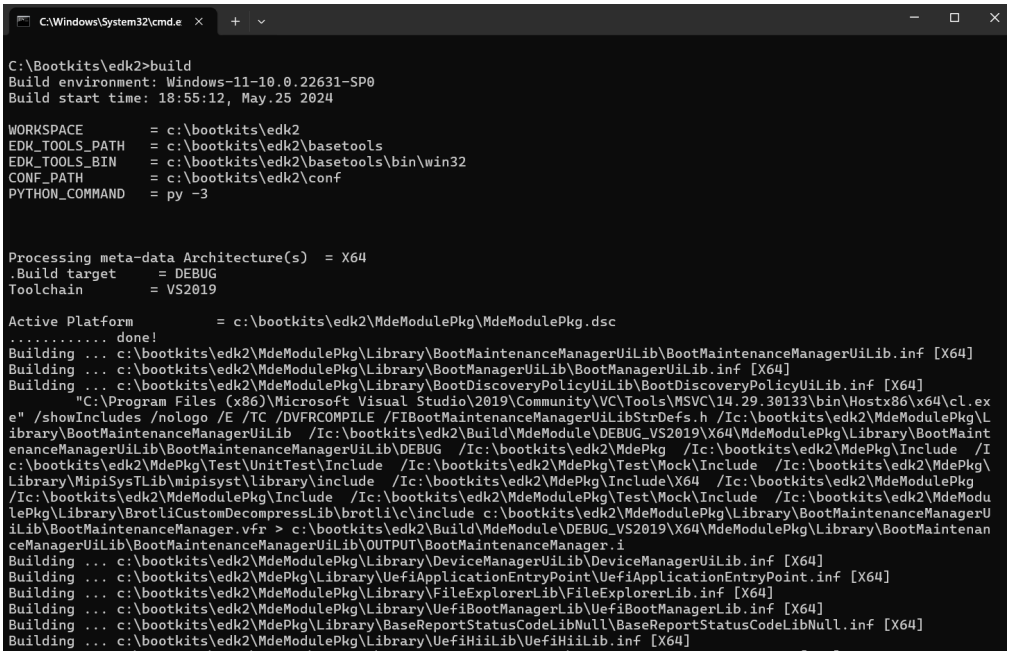


Figure 27: Compilación del paquete MdeModulePkg

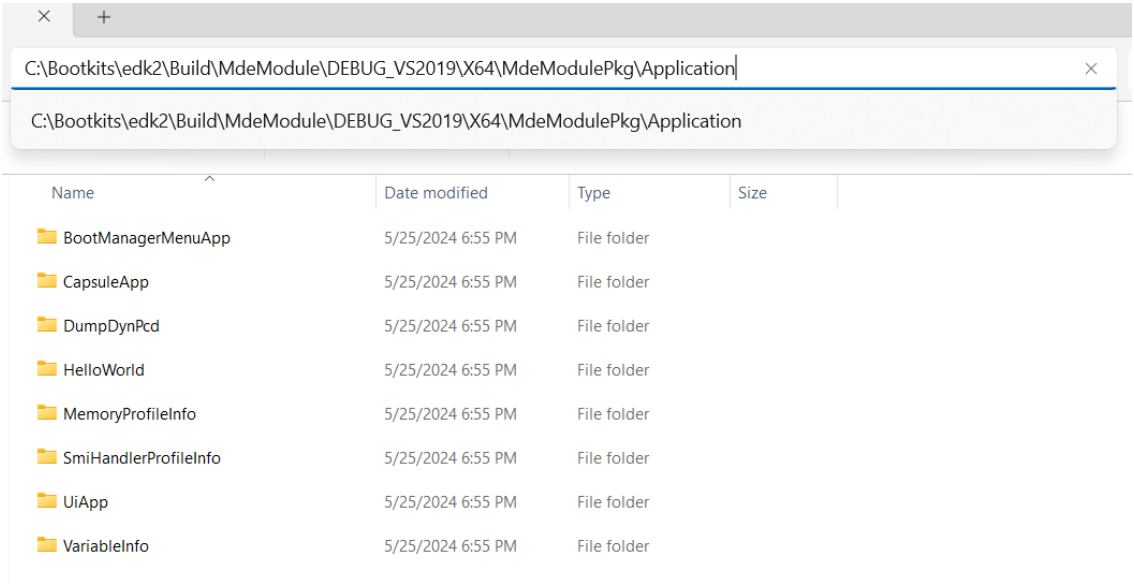


Figure 28: Listado de aplicaciones compiladas del paquete MdeModulePkg

Estas aplicaciones las podemos ejecutar de forma exitosa en una máquina con la arquitectura especificada al utilizar la shell UEFI, entrando en la bios y eligiendo esa opción.

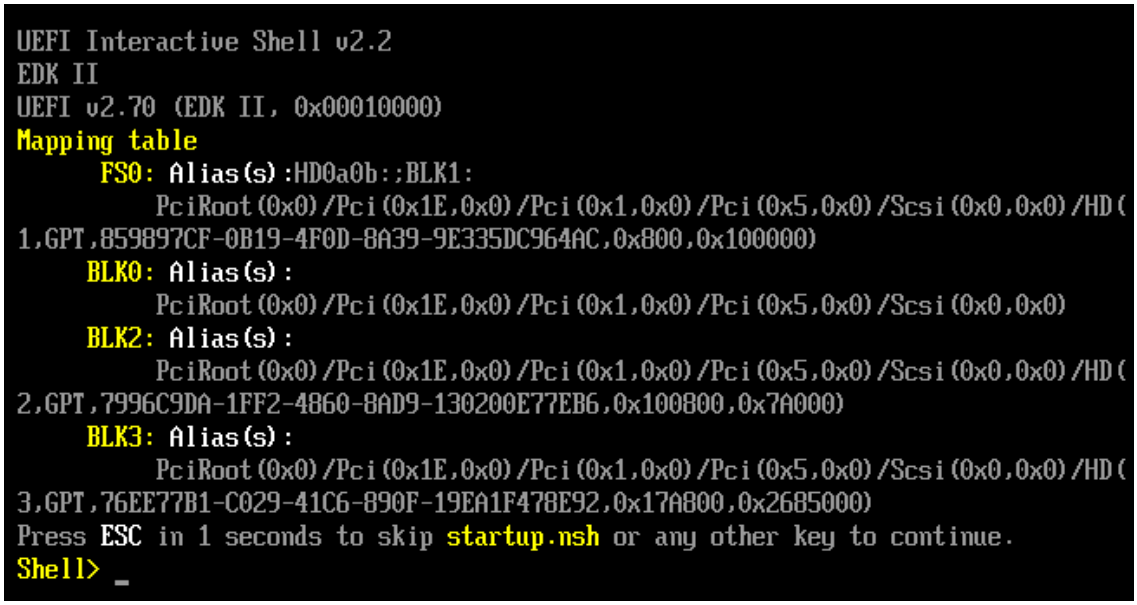


Figure 29: UEFI Shell

3.3.2. MI PRIMERA APLICACIÓN UEFI

Vamos ahora a desarrollar nuestra primera Aplicación UEFI, el elemento que se corresponde con el inicio de un bootkit UEFI. El primer paso es establecer un espacio organizado dentro del directorio principal de EDK2, creando una carpeta específica para nuestra aplicación, por ejemplo, BootkitPocUEFIApplication, actuando como contenedor donde guardar todos los archivos relacionados con el desarrollo.

Dentro de este subdirectorio, debemos crear un archivo de configuración INF llamado BootkitPocUEFIApplication.inf en el que definir las bases de la aplicación, siendo un manifiesto para EDK2 en el que se establece cómo tratar y compilar la aplicación.

```
## @file
#  Bootkit PoC, UEFI Application, INF
##

[Defines]
    INF_VERSION             = 1.25
    BASE_NAME               = BootkitPocUEFIApplication
    FILE_GUID               = #Copy and paste GUID from guidgen.com
    MODULE_TYPE             = UEFI_APPLICATION
    VERSION_STRING          = 1.0
    ENTRY_POINT             = UefiMain

[Sources]
    BootkitPocUEFIApplication.c

[Packages]
    MdePkg/MdePkg.dec

[LibraryClasses]
    UefiApplicationEntryPoint
    UefiLib

[Guids]

[Ppis]

[Protocols]

[FeaturePcd]

[Pcd]
```

Figure 30: Archivo de la aplicación UEFI (Configuración)

El núcleo de nuestra aplicación UEFI reside en su código fuente, que alojaremos en el archivo BootkitPocUEFIApplication.c. Este archivo debe incluir las bibliotecas UEFI necesarias, como Uefi.h y Library/UefiLib.h, y definir una función de entrada llamada UefiEntry, que será el punto de partida, donde inicializamos los servicios del sistema UEFI y podemos imprimir un mensaje inicial en la consola para fines de depuración o para proporcionar una interacción básica con el usuario.

```
/** @file
    Bootkit PoC, UEFI Application, C
**/

#include <Uefi.h>
#include <Library/UefiApplicationEntryPoint.h>
#include <Library/UefiLib.h>

/**
  as the real entry point for the application.

  @param[in] ImageHandle    The firmware allocated handle for the
                            EFI image.
  @param[in] SystemTable    A pointer to the EFI System Table.

  @retval EFI_SUCCESS       The entry point is executed successfully.
  @retval other             Some error occurs when executing
                            this entry point.

**/

EFI_STATUS
EFIAPI
UefiMain (
  IN     EFI_HANDLE          ImageHandle,
  IN     EFI_SYSTEM_TABLE    *SystemTable
  )
{
  Print(L"Bootkit␣PoC\n");
  return EFI_SUCCESS;
}
```

Figure 31: Archivo de la aplicación UEFI (Código Fuente)

Con el código fuente listo y el entorno preparado, procedemos a compilar nuestra aplicación, ejecutando el comando "edksetup.bat", que prepara todo el entorno de EDK2.

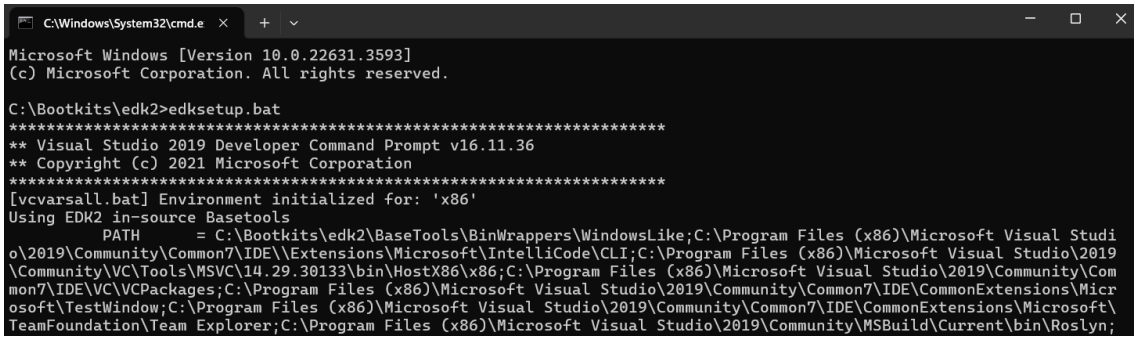


Figure 32: Configurar entorno de compilación

Luego, es crucial incluir la referencia de nuestra aplicación en la sección "Components" del descriptor de módulo, como MdeModulePkg/MdeModulePkg.dsc, asegurando que se compile junto con el paquete de módulos.

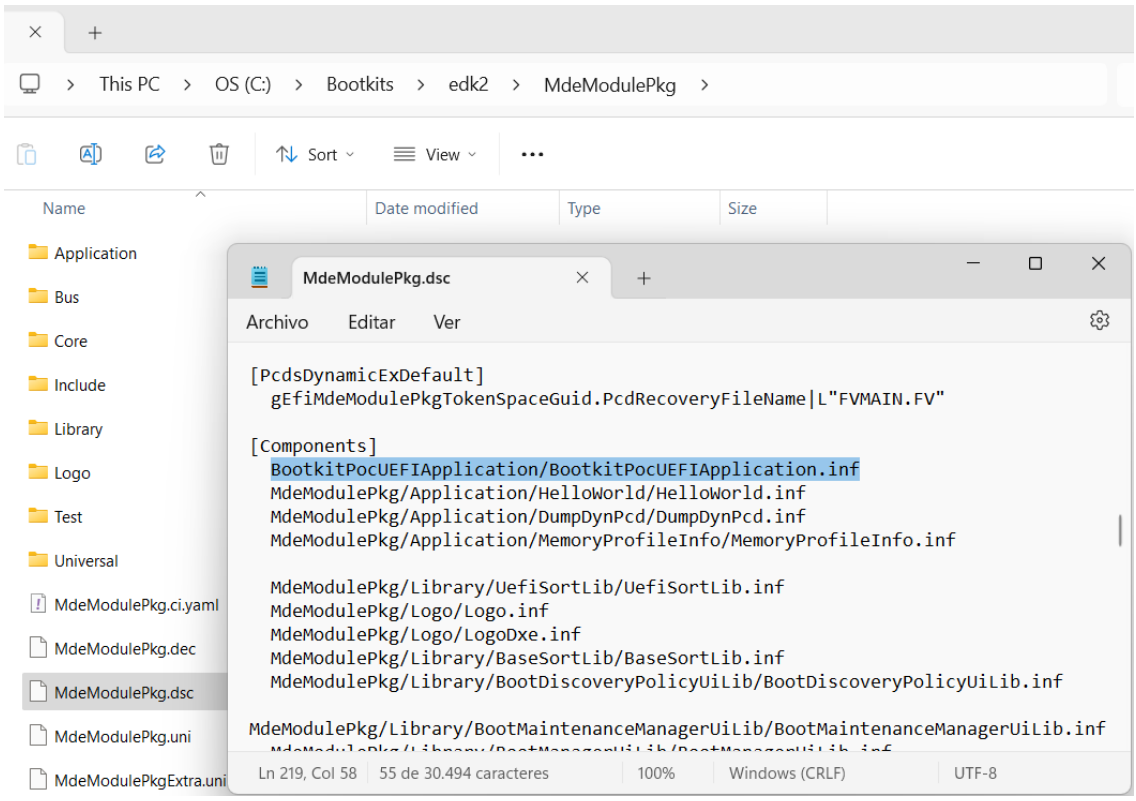


Figure 33: Incluir aplicación UEFI en paquete MdeModulePkg

Finalmente utilizamos el sistema de construcción de EDK2 con el comando "build" y el ejecutable resultante, BootkitPocUEFIApplication.efi, el principio de nuestro bootkit, ese elemento que substituirá al Windows Boot Manager, se encontrará en el directorio correspondiente bajo Build/MdeModule/DEBUG_GCC5/YOUR_ARCH.

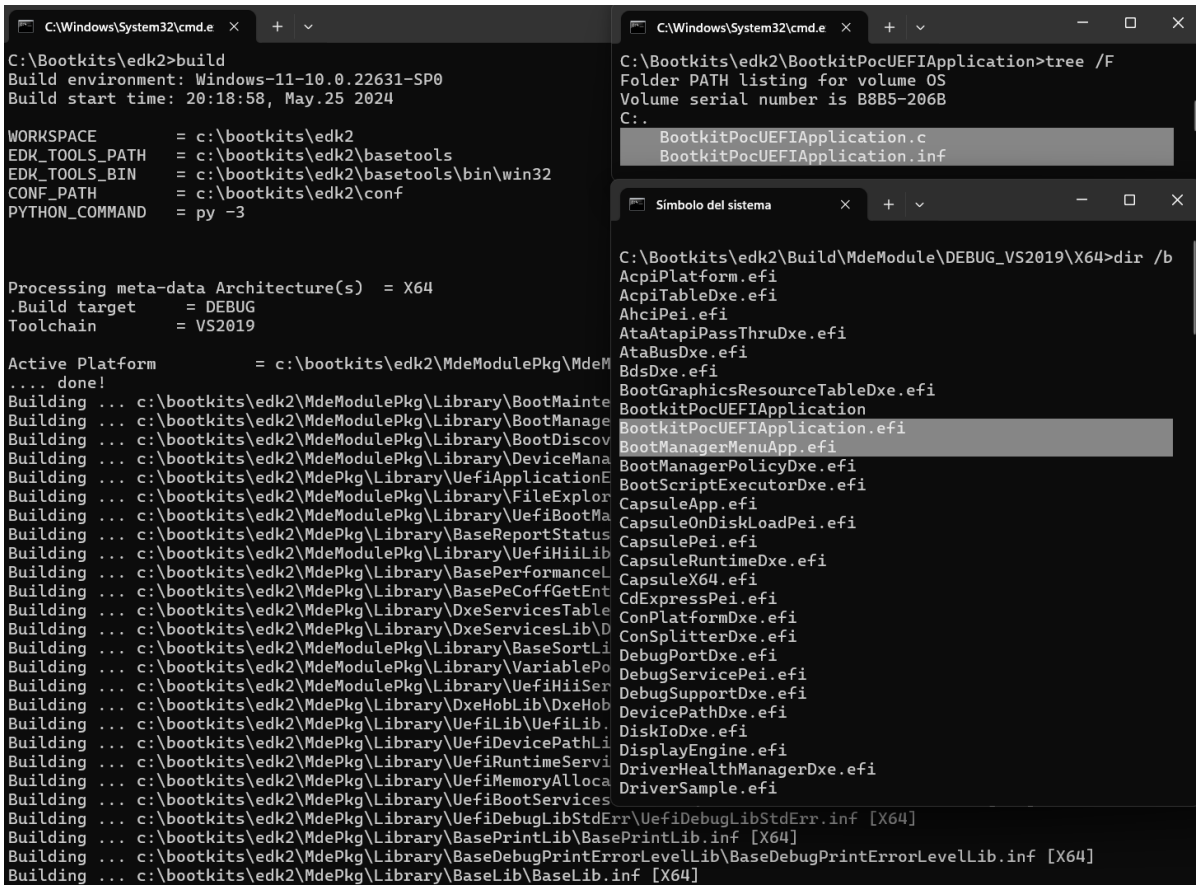


Figure 34: Aplicación UEFI compilada

3.3.3. DISEÑO DE BOOTKITS MODERNOS

Una vez que hemos cubierto cómo se desarrolla y prueba una aplicación UEFI, es esencial profundizar en un tema más avanzado y delicado: el Diseño de Bootkits modernos. Como hemos mencionado anteriormente, una aplicación UEFI puede considerarse el inicio de este malware, pero hay muchos elementos más y para desarrollarlos, un atacante, debe disponer de extensos conocimientos de sistemas, redes, hacking, ingeniería inversa, abuso de funciones de la especificación UEFI, y un muy largo etcétera de otras áreas relacionadas con la ciberseguridad.

UEFI Application	DXE Runtime Driver	Kernel Mode Driver	Client
UEFI Specification	Runtime Services	Hide Processes	IOCTLs
Boot Services	Hooking	Block Connections	IRPs
Protocols	Patching	Keyloggers	Communication

User-Mode Components	Persistence Tools	C&C Interface	Additional Malware
HTTP Downloader	Services	Data Encoding	Trojan
Cryptomining	Registry	Encrypted Channel	Spyware
Stealer	Com Hijacking	Protocol Tunneling	Ransomware

Figure 35: Componentes de un Bootkit moderno

Desarrollar un bootkit, o cualquier software avanzado que opere en el nivel de UEFI, requiere una comprensión profunda de la especificación, ya que en ese documento se detallan todos los aspectos de la interfaz, desde los servicios básicos que define hasta los complejos protocolos y drivers. Para los desarrolladores de malware interesados en crear un bootkit funcional, es indispensable familiarizarse con esta especificación persiguiendo desarrollar funcionalidades que permitan interactuar con el hardware y el firmware a un bajo nivel y asegurar que el bootkit pueda operar de manera eficiente y segura dentro de los límites del entorno UEFI.

Algunos de los aspectos más importantes que hay que dominar incluyen los servicios de boot (arranque), que están disponibles solo hasta que el sistema operativo toma el control, teniendo eso lugar cuando se hace la llamada a la función ExitBootServices(); los servicios de runtime (tiempo de ejecución), que están disponibles para ser consumidos durante todo el tiempo de ejecución, facilitando la interacción del sistema operativo con el firmware después del arranque; los aplicativos de tipo Drivers UEFI y Drivers DXE; y las funciones para gestionar todo tipo de protocolos. Estos elementos, que se describen en las miles de páginas que tiene esa especificación, son fundamentales para entender cómo desarrollar y operar software en el nivel de UEFI, incluyendo los bootkits.

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Un buen proyecto para que vosotros, los estudiantes, pongáis en práctica estos conceptos y desarrolléis un bootkit básico es Abismo. Este proyecto educativo, presentado durante Rooted Madrid 2024, está diseñado para introducir a estudiantes interesados en el desarrollo de software a nivel de firmware, dándoles la oportunidad de experimentar con los elementos de UEFI que hemos discutido hasta este punto. Abismo ofrece un entorno controlado donde los estudiantes pueden aprender sobre la manipulación de firmware y la creación de software de bajo nivel, combinando teoría y práctica de manera efectiva.

Trabajar en un proyecto como Abismo, después de realizar todos los módulos del máster, no solo ayuda a solidificar la comprensión teórica de los componentes y servicios de UEFI, sino que también brinda experiencia práctica en la manipulación de estos elementos para crear soluciones de software a nivel de sistema operativo y firmware, lo cual es fundamental para vuestro desarrollo como profesionales en el campo de la ciberseguridad y la ingeniería inversa de sistemas operativos Windows.

En el proyecto Abismo, el primer objetivo es estudiar la especificación UEFI y comprender los componentes clave que permiten la creación y modificación de aplicaciones de arranque, configurando, previamente, un entorno de desarrollo adecuado para trabajar. Una vez familiarizados con la especificación, viene el crear módulos básicos que interactúen con los servicios de arranque de UEFI. Esto incluye el desarrollo de funciones para la manipulación de la partición del sistema EFI (ESP). Y el último objetivo sería integrar esas técnicas mencionadas para modificar el proceso de arranque del sistema operativo, implementando mecanismos de persistencia, asegurando que el bootkit permanece activo tras varios reinicios de Windows.

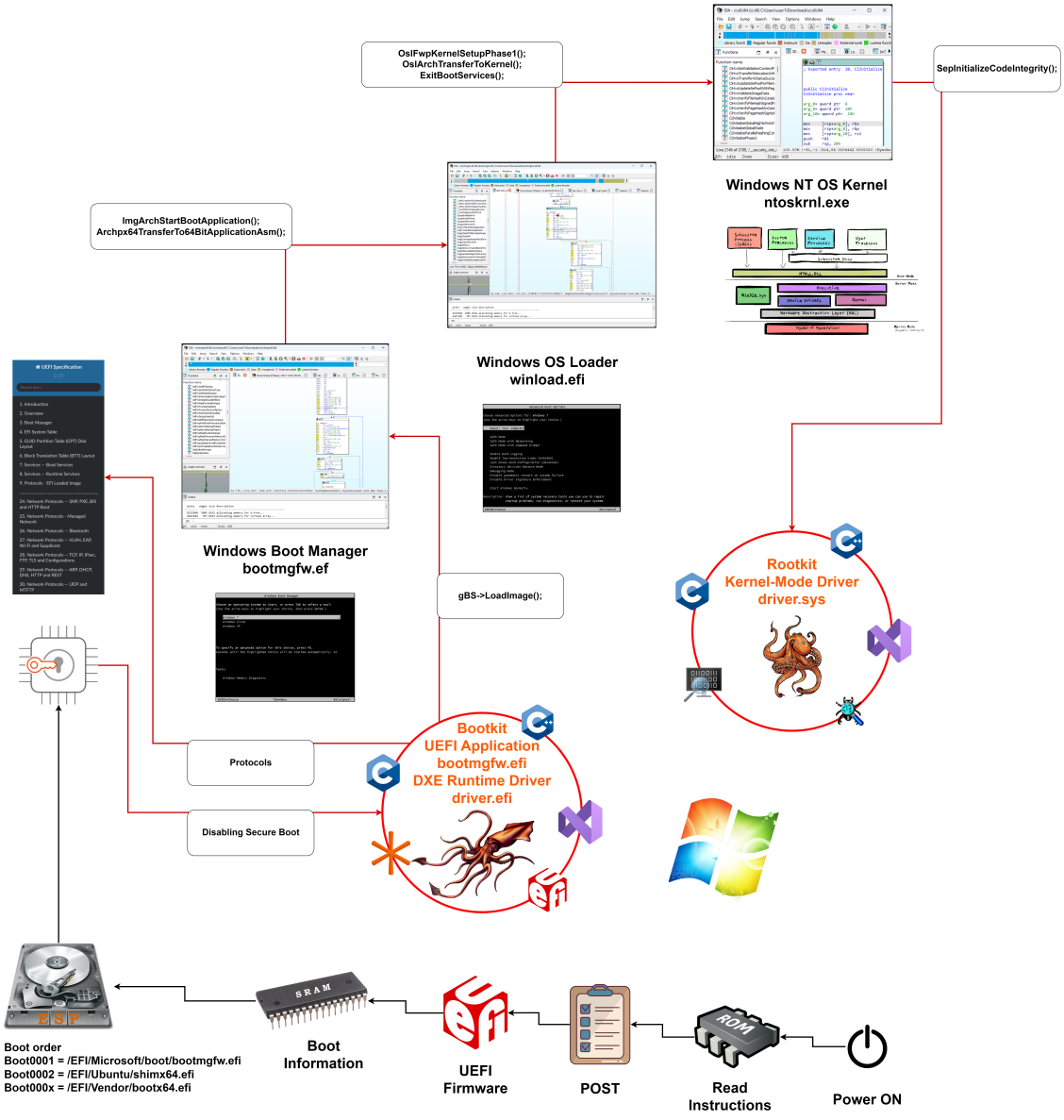


Figure 36: Proceso de infección de UEFI en el bootkit Abismo

4. MODO USUARIO Y MODO KERNEL

Visto ya el proceso de arranque, la teoría que gira en torno al mismo, y el malware orientado a infectarlo, los bootkits, vamos a introducir el tema principal en cuanto a arquitectura Windows respecta, y ver los dos modos de operación de este sistema operativo, modo usuario y modo kernel.

Uno (modo usuario) lo podemos concebir como el entorno en el que se ejecutan las aplicaciones y los programas del usuario, mientras que el otro (modo kernel) corresponde a la parte del sistema operativo que tiene acceso completo al hardware y a todas las funciones de bajo nivel.

En este punto, nos centraremos en ellos viendo cómo trabajan juntos para garantizar el correcto funcionamiento del sistema operativo en cuanto a gestión de recursos, procesos…, además de ver sus diferencias y particularidades.

Aunque incidiremos mucho más en todo lo relacionado con el modo kernel, ya que nos será necesario en los próximos temas, centrados en la aplicación práctica y la depuración de Windows.

4.1. ARQUITECTURA GENERAL DE UN SISTEMA WINDOWS

La arquitectura de Windows se puede entender como una estructura compleja de diferentes capas y componentes que interactúan entre sí, proporcionando una plataforma estable y fácil de usar para los usuarios y desarrolladores.

El adquirir un conocimiento general, pero amplio de esta arquitectura, es un paso fundamental para que posteriormente podamos hacer reversing del propio sistema operativo. Como podemos ver en la imagen, la arquitectura se subdivide en diferentes niveles:

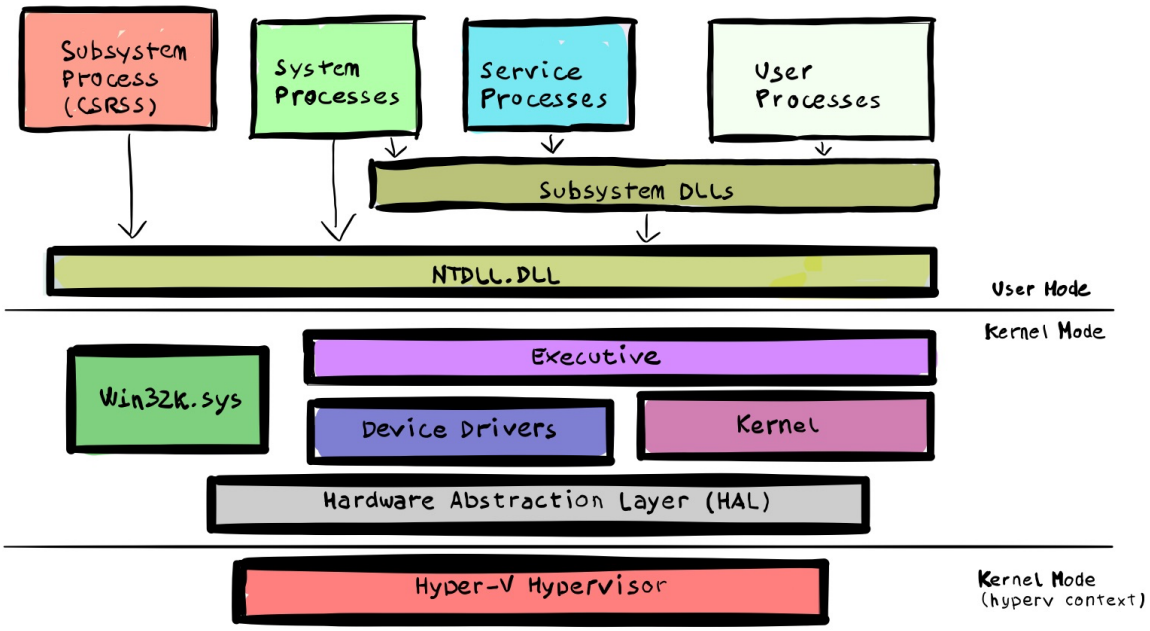


Figure 37: Arquitectura de un sistema Windows

En este caso, las líneas horizontales nos indican qué elementos se encuentran en modo usuario y cuales en modo kernel.

Dichos modos tienen diferentes niveles de privilegios, por definición, y se representan mediante el siguiente gráfico de anillos, en donde se ve que el modo usuario es el menos privilegiado (Ring 3) y el modo kernel cuenta con más privilegios (Ring 0):

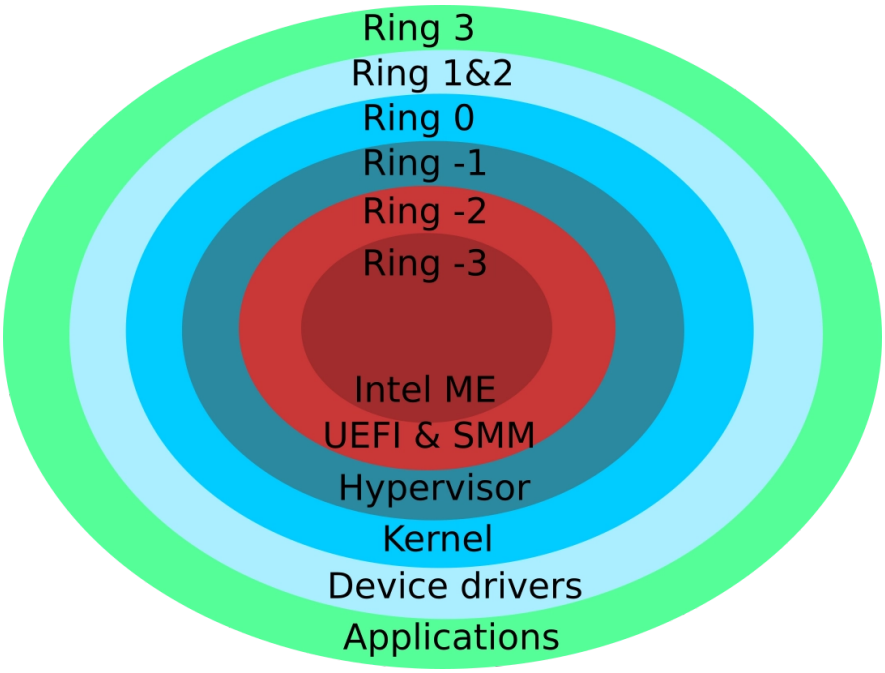


Figure 38: Niveles de Privilegios

Estos niveles de privilegio, conocidos como anillos, también forman parte de la arquitectura de Windows, donde cada anillo representa un nivel diferente de privilegios en cuanto al acceso al sistema.

Mediante este diseño, en el que se hace una protección por capas donde las aplicaciones de usuario, los drivers y el kernel tienen diferentes limitaciones en el acceso a recursos críticos de nuestra máquina, se previene que los programas o usuarios no autorizados realicen acciones que contravengan la seguridad y estabilidad del sistema operativo.

4.1.1. MODO USUARIO

Seguidamente, vamos a introducir los elementos que, como se ha visto en el esquema de arquitectura, forman parte del modo usuario.

4.1.1.1. SUBSISTEMA

Una vez que Windows se convirtió en un sistema operativo más maduro, Microsoft agregó nuevas características como el soporte para diferentes subsistemas, siendo los principales actualmente: el propio subsistema Windows, POSIX y OS/2.

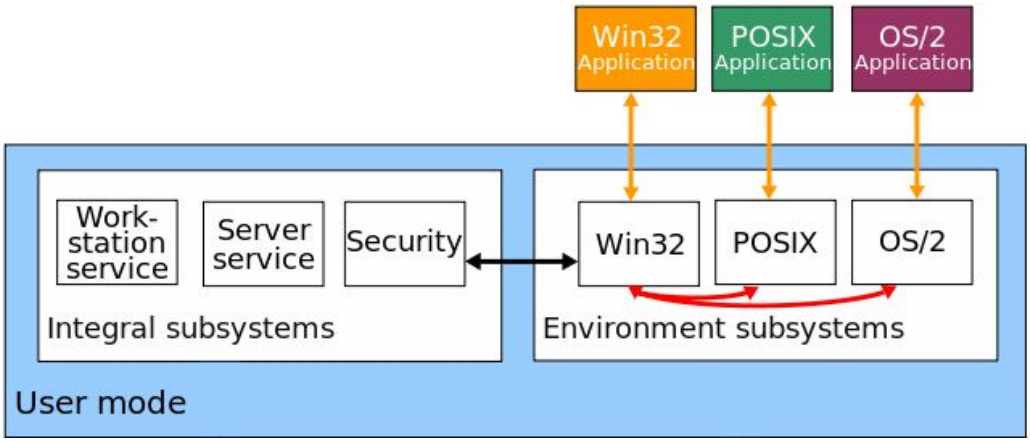


Figure 39: Subsistemas de Windows

Estos subsistemas son una parte fundamental del modo usuario, ya que se encargan de proporcionar una interfaz entre las aplicaciones y el sistema operativo. Además de añadir la posibilidad de ejecutar binarios no diseñados para Windows.

En concreto, los binarios de Windows cuentan con el campo "Subsystem" en su cabecera, campo en el que se define el subsistema al que pertenecen, y que el sistema operativo utiliza para saber cómo debe tratar a ese archivo ejecutable. Por ejemplo, si un binario tiene el valor "Console" en el campo "Subsystem", Windows proporcionará una interfaz de consola para esa aplicación.

Como vemos en la arquitectura, los subsistemas se apoyan en el binario csrss.exe (Client Server Runtime Subsystem), responsable de proporcionar una API para que las aplicaciones y otros procesos del sistema operativo interactúen con el subsistema.

Además, estos también se apoyan en una serie de dlls que sirven de interfaz hacia la tan importante NTDLL.dll, introducida más adelante. Estas dlls, cargadas por el proceso csrss.exe cuando se inicia el subsistema, le proporcionan funcionalidades adicionales.

4.1.1.2. PROCESOS DEL SISTEMA

En Windows existen varios procesos que son esenciales para el correcto funcionamiento del sistema operativo. Entre estos procesos, propios del S.O., podríamos mencionar Services.exe (proceso responsable de la gestión de los servicios), Smss.exe (proceso responsable de la gestión de las sesiones), Lsass.exe (proceso responsable de la gestión de la seguridad) o System (proceso responsable de la coordinación y gestión del resto de procesos).

De estos se destaca el proceso System, uno de los procesos más conocidos de Windows ya que cuenta con el famoso PID número 4 (el más bajo del sistema). Este proceso ejecuta el binario ntoskrnl.exe representando al propio kernel.

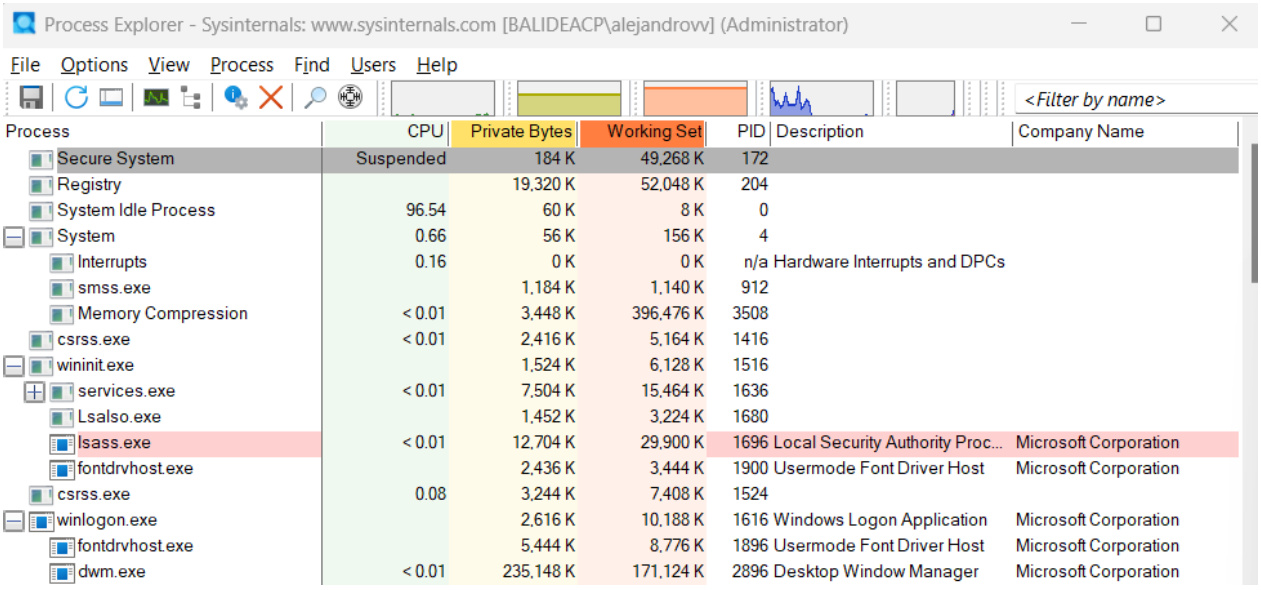


Figure 40: Monitorización de procesos con Process Explorer

4.1.1.3. SERVICIOS

Los servicios, son programas que se ejecutan en segundo plano y proporcionan funcionalidades a otros programas y al sistema operativo. Estos pueden configurarse para iniciar automáticamente en el arranque y, a diferencia de los procesos, carecen de interfaz gráfica, ejecutándose sin necesidad de que un usuario se autentique en el sistema.

Para nosotros estos elementos son muy importantes ya que, como comentaremos en próximos temas, los drivers que se ejecutan en modo kernel, pueden correr como un servicio de Windows. Para hacer esto, se puede utilizar la utilidad sc.exe que nos ayudará en la gestión y configuración del servicio:

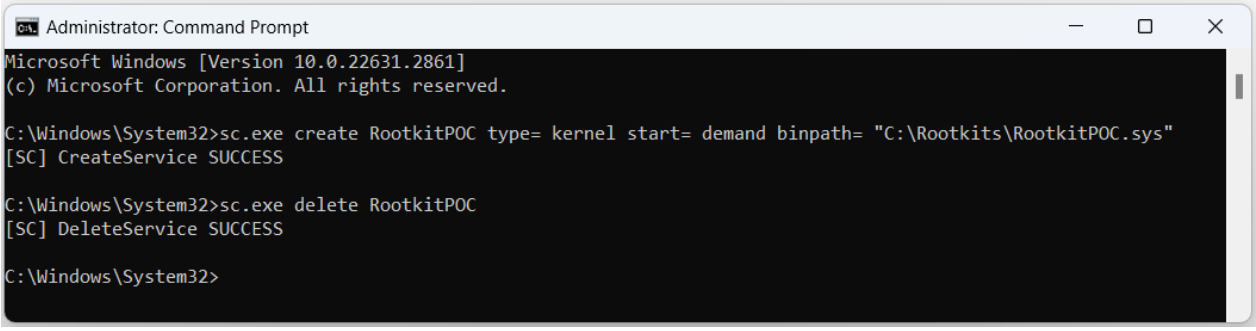


Figure 41: Creación de un servicio de tipo kernel

Como podéis ver en la imagen anterior, se ha creado un servicio para el driver, que se corresponde con una prueba de concepto de un rootkit. Para ello se han especificado tres parámetros:

- **binpath= "driver.sys"**: Es la ruta correspondiente al archivo .sys creado como consecuencia de compilar el código del driver.
- **type= kernel**: Indica que el tipo que se está cargando es un driver.
- **start= demand**: Indica que el servicio no se ejecuta de inmediato sino cuando el usuario lo especifique.

Al ejecutar el comando, se configura el servicio dejando todos los datos en la siguiente clave del registro de Windows "HKLM\System\CurrentControlSet\Services".

4.1.1.4. PROCESOS DE USUARIO

Estos procesos se refieren a las aplicaciones y programas propios o de terceros que se ejecutan en el contexto de un usuario en específico, y que no caen en el resto de clasificaciones vistas anteriormente.

Por ejemplo: explorer.exe, firefox.exe, etc.

4.1.1.5. NTDLL

NTDLL es una de las principales bibliotecas de Windows, puesto que proporciona una API para que las aplicaciones interactúen con el sistema operativo, y realicen tareas de bajo nivel como la gestión de archivos, procesos, etc.

NTDLL proporciona una amplia variedad de funciones, muchas de las cuales son utilizadas por programas maliciosos para ejecutar diversas operaciones, categorizadas como críticas, sobre un sistema operativo. Algunos ejemplos de funciones serían:

- **NtCreateFile**: Se utiliza para crear archivos.
- **NtLoadDriver**: Se puede utilizar para cargar un controlador en el equipo.
- **NtOpenProcessToken**: Se utiliza para obtener un token de acceso a un proceso, pudiendo realizar operaciones en el contexto de ese proceso.
- **NtCreateProcess**: Se utiliza para crear un nuevo proceso.

Estas funciones, son las que llaman al kernel a través de los mecanismos de syscall (llamadas al sistema) y sysenter (mecanismo de los procesadores Intel, más eficiente que syscall).

4.1.2. MODO KERNEL

En este punto, vamos a introducir los elementos del modo kernel, el modo más crítico para el sistema operativo ya que permite a los procesos interactuar directamente con el hardware pudiendo llevar a cabo las operaciones más fundamentales en dicho sistema.

4.1.2.1. EXECUTIVE

El Executive se corresponde con una capa que se encuentra por encima del kernel, la cual proporciona una variedad de servicios y herramientas para el resto del S.O. Dentro de este podemos destacar las siguientes secciones:

- **Gestión de la memoria**: El executive es responsable de administrar la memoria del sistema, lo que incluye llevar a cabo diferentes tareas como la paginación, la asignación y liberación de memoria a los procesos, etc.
- **Gestión de los procesos**: La creación, ejecución y finalización de los procesos del sistema son tareas que, aunque compartidas con el propio kernel, también realiza el executive, además de comprobar que estos tienen acceso a todos los recursos que necesitan para ejecutarse de forma correcta.
- **Gestión de entrada/salida**: Como papel crucial dentro del executive, se lleva a cabo la creación de paquetes de entrada/salida, los denominados IRPs (Input/Output Request Packets), que son enviados a su destino correspondiente. Este proceso de creación permite una comunicación efectiva entre los diferentes programas y drivers a los que se envían esos paquetes, generando en este sentido lo que denominaríamos una arquitectura de comunicación.
- **Gestión de la configuración**: Este elemento está estrechamente relacionada con el Registro de Windows (Windows Registry), ya que se encarga de gestionar la configuración del sistema operativo y de las aplicaciones que se ejecutan en él, incluyendo información sobre los dispositivos hardware instalados, los controladores, etc.
- **Funciones NT**: Cuando una aplicación llama a una función NT, es decir, una función de la biblioteca NTDLL.dll, se hace una llamada al sistema (syscall) que es traducida por el System Service Dispatcher (SSD), el cual enrutará dicha llamada a la implementación adecuada en el Executive, donde está realmente la implementación de esa función, transfiriendo el control al kernel del sistema operativo.

  Seguidamente, una vez se ejecuta en modo kernel la función NT equivalente del executive, el SSD traduce el resultado de vuelta a la aplicación en modo usuario.

- **Funciones ZW**: Se denominan funciones espejo y son equivalentes a las funciones NT, pero están diseñadas para ser utilizadas específicamente en el modo kernel. Estas funciones sirven como puntos de entrada a los servicios del sistema (Nt) y establecen el modo de acceso previo a kernel, eliminando la validación de parámetros en las funciones que solo ocurre si el modo previo es usuario.

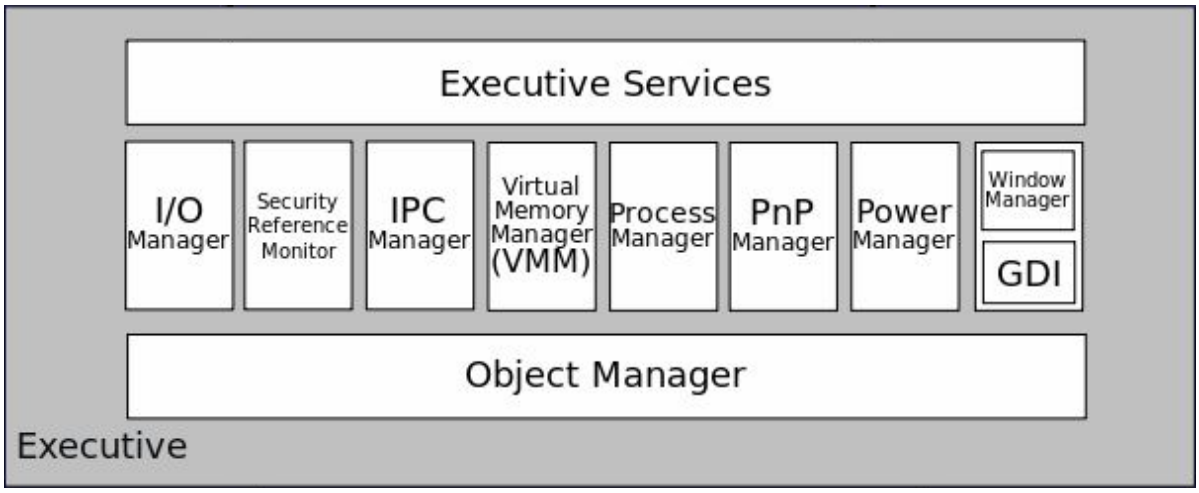


Figure 42: Servicios del Executive

4.1.2.2. KERNEL

El kernel, cuando nos referimos a la imagen de la arquitectura, es el elemento que se encuentra dentro del kernel de Windows, y que contiene la implementación de las operaciones más primitivas del sistema operativo.

Entre las tareas que lleva a cabo estarían, por ejemplo, la implementación del Scheduler, encargado de asignar los recursos del procesador a los distintos hilos que se encuentren en ejecución, o la gestión del ACPI (Advanced Configuration and Power Interface)

4.1.2.3. DRIVERS

Los drivers, en este contexto del modo kernel y aunque existen muchos tipos, de forma general se pueden ver como trozos de software que se ejecutan en Ring 0, con todas las posibilidades que eso ofrece en torno a acceso a recursos y privilegios.

Estos elementos del kernel, ya que son una parte indispensable para el módulo, los veremos exclusivamente y en detalle en próximos temas.

4.1.2.4. WIN32K

Win32k.sys es uno de los drivers del kernel más importantes para el sistema operativo, ya que proporciona soporte para la interfaz gráfica de usuario siendo responsable de la gestión de ventanas, por lo que cuando un programa necesita mostrar algo por pantalla, no se comunica directamente con el hardware, sino que lo hace a través de la interfaz gráfica, gestionada por Win32k.sys.

4.1.2.5. HAL

El HAL (Hardware Abstraction Layer) es una capa de abstracción de hardware que se encuentra entre el sistema operativo y los componentes físicos. Su función principal es proporcionar una interfaz para que el S.O., en este caso Windows, interactúe con el hardware, independientemente del fabricante y del modelo del equipo.

De esta forma, los drivers pueden hacer uso de las funciones del HAL, cuyo código se localiza en la librería hal.dll, sin tener que conocer las particularidades de cada máquina.

4.2. DIFERENCIAS ENTRE PROGRAMACIÓN EN MODO USUARIO Y MODO KERNEL

Dadas las diferencias entre el modo usuario y el modo kernel, siendo la principal el nivel de privilegios y el acceso a los recursos, hemos de tener en cuenta varios factores a la hora de desarrollar un programa en uno u otro modo o al hacer un análisis del mismo.

4.2.1. GESTIÓN DE LA MEMORIA

La gestión de la memoria en estos dos modos es diferente. En modo usuario cada aplicación se ejecuta en un espacio de memoria privado y virtual, separado del resto de aplicaciones, y, en consecuencia, no tienen acceso directo a la memoria.

El sistema operativo asigna un espacio virtual a cada proceso del modo usuario, adaptándose a las necesidades de cada uno. Esto quiere decir que, si una aplicación de usuario escribe en una determinada posición de memoria, y otra aplicación distinta escribe en esa misma dirección, los datos no se sobrescribirán porque esa dirección es virtual y única para cada proceso.

En contraposición, la gestión de la memoria en modo kernel es más compleja, ya que la memoria a la que se accede es compartida por todos los procesos del sistema. Si por ejemplo dos drivers escribiesen en la misma dirección de memoria, los datos se sobrescribirían ya que ambos sí que tendrían acceso a ese espacio.

Esto, que en parte es lo gracioso del modo kernel, es algo crítico y nos da paso a tratar el siguiente punto, la gestión de errores.

4.2.2. GESTIÓN DE ERRORES

La gestión de errores se convierte en una buena práctica al desarrollar aplicaciones de usuario, porque así contemplamos cualquier incidencia que pueda surgir durante el uso de esta, avisando al usuario o no de que algo ha fallado.

Si bien esta no es una tarea crítica en una aplicación de usuario que desarrollemos para nuestro equipo, en modo kernel cambia, ahora los errores de programación no solo afectan al proceso, sino a todo el sistema y pueden desencadenar la famosa Blue Screen of Death (BSOD).

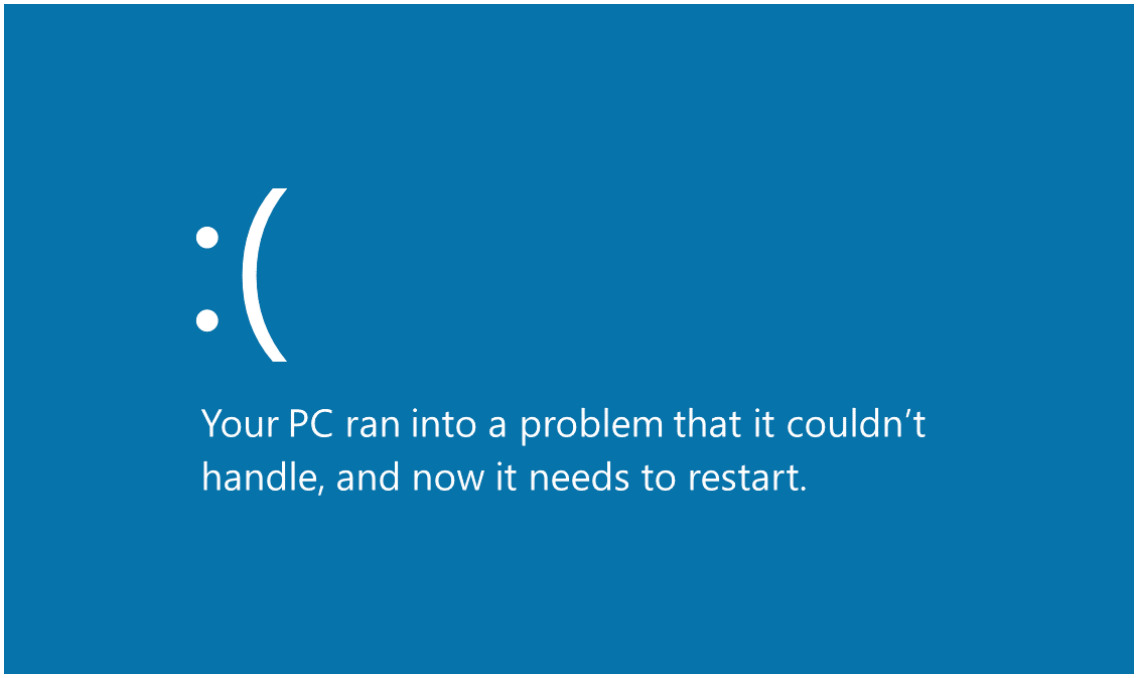


Figure 43: Blue Screen of Death

4.2.3. INTERRUPT REQUEST LEVEL (IRQL)

El término Interrupt Request Level o IRQL, se refiere a un mecanismo que define la prioridad de las solicitudes de acceso al hardware de nuestro equipo. Cada solicitud tiene una prioridad asignada, nivel IRQL, cuyo máximo define el procesador, y este se utiliza para decidir si un hilo puede ser interrumpido o no.

De esta forma se garantiza que los hilos con un mayor nivel, los cuales lleven a cabo tareas más críticas, puedan acceder a los recursos del sistema cuando lo necesiten.

En modo usuario los hilos siempre se ejecutan con un IRQL de valor 0, el nivel más bajo posible. Por lo que estos no pueden interrumpir a otros hilos y tienen que esperar a que el Scheduler, que ya mencionamos al ver el kernel, les asigne tiempo de CPU.

Lo importante aquí es que el nivel de IRQL del Scheduler es 2, por lo que todo código cuyo IRQL sea menor que 2 cumplirá con sus pautas, pero si el código que estamos ejecutando en el kernel tiene un valor igual o superior, puede llegar a producirse un bloqueo del sistema si este se ejecuta durante un tiempo elevado, no permitiendo la ejecución de otros hilos.

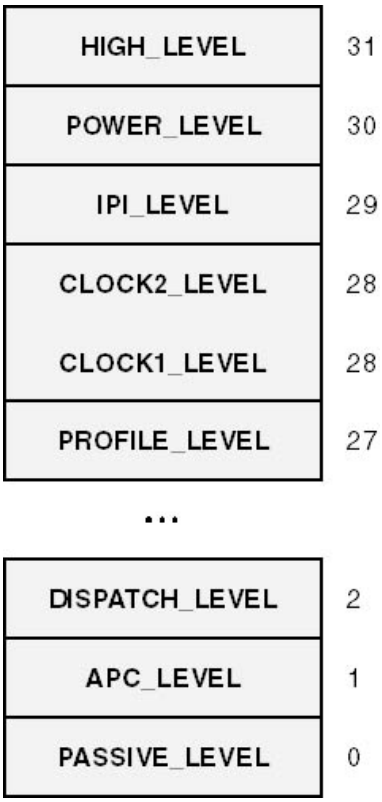


Figure 44: Interrupt request levels

4.3. ESTRUCTURAS HABITUALES EN MODO KERNEL

Al desarrollar o hacer reversing del código que se ejecuta en el kernel, hemos de tener siempre en mente diferentes estructuras de datos, ampliamente utilizadas:

4.3.1. UNICODE_STRING

Esta estructura de datos se utiliza constantemente en el modo kernel para representar cadenas de caracteres Unicode:

```
C++                                                    Copiar

typedef struct _UNICODE_STRING {
  USHORT Length;
  USHORT MaximumLength;
  PWSTR  Buffer;
} UNICODE_STRING, *PUNICODE_STRING;
```

Figure 45: Estructura UNICODE_STRING

Los campos de la estructura son los siguientes:

- **Length**: Es la longitud, en bytes, de la cadena de caracteres.
- **MaximumLength**: Es la longitud máxima, en bytes, de la memoria asignada al buffer.
- **Buffer**: Es el puntero al buffer que contiene la cadena que hayamos definido.

Esta estructura la veremos mucho como parámetro de entrada y salida, y para ayudarnos en la gestión de la misma, podemos hacer uso de la función "RtlInitUnicodeString", que nos permite inicializar una cadena UNICODE_STRING.

```
C++                                                    Copiar

NTSTATUS WdmlibRtlInitUnicodeStringEx(
  [out]          PUNICODE_STRING DestinationString,
  [in, optional] PCWSTR          SourceString
);
```

Figure 46: Función RtlInitUnicodeString

4.3.2. DRIVER_OBJECT

Esta estructura de datos, representa la imagen de un driver de modo kernel cargado y contiene la información importante del mismo:

C++ Copiar

```
typedef struct _DRIVER_OBJECT {
  CSHORT             Type;
  CSHORT             Size;
  PDEVICE_OBJECT     DeviceObject;
  ULONG              Flags;
  PVOID              DriverStart;
  ULONG              DriverSize;
  PVOID              DriverSection;
  PDRIVER_EXTENSION  DriverExtension;
  UNICODE_STRING     DriverName;
  PUNICODE_STRING    HardwareDatabase;
  PFAST_IO_DISPATCH  FastIoDispatch;
  PDRIVER_INITIALIZE DriverInit;
  PDRIVER_STARTIO    DriverStartIo;
  PDRIVER_UNLOAD     DriverUnload;
  PDRIVER_DISPATCH   MajorFunction[IRP_MJ_MAXIMUM_FUNCTION + 1];
} DRIVER_OBJECT, *PDRIVER_OBJECT;
```

Figure 47: Estructura DRIVER_OBJECT

Dentro de los múltiples campos de esta estructura, podemos destacar los siguientes:

- **DriverStart**: Este campo es un puntero a la dirección de inicio del código ejecutable del driver.
- **DriverUnload**: Se corresponde con la función que se llama cuando el driver ya no es necesario, y el sistema operativo lo "descarga" de la memoria.
- **MajorFunction**: Es un vector de punteros donde cada elemento corresponde a una función específica que se encarga de manejar una solicitud de Entrada/Salida.

4.3.3. LIST_ENTRY

Esta estructura de datos, utilizada como parte de una lista doblemente enlazada, representa a un elemento/nodo de dicha lista, donde cada uno de estos contiene dos campos, que hacen referencia al nodo anterior y siguiente:

```
C++                                                    Copiar

typedef struct _LIST_ENTRY {
  struct _LIST_ENTRY *Flink;
  struct _LIST_ENTRY *Blink;
} LIST_ENTRY, *PLIST_ENTRY, PRLIST_ENTRY;
```

Figure 48: Estructura LIST_ENTRY

Gráficamente, esta lista se puede ver de la siguiente forma:

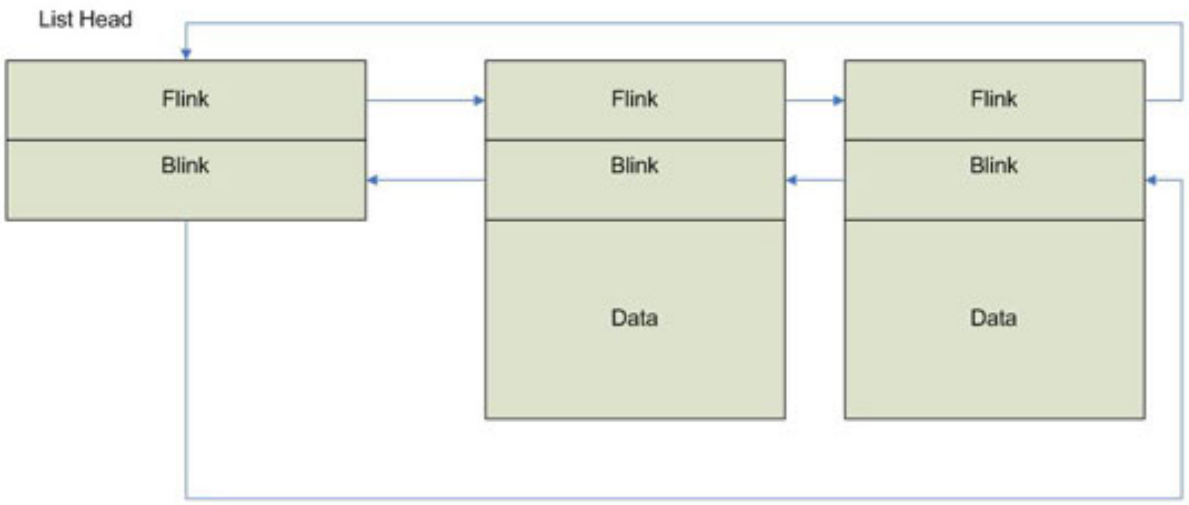


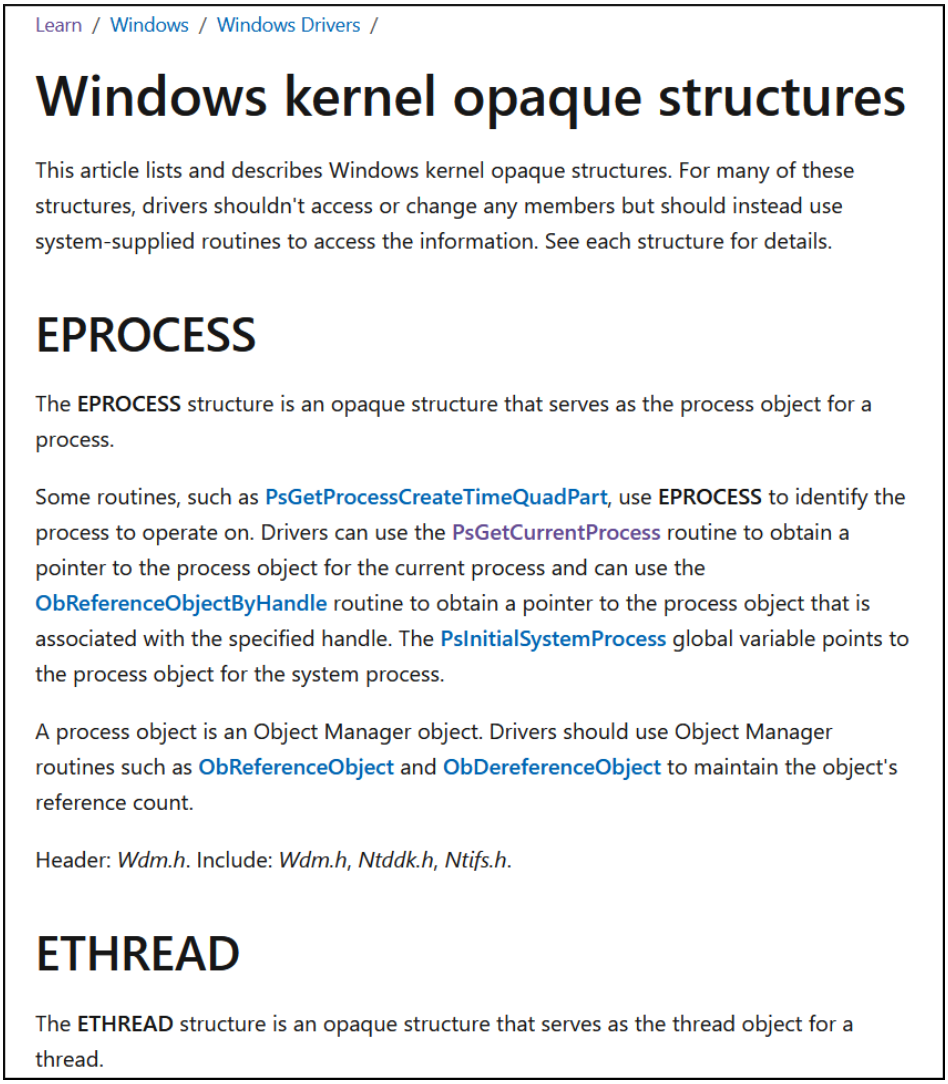
Figure 49: Lista doblemente enlazada

Cada nodo de la lista consta de las siguientes partes:

- **Flink**: Contiene un puntero al siguiente elemento de la lista.
- **Blink**: Contiene un puntero al elemento anterior de la lista.
- **Data**: Contiene los datos en sí.

4.3.4. ESTRUCTURAS OPACAS DEL KERNEL

A menudo, cuando hacemos reversing o analizamos malware nos encontraremos con estructuras que no están documentadas, de echo Microsoft dedica un artículo a este tipo de estructuras "opacas", cuyo uso no se recomienda al llevar a cabo un desarrollo normal, pero sí que veremos como parte fundamental del kernel.



Learn / Windows / Windows Drivers /

Windows kernel opaque structures

This article lists and describes Windows kernel opaque structures. For many of these structures, drivers shouldn't access or change any members but should instead use system-supplied routines to access the information. See each structure for details.

EPROCESS

The **EPROCESS** structure is an opaque structure that serves as the process object for a process.

Some routines, such as PsGetProcessCreateTimeQuadPart, use **EPROCESS** to identify the process to operate on. Drivers can use the PsGetCurrentProcess routine to obtain a pointer to the process object for the current process and can use the ObReferenceObjectByHandle routine to obtain a pointer to the process object that is associated with the specified handle. The PsInitialSystemProcess global variable points to the process object for the system process.

A process object is an Object Manager object. Drivers should use Object Manager routines such as ObReferenceObject and ObDereferenceObject to maintain the object's reference count.

Header: *Wdm.h*. Include: *Wdm.h*, *Ntddk.h*, *Ntifs.h*.

ETHREAD

The **ETHREAD** structure is an opaque structure that serves as the thread object for a thread.

Figure 50: Estructuras opacas del kernel de Windows

Para confrontar esta situación, al toparnos con una estructura o función de la que no existe documentación oficial, tendremos que encontrar la forma de indagar en ella y conocer qué acción se está llevando a cabo, buscando en repositorios orientados a centralizar dicha información y realizando un proceso de depuración/debugging en el sistema operativo.

4.3.4.1. RECURSOS EN INTERNET

En primer lugar, y como a lo largo de los años se ha centralizado en varios sitios web la información referente a estas estructuras, ya fuese gracias al análisis de algunos investigadores independientes, filtraciones, etc, se debe acudir a estos recursos públicos en internet pues es la forma más sencilla de tener un punto de partida.

Como ejemplo tendríamos, EPROCESS, ya conocida por ser la estructura del kernel que representa a un proceso.

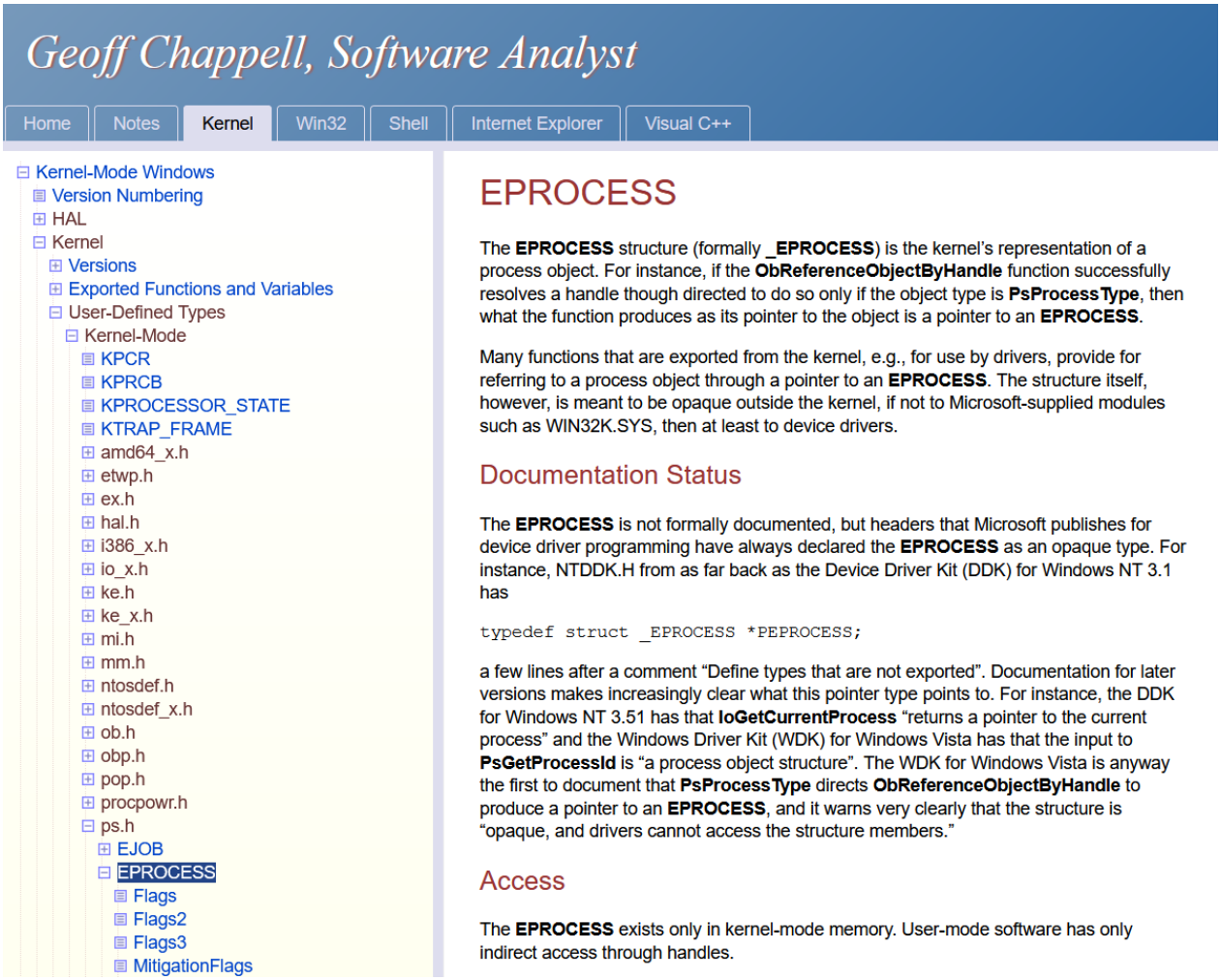


Figure 51: Información sobre EPROCESS en geoffchappell

Esta, como veis, es una de las páginas en las que podemos acceder a información más detallada como los cambios sufridos entre diferentes versiones, en cuanto al tamaño y los offsets o desplazamientos de cada campo dentro de la estructura:

Version	Size (x86)	Size (x64)
2004	0x04C0	0x0A40
1903	0x0480	0x0880
1809	0x0408	0x0850

Figure 52: Tamaños de la estructura EPROCESS

Figure 53: Offsets de la estructura EPROCESS en Vergilius

4.3.4.2. WINDBG

Otra de las formas para encontrar información sobre estas estructuras es hacer reversing del sistema operativo utilizando WinDbg, una herramienta que introducimos en detalle en temas posteriores y con la cual podemos consultar todo tipo de estructuras y elementos del kernel, en este caso concreto la estructura EPROCESS del proceso mspaint.exe.

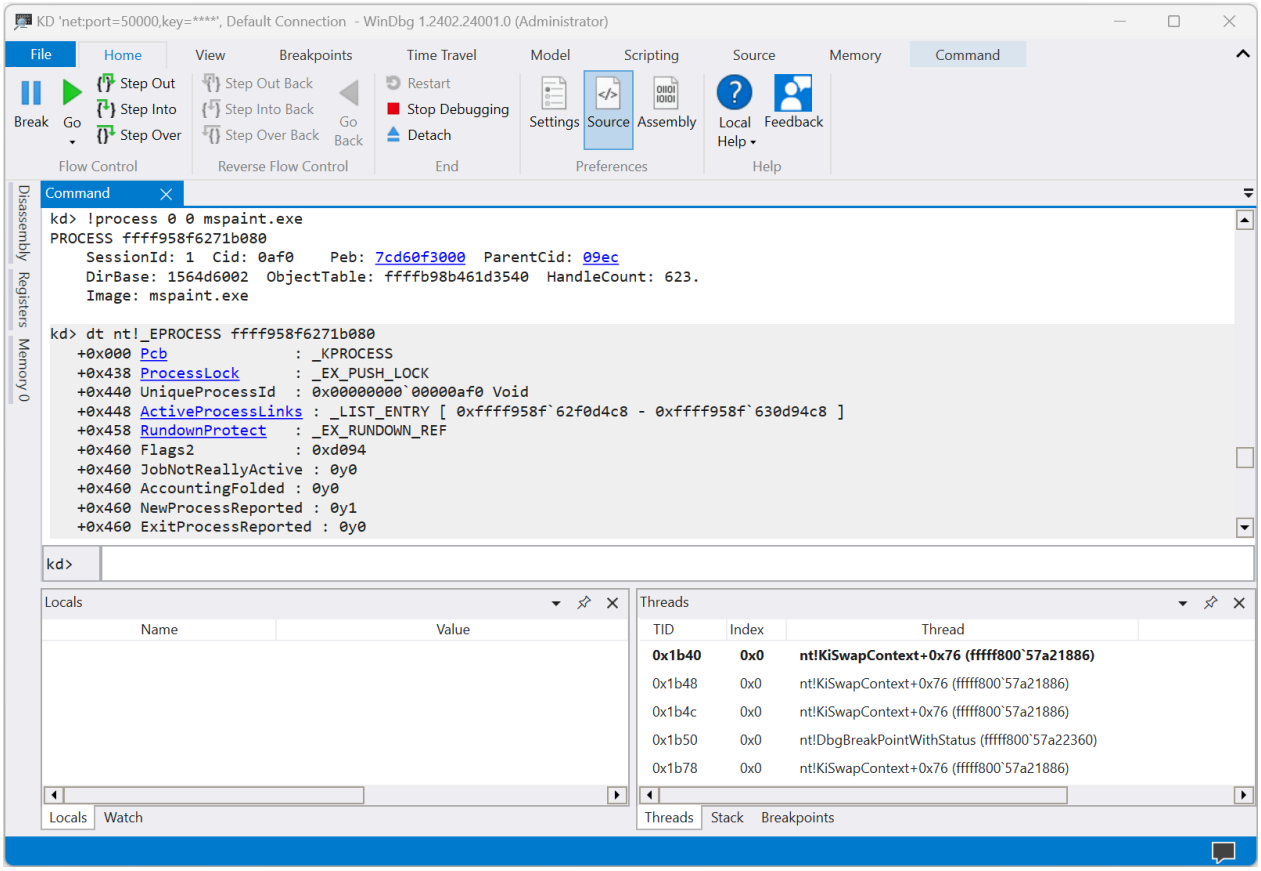


Figure 54: Consultar estructura EPROCESS del proceso mspaint.exe mediante WinDbg

4.3.5. PREFIJOS COMUNES

Al examinar ciertas funciones y estructuras del kernel, frecuentemente se puede relacionar el nombre del elemento con su funcionalidad, y, además, al hacer el ejercicio de listar funciones documentadas o no de librerías del sistema como ntdll.dll, kenerl32.dll o similares, se puede establecer la siguiente relación en cuanto a la convención de la nomenclatura, utilizada para nombrar esas rutinas del sistema de Windows. Sin embargo, este formato general puede incluir variaciones, por ejemplo, la letra "i" seguida del prefijo indicaría una función interna y la letra "p" indicaría una función privada.

<Prefijo><Operación><Objeto>

En este formato, el prefijo es el componente interno que exporta la rutina, la operación indica qué se está haciendo con el objeto o recurso, y el objeto identifica sobre qué se está operando. Seguidamente se listan algunos de los prefijos de mayor importancia para este módulo, encontrándose la lista completa en la sección oportuna del libro, mencionado en la bibliografía, "Windows Internals":

Prefijo	Componente
Cm	Gestor de configuración (Registro de Windows)
Dbg	Soporte para depuración del kernel
Ex	Executive
Io	Gestor de entrada/salida
Kd	Depuración del kernel
Ke	Funciones del propio kernel
Lsa	Seguridad local
Mm	Gestor de memoria
Ps	Soporte a procesos
Zw	Funciones espejo de Nt

Como se puede observar, con esta convención se descifran más fácilmente y comprenden mejor las funciones internas del sistema operativo, así como para qué se utilizan. Aunque es importante tener en cuenta que muchas funciones y estructuras no están documentadas pudiendo cambiar a lo largo del tiempo, por ello, como se comentó anteriormente, no se recomienda utilizarlas directamente siendo parte del desarrollo normal de un aplicativo.

5. MECANISMOS DE PROTECCIÓN DEL SISTEMA OPERATIVO

En este apartado, veremos qué tipo de medidas de seguridad se han añadido en el S.O. Windows en los últimos años, las cuales tendremos que tener en cuenta a la hora de realizar nuestros análisis, desarrollos y operaciones de reversing.

5.1. DRIVER SIGNATURE ENFORCEMENT (DSE)

Esta característica de seguridad, la cual es parte importante al tratar el tema de desarrollo de drivers, hace referencia a que Windows requiere que los drivers (controladores de dispositivos) estén firmados digitalmente por un certificado de confianza, antes de ser instalados en nuestro equipo.

De esta forma, el sistema operativo verifica la autenticidad e integridad de los controladores antes de cargarlos, y si estos no están firmados por un certificado confiable para Microsoft, Windows bloqueará la instalación y mostrará un mensaje de error.

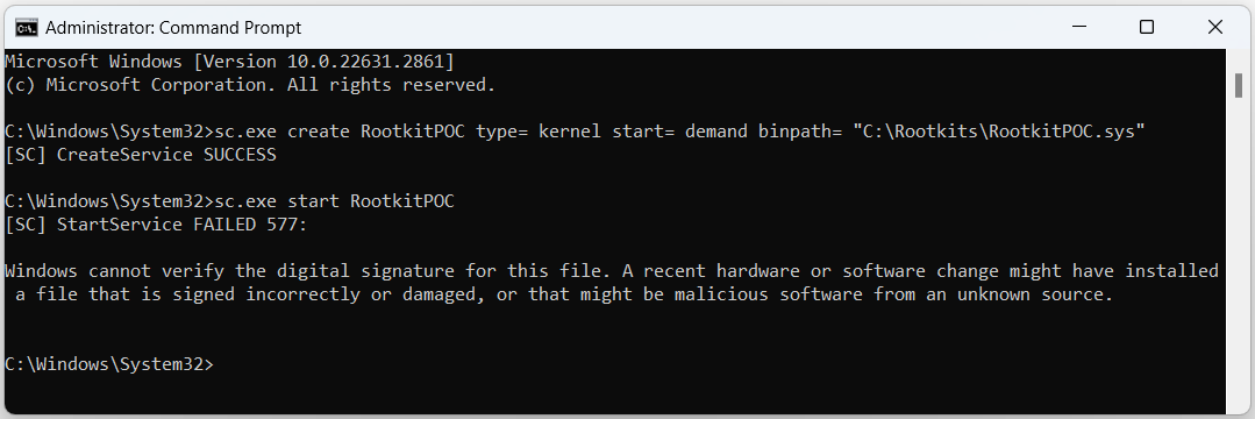


Figure 55: Error mostrado por DSE al tratar de ejecutar un driver no confiable

Sin embargo, puede ocurrir que algún driver no esté firmado, bien por su antigüedad o porque el proveedor no sea de confianza, y que nosotros queramos instalarlo igualmente, teniendo que evadir dicha protección. Como esta situación llega a ser común, existe la posibilidad de deshabilitar temporalmente la característica DSE de una forma sencilla, a través de las opciones avanzadas en el inicio.

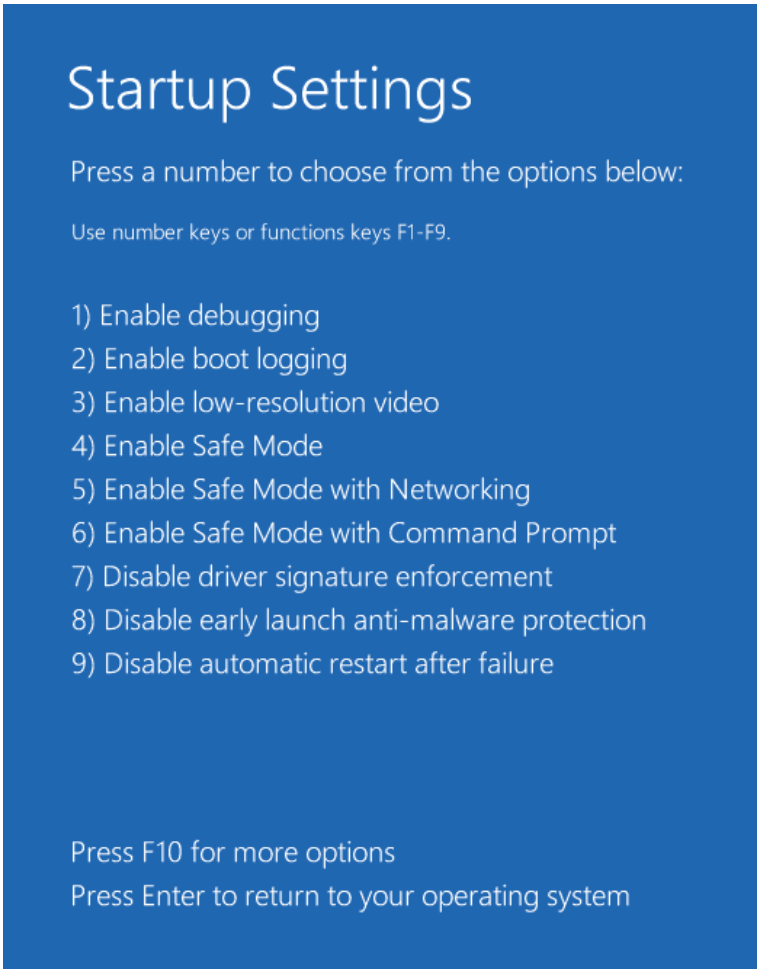


Figure 56: Opciones avanzadas del inicio

Para nuestro caso, donde buscamos que nos sea sencillo trabajar en el entorno de laboratorio que necesitamos crear para el módulo, es recomendable deshabilitar la característica de forma permanente y así no tener que reiniciar el equipo o máquina virtual cada vez que queramos hacer pruebas.

Esto se puede llevar a cabo de dos formas haciendo uso de la herramienta bcdedit.exe, una utilidad de la línea de comandos que sirve para administrar almacenes BCD, modificando opciones del arranque:

- **Deshabilitar firma de controladores**: Al ejecutar el comando "bcdedit.exe /set nointegritychecks on" en una línea de comandos o CMD con permisos de administrador, se deshabilita automáticamente la protección Driver Signature Enforcement (DSE).

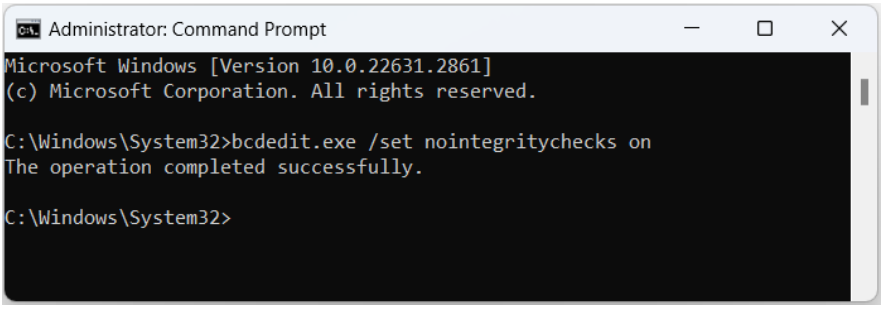


Figure 57: Deshabilitar firma de controladores en Windows

- **Habilitar modo test**: El modo de prueba de Windows nos permite instalar y probar controladores no firmados (es decir, no aprobados por Microsoft aún) y que con DSE no podrían ejecutarse en condiciones normales. Cuando se habilita este modo, al ejecutar el comando "bcdedit.exe /set testsigning on" en una línea de comandos o CMD con permisos de administrador, podremos ver una marca de agua en la esquina inferior derecha de la pantalla, mostrándonos el mensaje "Test Mode".

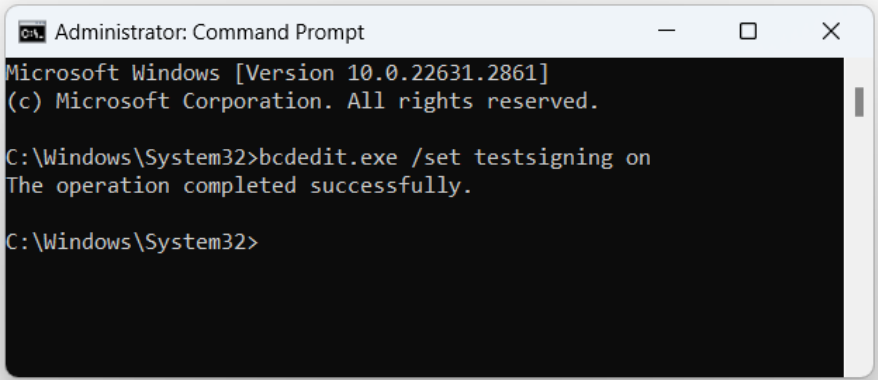


Figure 58: Habilitar modo test de Windows

5.2. PROCESOS PROTEGIDOS

A partir de Windows Vista y para mejorar la seguridad del sistema operativo, limitando los privilegios que los programas en ejecución tenían sobre otros procesos, se incluyó este nuevo concepto "Procesos Protegidos". Básicamente lo que se consigue, es que, sobre este tipo de proceso, otros no puedan realizar ciertas operaciones.

Este mecanismo de seguridad, añade un nuevo escenario en relación al malware, ya que la limitación de privilegios impide que un programa malicioso dañe el sistema o acceda a información considerada sensible, aun ejecutándose con privilegios de administrador.

Entre los procesos que en las versiones actuales de Windows se presentan como protegidos, nos encontramos los siguientes ya comentados en temas anteriores:

- Lsass.exe
- Smss.exe
- Csrss.exe
- Services.exe
- …

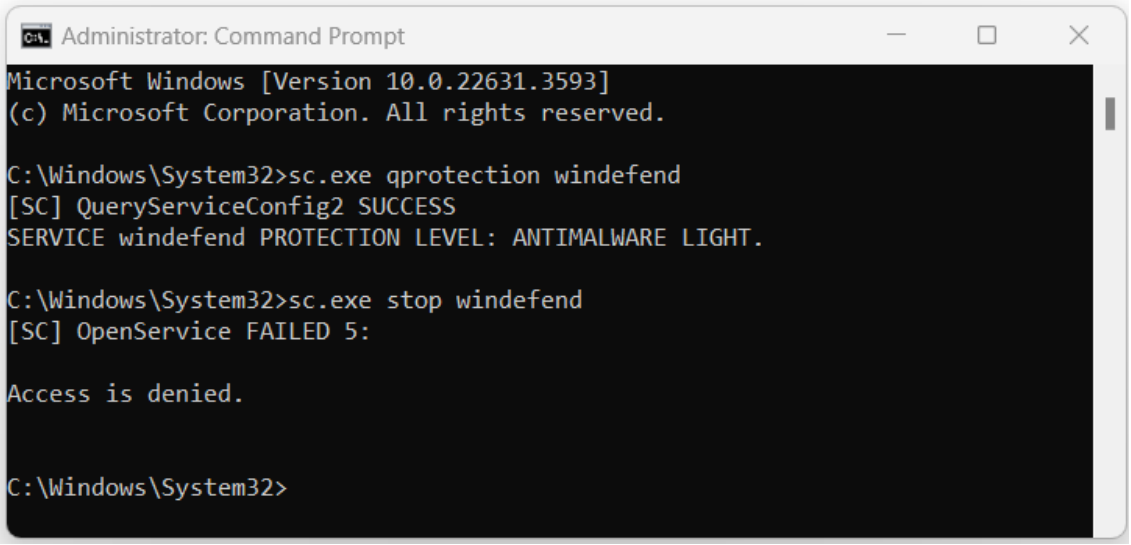


Figure 59: Comprobar protección de proceso asociado a Windows Defender

Algo importante a mencionar en este punto es que los procesos sólo están protegidos en modo usuario, Windows utiliza la estructura PS_PROTECTION como un campo de la conocida EPROCESS para determinar la existencia o no de esta protección, por lo que cualquier programa que se ejecute en modo kernel, podrá acceder a la memoria de esos procesos, modificando el valor o valores que quiera.

Veamos el ejemplo de Mimikatz. Mimikatz es un software que entre otras operaciones es capaz de recuperar contraseñas y hashes de un sistema alojados en un equipo Windows, leyendo la memoria del proceso lsass.exe. En la actualidad esto no es posible de forma directa, por lo que Mimikatz necesita de un driver (mimidrv.sys) para poder realizar esas acciones, consideradas "peligrosas" o cuya finalidad no se concibe en un uso normal.

Siguiendo con la explicación, ese driver es capaz de "desproteger" el proceso lsass.exe, volviendo a dejar el sistema vulnerable a los ataques definidos en dicha herramienta.

Una pequeña curiosidad de esto es que mimidrv.sys sí que está debidamente firmado, o al menos lo estuvo en un pasado ya que la fecha de validez del certificado corresponde al año 2014. Sin embargo, Windows todavía permite la instalación de este y otros certificados cuya situación es similar, aunque su uso sea extensamente conocido en la ejecución de acciones maliciosas.

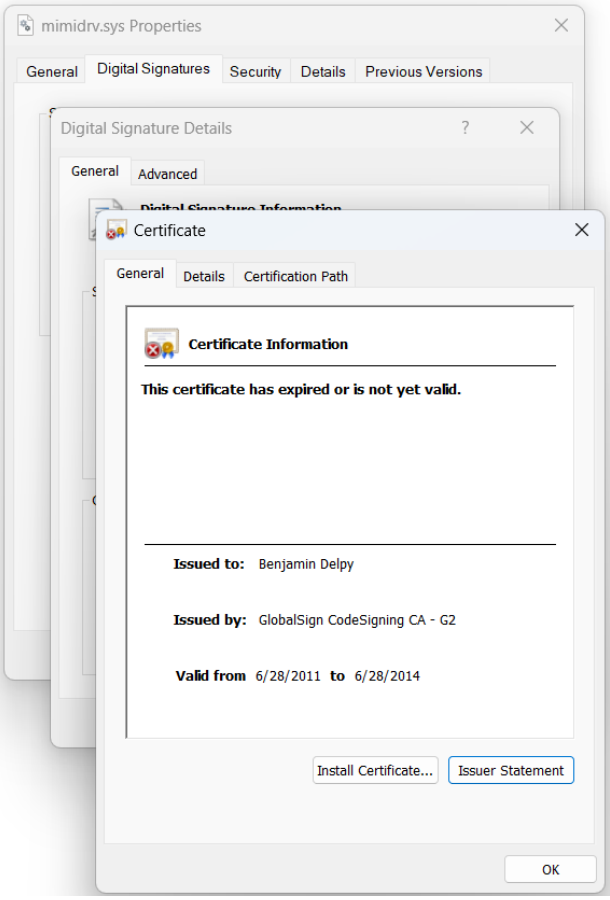


Figure 60: Firma del driver mimidrv.sys

5.3. VIRTUALIZATION BASED SECURITY (VBS)

Aunque con el paso de los años y debido a la implementación de una serie de medidas de seguridad, se les ha hecho más difícil a los atacantes el poder acceder al modo kernel, lo cierto es que todavía sigue siendo una posibilidad real. E incluso teniendo en cuenta que las nuevas versiones de Windows sólo pueden cargar, por defecto, drivers firmados, es prácticamente imposible controlar que no tengan vulnerabilidades o que alguna firma no haya sido robada, por no hablar también de la posibilidad de implantar un bootkit en la máquina.

Esto, una situación de la que Microsoft es plenamente consciente, que de alguna forma se comprometa el modo kernel, da como resultado la aparición de VBS para mitigar el riesgo si eso llega a suceder. Esta "característica de Windows", VBS, utiliza la virtualización de hardware para crear un entorno de ejecución aislado, protegiendo ciertos componentes críticos del sistema operativo. En otras palabras, con VBS se pueden ejecutar procesos de forma aislada al resto del sistema, en un entorno virtual, protegiendo la información que contenga o con la que trabaje ese proceso.

El utilizar VBS añade otros elementos a tener en el radar, como son los denominados VTLs (Virtual Trust Levels) o niveles de confianza establecidos por Microsoft.

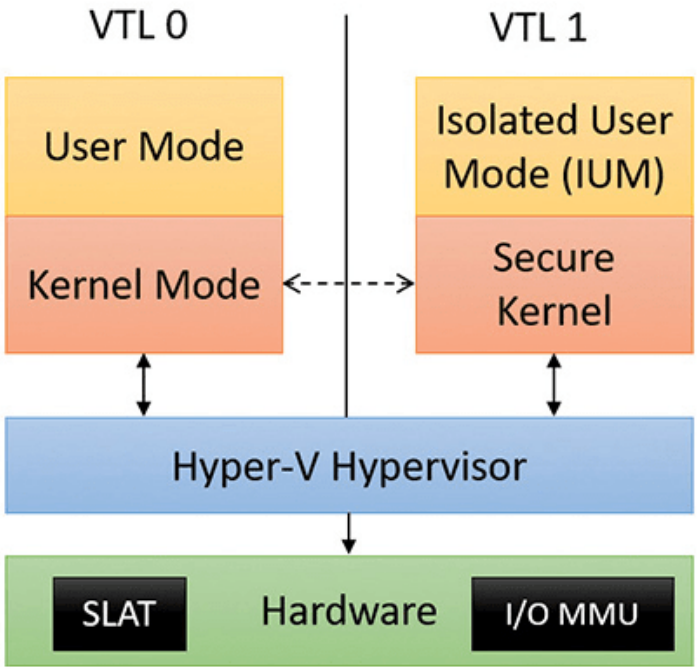


Figure 61: Sistema Windows con VBS habilitado

Digamos que la idea subyacente detrás de esto es realizar una clasificación de procesos más confiables o menos, para proteger unos de otros. Y si vemos los VTLs como una especie de máquinas virtuales independientes entre sí y protegidas por medio del hipervisor, podemos llegar a hacernos una mejor idea de esta protección.

Veamos un ejemplo: digamos que nuestro equipo ha sido "infectado" y que a consecuencia se ha implantado un driver malicioso en él, pero tenemos la protección VBS habilitada. ¿Qué ocurriría si ese driver intentase acceder a la memoria del proceso lsass.exe, para realizar un volcado de las credenciales? Pues podría acceder, pero no encontraría ningún tipo de información porque esta se localiza en un espacio protegido y no está permitido el acceso desde VTL0 a VTL1.

5.4. EARLY LAUNCH ANTI-MALWARE (ELAM)

Otra funcionalidad importante de Windows en relación a la posible ejecución de drivers maliciosos, es ELAM, que proporciona una protección temprana durante el proceso de arranque, trabajando conjuntamente con el motor de antivirus y antimalware de Windows para proporcionar una capa de seguridad adicional.

Al tener habilitada esta protección, se comprueban los drivers críticos del sistema, antes de que se carguen en memoria, en busca de malware y, si se detecta algún driver sospechoso se bloqueará la carga de este.

Si necesitásemos deshabilitarlo, en las opciones avanzadas de inicio, al igual que cuando tratamos el DSE, también aparece una opción para ELAM.

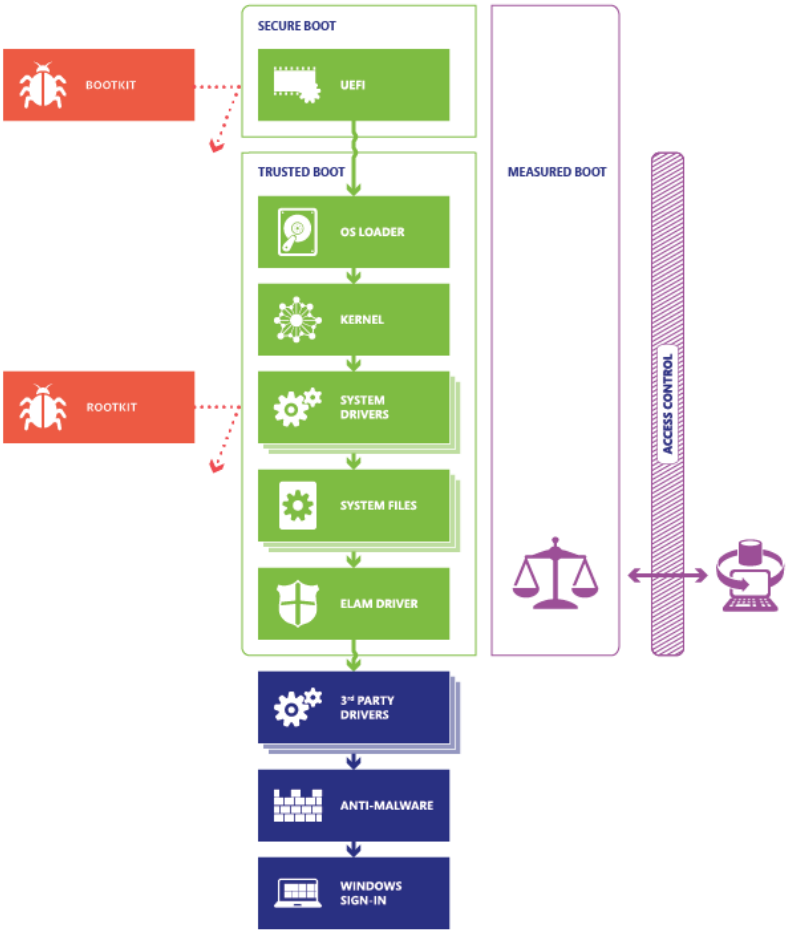


Figure 62: Inicio seguro en Windows

5.5. KERNEL PATCH PROTECTION (KPP)

Kernel patch protection, más conocido como PatchGuard, es una característica de seguridad de Windows que protege el kernel de nuestro sistema operativo de posibles vulnerabilidades que puedan ser aprovechadas por el malware.

Básicamente, esta protección impide que los drivers alteren el kernel de Windows, supervisando el mismo y detectando cualquier intento de modificación de estructuras fundamentales, que de ser así implicaría la aparición de un BSOD con un mensaje de error parecido a "KERNEL_SECURITY_CHECK_FAILURE".

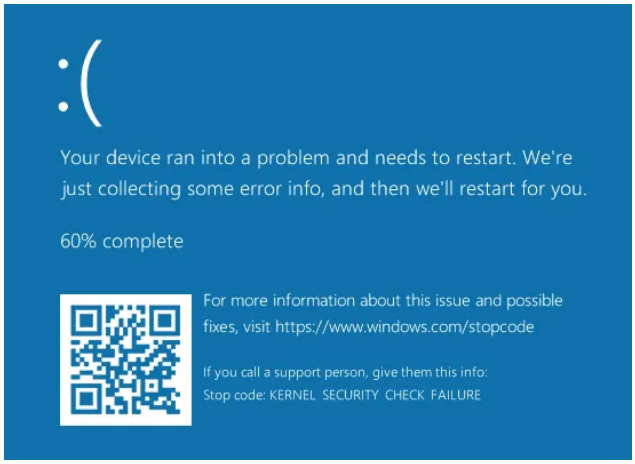


Figure 63: BSOD lanzada por PatchGuard

PatchGuard monitoriza, periódicamente, una lista predefinida de estructuras que normalmente suelen ser objetivo de ataques, así como cambios en los registros, aunque como no corre todo el tiempo puede darse la situación de que un malware modifique una estructura y, antes de que PatchGuard vuelva a realizar el escaneo la revierta a su estado original, no detectando que el corazón del sistema, el kernel, ha sido alterado y que el equipo está infectado.

Además, uno de los puntos débiles y la razón de que no exista documentación oficial sobre su implementación, es que, aunque monitoriza lo que entendemos como Ring 0, nada impide que un rootkit que ya ha ganado esos privilegios de acceso modifique el código y deshabilite/evada esta protección. Es por este motivo, que os resultará bastante complicado analizar el código de PatchGuard, ya que se utilizan varias técnicas de ofuscación a fin de que los atacantes no puedan descifrar cómo se ha desarrollado e implementar PoCs para saltárselo.

5.6. CONTROL FLOW GUARD (CFG)

Control Flow Guard (CFG) es una característica de seguridad altamente optimizada y diseñada para proteger las aplicaciones contra vulnerabilidades que dañen la memoria. Al imponer restricciones estrictas sobre los lugares desde donde una aplicación puede ejecutar código, CFG dificulta significativamente que las vulnerabilidades de seguridad ejecuten código arbitrario mediante técnicas como los desbordamientos de búfer. Esta característica se construye sobre tecnologías de mitigación de vulnerabilidades, como /GS, DEP y ASLR, proporcionando una capa adicional de protección. La opción /GS añade comprobaciones para evitar desbordamientos de búfer en la pila, DEP (Data Execution Prevention) previene la ejecución de código desde regiones de memoria marcadas como no ejecutables, y ASLR (Address Space Layout Randomization) aleatoriza la disposición de áreas clave de datos en la memoria para dificultar la predicción de direcciones útiles en un ataque.

La principal función de CFG es evitar daños en la memoria y mitigar ataques, restringiendo las áreas desde donde se puede ejecutar el código.

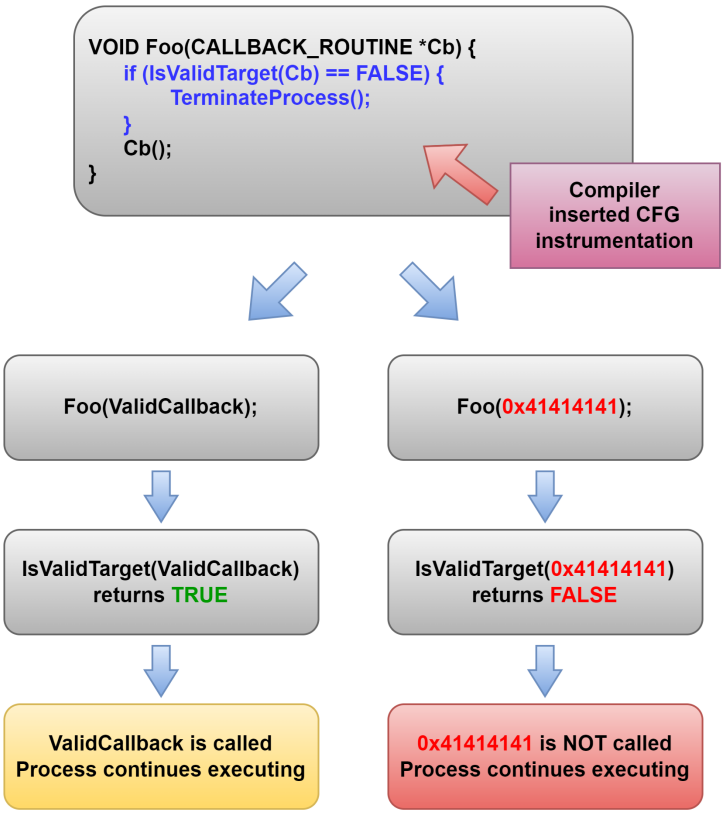


Figure 64: Control Flow Guard

Las vulnerabilidades de software suelen ser explotadas proporcionando datos inesperados o extremos a un programa en ejecución. Por ejemplo, un atacante podría explotar una vulnerabilidad proporcionando más datos de los que un programa espera, sobrescribiendo así áreas de memoria adyacentes que podrían contener punteros a otras funciones. Cuando el programa intenta utilizar estos punteros, podría saltar a una ubicación especificada por el atacante, permitiendo la ejecución de código malicioso.

CFG combate estas técnicas mediante una combinación de compatibilidad en tiempo de compilación y tiempo de ejecución que implementa la integridad del flujo de control. En tiempo de compilación, el compilador añade comprobaciones de seguridad ligeras al código e identifica el conjunto de funciones que son destinos válidos para llamadas indirectas. En tiempo de ejecución, el kernel de Windows mantiene un estado que identifica estos destinos válidos y ejecuta la lógica que verifica la validez de un destino de llamada indirecta.

Cuando una comprobación de CFG falla en tiempo de ejecución, Windows termina inmediatamente el programa, interrumpiendo cualquier intento de explotar una vulnerabilidad de seguridad mediante llamadas a direcciones no válidas. Este enfoque asegura que las aplicaciones se protejan contra una amplia gama de técnicas de explotación, haciendo que sea mucho más difícil para los atacantes ejecutar código arbitrario.

Para habilitar CFG en un proyecto del IDE Visual Studio, podemos utilizar la opción /guard del compilador, como se ve en la siguiente imagen, que analiza el flujo de control de los destinos de llamadas indirectas en tiempo de compilación e inserta el código necesario para comprobar esos destinos en tiempo de ejecución.

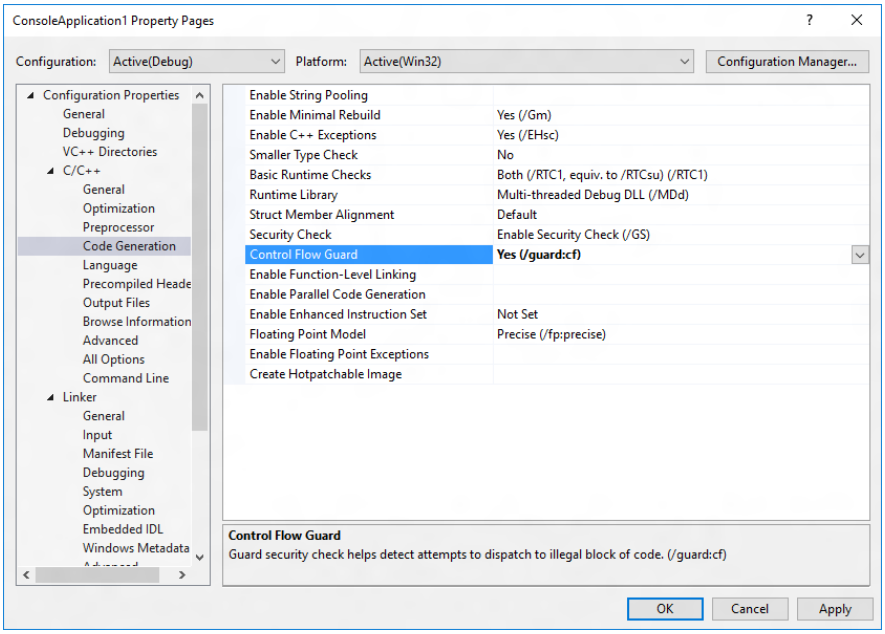


Figure 65: Añadir la protección CFG a un proyecto compilado con Visual Studio

6. DEBUG Y ANÁLISIS DEL SISTEMA OPERATIVO

Dado que en este punto del módulo ya se tiene un conocimiento apropiado del kernel, así como de las protecciones que ha incluido este sistema operativo a lo largo del tiempo, introduciremos ahora todo lo relacionado con la depuración de código en el kernel de Windows. Para ser específicos, durante el proceso de debugging lo que se analiza en verdad es el ejecutable ntoskrnl.exe, que se corresponde con el propio kernel.

Este análisis/depuración de código se lleva a cabo mediante el uso de un debugger (Kernel Debugger) como WinDbg, una herramienta proporcionada por Microsoft que permite a los desarrolladores examinar y analizar el sistema operativo en tiempo real.

WinDbg se puede utilizar para realizar pruebas en el código del kernel, detectar malware, analizar volcados de memoria, encontrar errores en el código y, en definitiva, cualquier tarea que implique la depuración y análisis de nuestro equipo.

Este proceso, aunque es complejo y requiere un sólido conocimiento de las estructuras y funciones del sistema operativo, nos permite examinar el comportamiento del sistema a bajo nivel, y obtener una comprensión profunda de su funcionamiento interno necesario tanto a la hora de descubrir nuevas vulnerabilidades como de detectar ataques avanzados.

6.1. LOCAL KERNEL DEBUGGING

La herramienta WinDbg permite la depuración local del kernel, es decir, el depurador se ejecuta en el mismo equipo que se está depurando. Aunque en este modo, como bien se explica en la documentación de Microsoft, no se pueden realizar ciertas acciones.

Los comandos de interrupción (breakpoints) y volcado, entre otros, no están disponibles ya que, en el modo local, por seguridad, no se depura el sistema operativo, sino que se crea un dump de la memoria que se va actualizando, con lo que parece que estamos en una sesión de depuración real, pero no es el caso.

Commands That Are Not Available

Not all commands are available in a local kernel debugging session. Typically, you cannot use any command that causes the target computer to stop, even momentarily, because you cannot resume operation.

In particular, you cannot use the following commands:

- Execution commands, such as **g (Go)**, **p (Step)**, **t (Trace)**, **wt (Trace and Watch Data)**, **tb (Trace to Next Branch)**, **gh (Go with Exception Handled)**, and **gn (Go with Exception Not Handled)**
- Shutdown and dump file commands, such as **.crash**, **.dump**, and **.reboot**
- Breakpoint commands, such as **bp**, **bu**, **ba**, **bc**, **bd**, **be**, and **bl**
- Register display commands, such as **r** and variations
- Stack trace commands, such as **k** and variations

If you are performing local kernel debugging with WinDbg, all of the equivalent menu commands and buttons are also unavailable.

Figure 66: Comandos no disponibles en una sesión de depuración de kernel local

Para llevar a cabo este análisis en local, siendo recomendable utilizar una máquina virtual, olvidándonos de posibles pérdidas de información al provocar un fallo en el equipo realizando modificaciones en el código del kernel y la memoria, primeramente, tendremos que instalar WinDbg.

Una vez instalada esta herramienta, la ejecutamos con permisos de administrador y podremos entrar en una sesión de depuración local mediante las opciones:

File -> Start debugging -> Attach to kernel -> Local -> OK

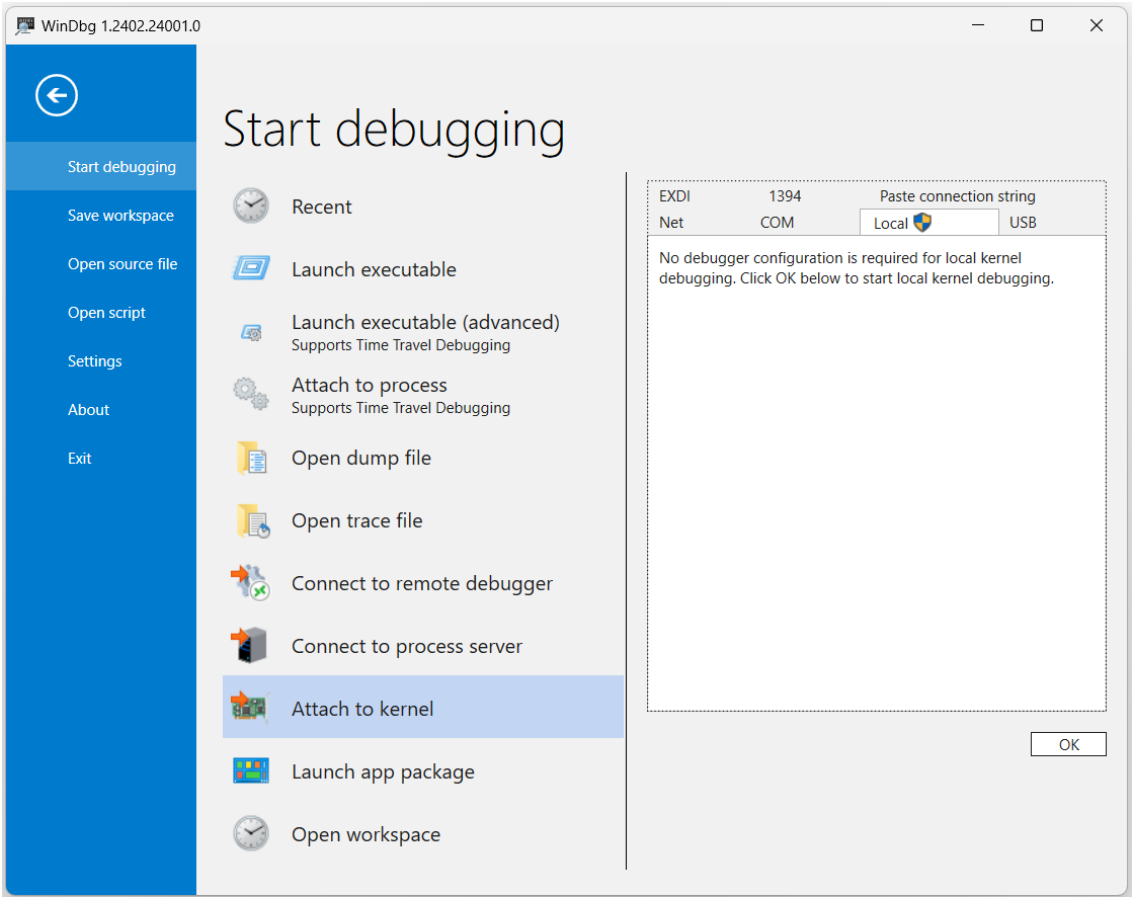


Figure 67: Herramienta WinDbg

Si es la primera vez que se configura WinDbg en el equipo, se mostrará un error indicando que el sistema no soporta la opción de depuración, y que se debe habilitar esta característica. Para ello, como se indica en el mensaje, se tiene que ejecutar el comando "bcdedit.exe /debug on" en una terminal con permisos de administrador, reiniciando posteriormente la máquina para que los cambios tengan efecto.

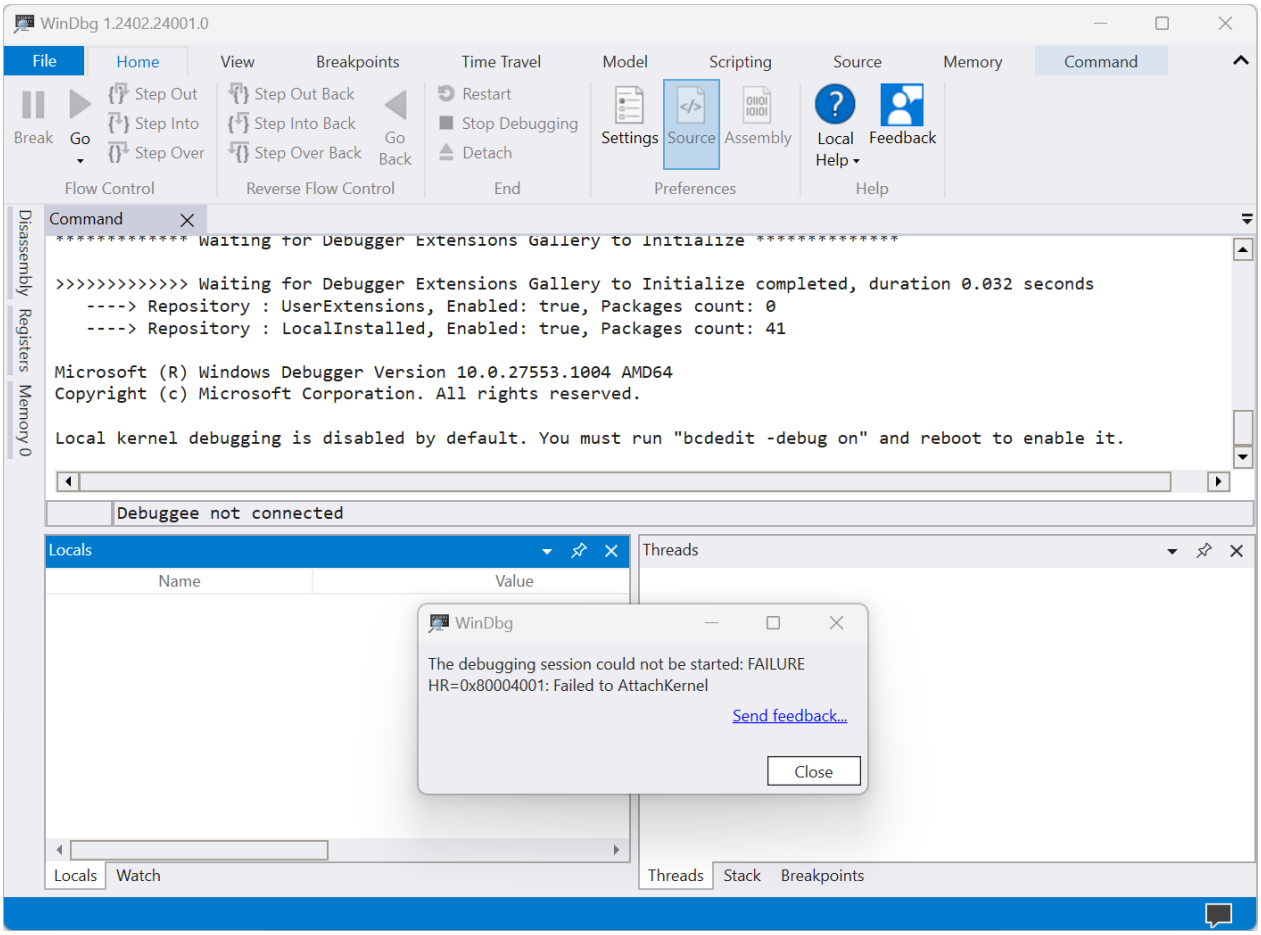


Figure 68: Mensaje de error en WinDbg (opción de depuración deshabilitada)

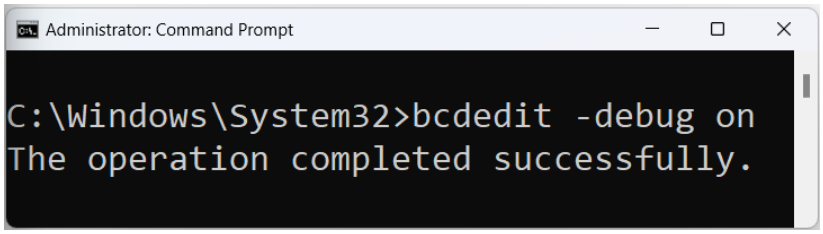


Figure 69: Habilitar depuración de Windows

Al habilitar esta característica, reiniciar y seguir de nuevo los pasos, WinDbg muestra un mensaje como el siguiente, indicando que ya nos encontramos en una sesión de depuración local del kernel, y que podemos empezar a depurar el sistema operativo.

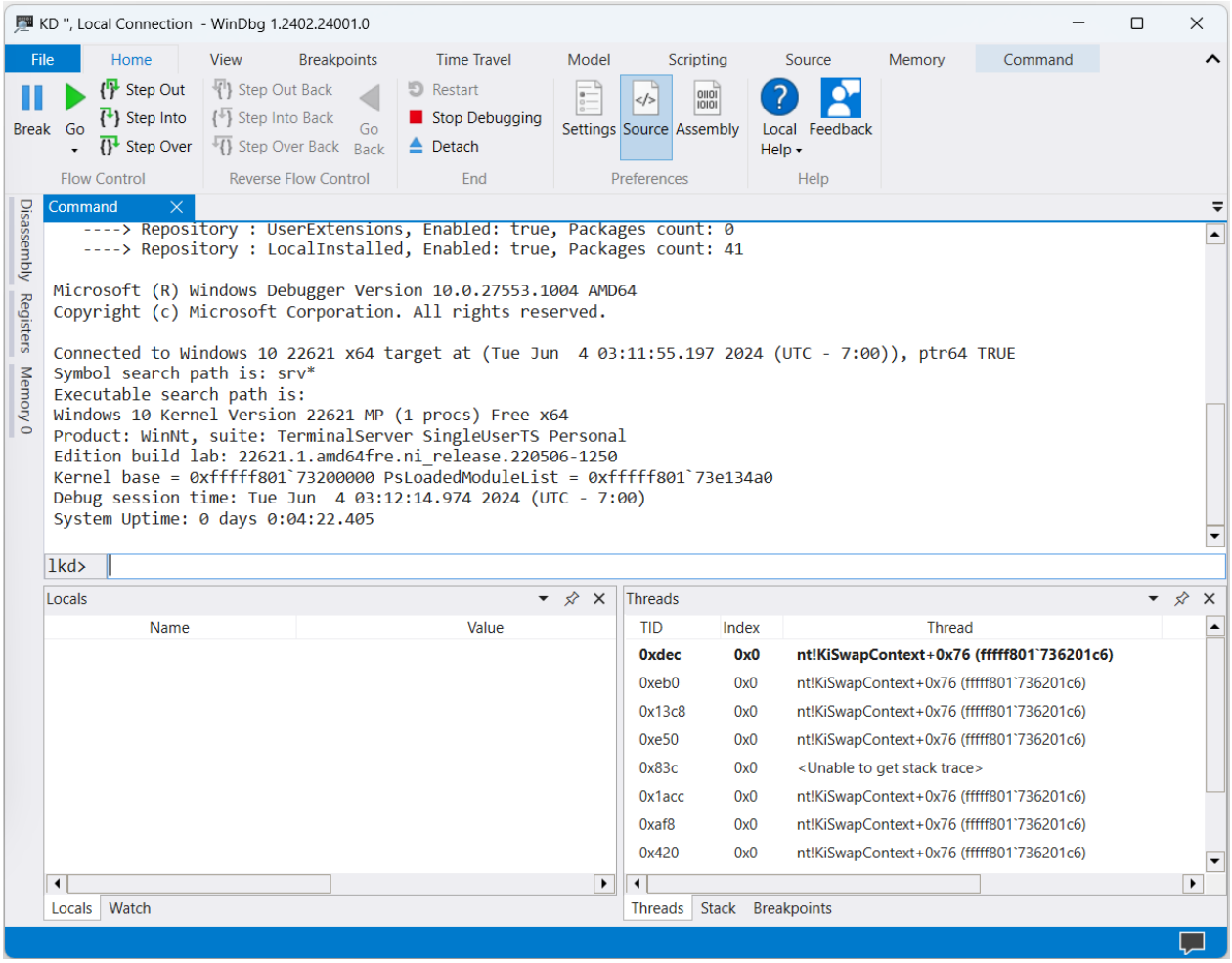


Figure 70: Sesión de depuración local del kernel en WinDbg

6.2. REMOTE KERNEL DEBUGGING

En ocasiones, la depuración en local puede no ajustarse a nuestras necesidades ya que los comandos están limitados, por lo que WinDbg nos ofrece otra opción; la depuración a través de red en la cual intervienen dos equipos, el equipo que ejecuta el depurador, host, y el equipo que se está depurando, equipo destino a analizar.

La configuración de esta posibilidad se realiza en el equipo host mediante la opción Attach to kernel -> NET, donde se configuran el puerto de conexión y una clave que recuperaremos posteriormente de la máquina que se quiere analizar.

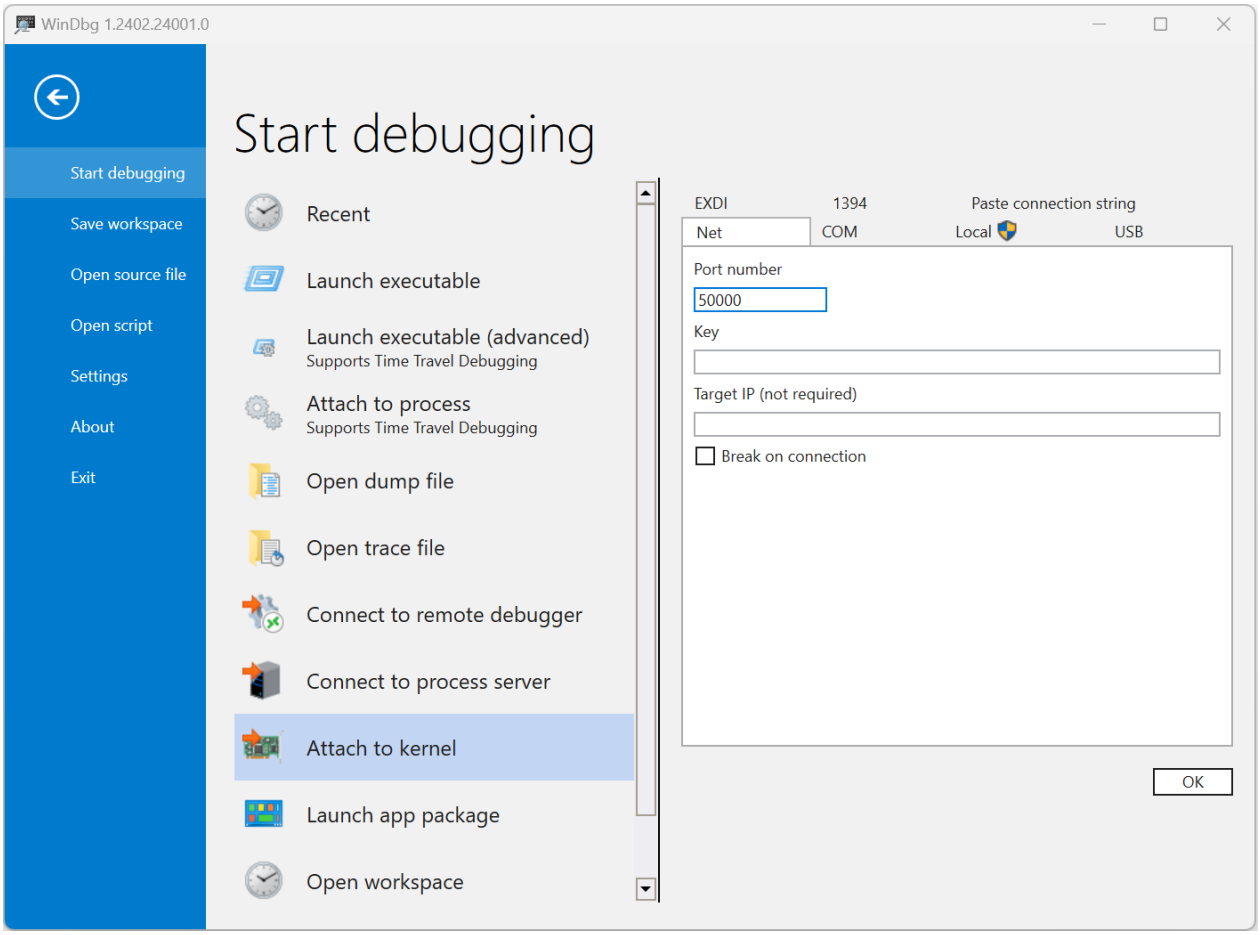


Figure 71: Opción de depuración remota en WinDbg

Primeramente, se tiene que configurar el equipo destino para que acepte la conexión y nos permita entrar en esa sesión de depuración remota. Para ello, se deben copiar los archivos kdnet.exe y VerifiedNICList.xml, dos archivos que forman parte de las herramientas de depuración de Windows, los cuales se encuentran en un directorio de nuestra máquina host "C:\Program Files (x86)\Windows Kits\10\Debugge- rs\x64".

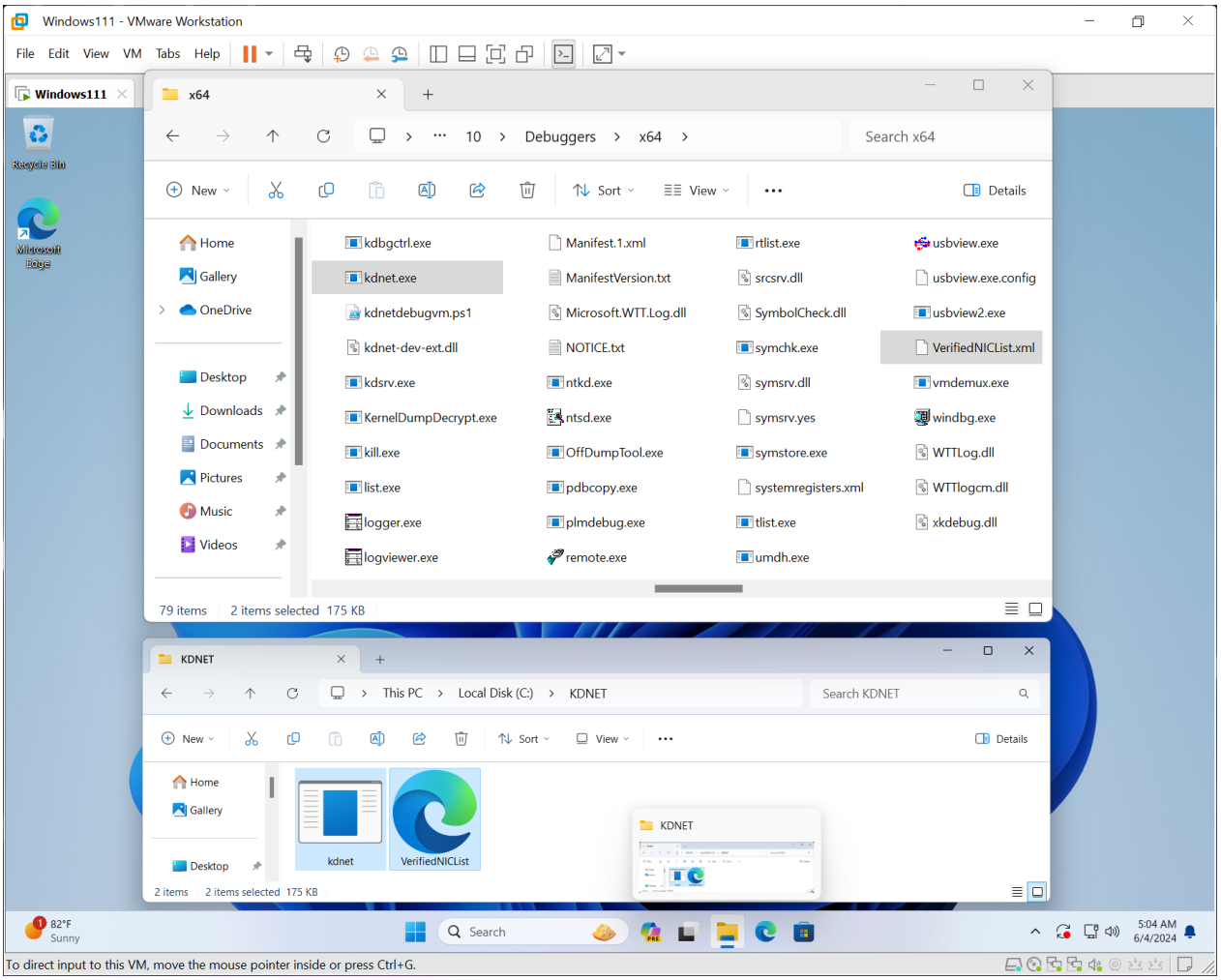


Figure 72: Traslado de archivos kdnet.exe y VerifiedNICList.xml a la máquina destino

Copiados estos dos archivos, en este ejemplo a la carpeta C:\KDNET de la máquina destino, abrimos una consola como administrador y ejecutamos el siguente comando:

"kdnet.exe <IP del equipo host> <Puerto de conexión>".

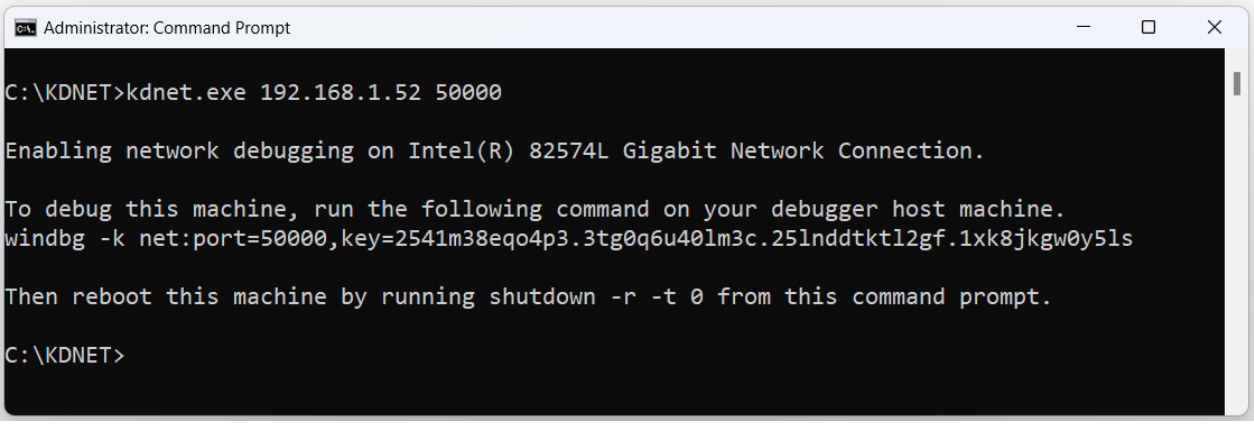


Figure 73: Configurar conexión para sesión de depuración remota

Este comando habilita la opción de depuración y nos proporcionará la clave que necesitamos indicar para realizar la conexión desde la máquina host, mediante la opción de WinDbg mencionada anteriormente.

En este punto, tenemos que recordar que ambas máquinas deben tener visibilidad entre ellas, por lo que seguramente haya que añadir una entrada para el firewall de Windows en la máquina destino, que permita la conexión al puerto indicado en el comando anterior, y comprobar que el puerto definido no esté en uso.

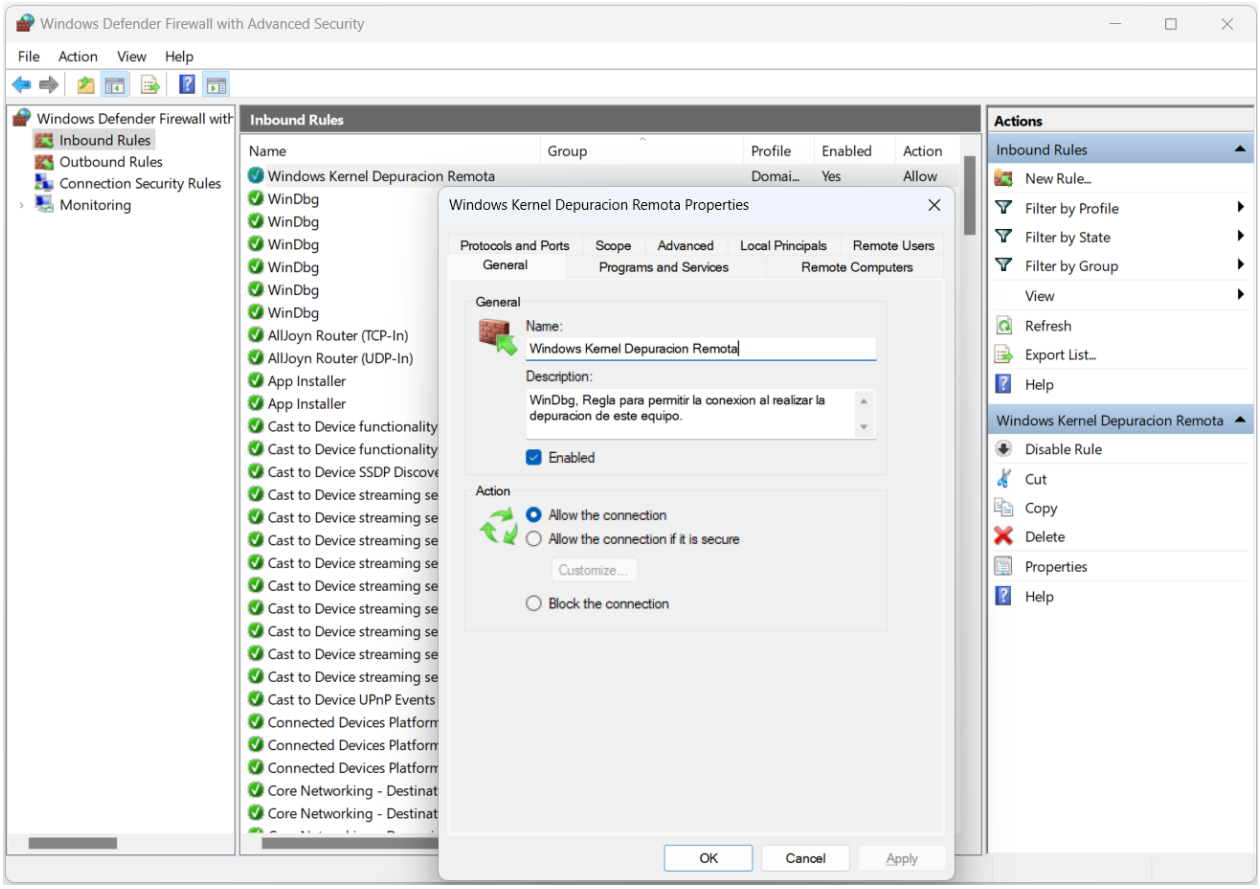


Figure 74: Regla del firewall de Windows para depuración mediante WinDbg

Finalmente, configuramos la opción de depuración en red en el equipo host, introduciendo la clave mostrada anteriormente al ejecutar la utilidad kdnet.exe, y reiniciamos la máquina destino ejecutando "shutdown -r -t 0", la cual se quedará congelada.

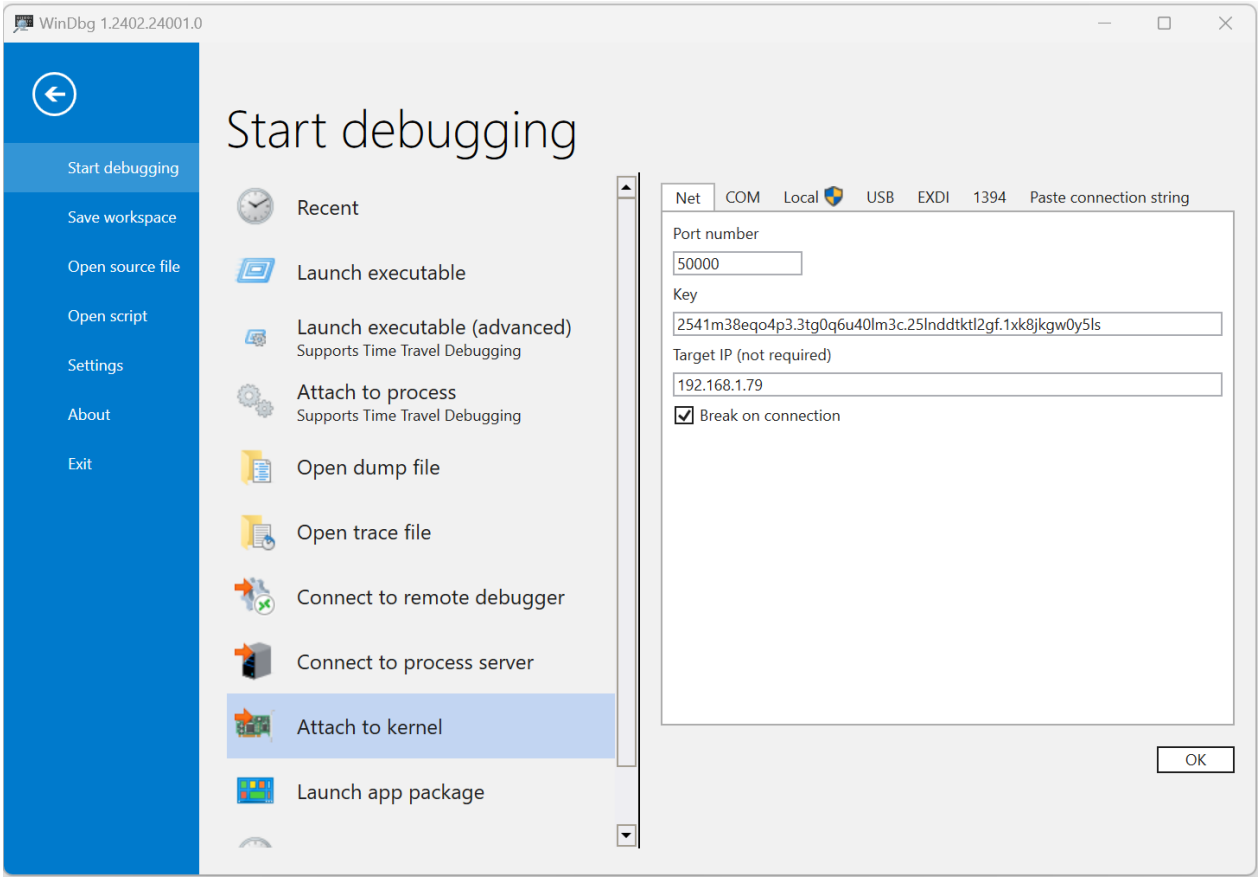


Figure 75: Conexión a la máquina destino en una sesión de depuración remota

Hecha la conexión, la herramienta WinDbg imprimirá un mensaje indicándonos que el proceso ha sido exitoso.

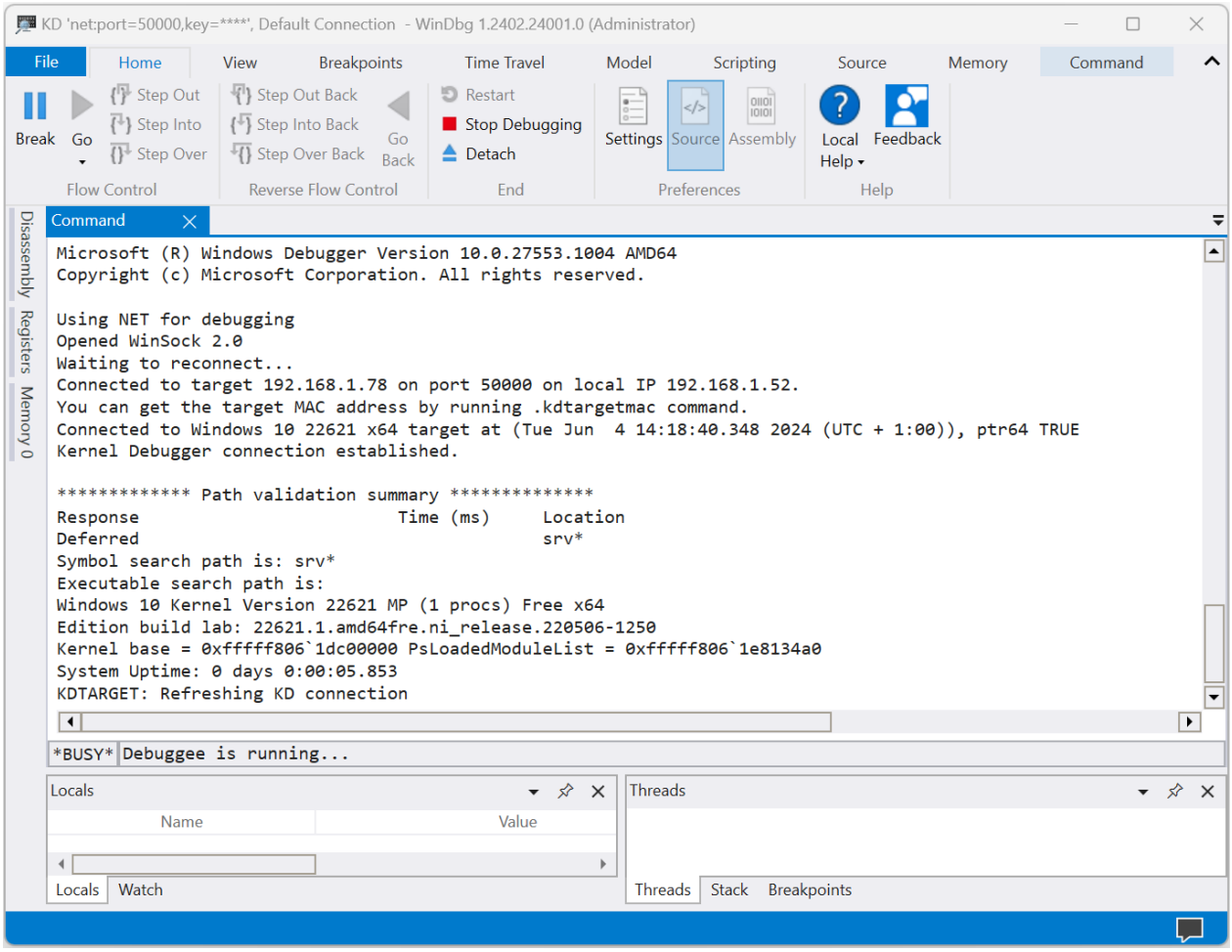


Figure 76: Conexión satisfactoria en una sesión de depuración remota

6.3. COMANDOS DE DEPURACIÓN

Al entrar en una sesión de depuración ya sea local o remota, utilizando WinDbg, tenemos a nuestra disponibilidad una gran lista de comandos a utilizar para depurar el sistema operativo, y en esta sección veremos los más prácticos en cuanto a su funcionalidad.

Cabe destacar que muchos de ellos se explican con detalle en la documentación de Microsoft, a la que podemos acceder directamente desde WinDbg mediante la instrucción ".hh comando".

De los comandos que vamos a ver, como introducción a WinDbg y a la depuración del sistema, podemos diferenciar tres categorías:

- **Comandos propios**: Estos elementos que ya vienen incluidos en la herramienta, conocidos como "built-in commands" o comandos predefinidos, son útiles para realizar tareas básicas y no están precedidos por ningún carácter especial.
- **Comandos de extensión**: Se corresponden con comandos que no están incluidos exactamente dentro de la herramienta, los cuales se identifican por estar precedidos de un símbolo de exclamación. Estos, son muy útiles para depurar problemas avanzados al profundizar en el detalle del sistema operativo y su comportamiento. Sin embargo, para utilizar adecuadamente los comandos de extensión, es necesario tener un conocimiento profundo de Windows y su arquitectura, así como de las librerías externas que contienen estos comandos.
- **Metacomandos**: Son instrucciones que se dan a la herramienta como ".hh" y que se distinguen por estar precedidos de un punto, denominándose "dot commands". Los metacomandos se utilizan para controlar la configuración de WinDbg, y proporcionan información útil sobre los procesos, los hilos, los módulos del sistema, los símbolos, y demás elementos relacionados con el sistema y su depuración/debugging.

6.3.1. SÍMBOLOS

Cuando se habla de los símbolos en WinDbg, nos referimos a una especie de archivos que contienen información sobre nombres de funciones y variables, a los que el depurador necesita tener acceso para poder mostrar información relevante sobre el código que se está ejecutando. Si no es así y los símbolos no están cargados, los nombres de las funciones y las variables no serán reconocidas, dificultando la depuración.

WinDbg pone a nuestra disposición tres metacomandos para gestionar los símbolos y asegurar que estén disponibles antes de proceder con la depuración del sistema:

- **.sympath**: Se utiliza para imprimir los directorios en los que WinDbg busca los símbolos, pudiendo añadir registros nuevos de la siguiente forma: ".sympath+ C:\NewPath".
- **.symfix**: Se utiliza para restablecer los directorios, donde WinDbg busca los símbolos, a la ubicación predeterminada de la tienda de símbolos de Microsoft. Este comando es útil si el depurador está teniendo problemas al cargar dichos archivos.
- **.reload**: Se utiliza para recargar la información de los símbolos aplicando la configuración actual, es decir, esta sería la opción para el caso donde se hayan agregado o modificado símbolos y se tengan que cargar de nuevo.

```
kd> .sympath
Symbol search path is: srv*
Expanded Symbol search path is: cache*;SRV*https://msdl.microsoft.com/download/symbols

************* Path validation summary **************
Response                         Time (ms)     Location
Deferred                                       srv*
kd> .symfix
kd> .reload
Connected to Windows 10 22621 x64 target at (Tue Jun  4 22:44:54.505 2024 (UTC + 1:00)), ptr64 TRUE
Loading Kernel Symbols
...............................................................
...............................................................
...............................................................
....
Loading User Symbols
.....................................
Loading unloaded module list
..........
```

Figure 77: Ejecución de metacomandos para gestión de símbolos en WinDbg

Una vez cargados los símbolos, se pueden listar los módulos accesible mediante la ejecución del comando "lm", y posteriormente consultar la funciones asociadas.

```
kd> lm
start             end                 module name
fffff75e`fe4a0000 fffff75e`fe54b000   win32k     (deferred)
fffff75e`fe600000 fffff75e`fe9a9000   win32kfull   (deferred)
fffff75e`fe9f0000 fffff75e`fed04000   win32kbase   (deferred)
fffff75e`fed10000 fffff75e`fed57000   cdd         (deferred)
fffff806`1a0e0000 fffff806`1a463000   mcupdate_GenuineIntel   (deferred)
fffff806`1a470000 fffff806`1a476000   hal         (deferred)
fffff806`1a4c0000 fffff806`1a51d000   kd_02_8086   (deferred)
fffff806`1a520000 fffff806`1a567000   kdcom       (deferred)
fffff806`1a5f0000 fffff806`1a5fd000   BOOTVID     (deferred)
fffff806`1dc00000 fffff806`1ec47000   nt          (pdb symbols)          C:\ProgramData\Dbg\sym\ntkrnlmp.pdb\
fffff806`1fe00000 fffff806`1fe29000   tm          (deferred)
fffff806`1fe30000 fffff806`1fe9f000   CLFS        (deferred)
fffff806`1fea0000 fffff806`1febb000   PSHED       (deferred)
fffff806`1fec0000 fffff806`1ffd5000   clipsp      (deferred)
fffff806`1ffe0000 fffff806`2005a000   FLTMGR      (deferred)
fffff806`20060000 fffff806`2008c000   ksecdd      (deferred)
fffff806`20090000 fffff806`200f3000   msrpc       (deferred)
```

Figure 78: Lista de módulos cargados en WinDbg

```
kd> x nt!*
fffff806`1e738e1c nt!MxPageAlwaysHot (void)
fffff806`1df0e878 nt!MiSyncSystemPdes (void)
fffff806`1def5da0 nt!RtlpHpVaMgrRangeCreate (void)
fffff806`1de00438 nt!MiProcessWsInSwapRanges (void)
fffff806`1e036e22 nt!KiInitializeUserApc$filt$0 (void)
fffff806`1e37fe64 nt!PopEtProcessSnapshotCreate (void)
fffff806`1e48b1ca nt!ExpGetProcessInformation$filt$4 (void)
fffff806`1e317e30 nt!AlpcpDeleteBlob (void)
fffff806`1e7616ac nt!IopStoreArcInformation (void)
fffff806`1de3a770 nt!PpmParkSteerInterrupts (void)
fffff806`1dfc2690 nt!CcSetLoggedDataThreshold (void)
fffff806`1de3b7c0 nt!PpmSnapPerformanceAccumulation (void)
fffff806`1e48b206 nt!ExpGetProcessInformation$filt$5 (void)
fffff806`1e756ff4 nt!MiFreeKernelPadSections (void)
fffff806`1de0a130 nt!RtlFindLastBackwardRunClear (void)
fffff806`1e48b0d4 nt!ExpGetProcessInformation$filt$6 (void)
fffff806`1e48b248 nt!ExpGetProcessInformation$filt$7 (void)
fffff806`1e297b84 nt!ExpGetSystemFirmwareTableInformation (void)
fffff806`1e48b020 nt!ExpGetProcessInformation$filt$0 (void)
fffff806`1e446b20 nt!_RegRtlOpenPredefinedKey (void)
fffff806`1e48b05c nt!ExpGetProcessInformation$filt$1 (void)
fffff806`1e31cca8 nt!SeValidateImageHeader (void)
fffff806`1e3e90fc nt!WbCreateWarbirdProcess (void)
fffff806`1e6ea240 nt!AnFwpDisableProgressTimer (void)
fffff806`1e48b098 nt!ExpGetProcessInformation$filt$2 (void)
```

Figure 79: Lista de funciones asociadas al módulo nt

6.3.2. PROCESOS E HILOS

De los comandos que proporciona WinDbg para consultar procesos, el más versátil es "!process", con el cuál se pueden listar todos los procesos activos y sus atributos, entre ellos: el PID, el nombre del ejecutable, la dirección de memoria de su estructura EPROCESS, y demás información.

```
kd> !process 0 0
**** NT ACTIVE PROCESS DUMP ****
PROCESS ffffc802bc8fd040
    SessionId: none  Cid: 0004    Peb: 00000000  ParentCid: 0000
    DirBase: 001ae002  ObjectTable: ffffdc8eb7882ec0  HandleCount: 2507.
    Image: System

PROCESS ffffc802bc9c8080
    SessionId: none  Cid: 0040    Peb: 00000000  ParentCid: 0004
    DirBase: 100245002  ObjectTable: ffffdc8eb787bd80  HandleCount:   0.
    Image: Registry

PROCESS ffffc802be91f040
    SessionId: none  Cid: 0158    Peb: 869cb4000  ParentCid: 0004
    DirBase: 102a05002  ObjectTable: ffffdc8eb814b480  HandleCount:  58.
    Image: smss.exe

PROCESS ffffc802be92f080
    SessionId: 0  Cid: 01f0    Peb: d217ccb000  ParentCid: 01d4
    DirBase: 2324e1002  ObjectTable: ffffdc8eb7d528c0  HandleCount: 483.
    Image: csrss.exe

PROCESS ffffc802bf409080
    SessionId: 0  Cid: 0234    Peb: 94e3065000  ParentCid: 01d4
    DirBase: 229be2002  ObjectTable: ffffdc8eb7d52e80  HandleCount: 146.
    Image: wininit.exe
```

Figure 80: Listar los procesos en ejecución

Además, mediante este comando también se pueden listar los hilos asociados a un proceso en concreto:

```
kd> !process 0 2 mspaint.exe
PROCESS ffffc802c1ae70c0
    SessionId: 1  Cid: 1944    Peb: e54e73a000  ParentCid: 1120
    DirBase: 1bd414002  ObjectTable: ffffdc8ebef4a680  HandleCount: 668.
    Image: mspaint.exe

        THREAD ffffc802c1057080  Cid 1944.12a4  Teb: 000000e54e73b000 Win32Thread: ffffc802c0abccd0 WAIT: (WrUserRequest) UserMode Non-Alertable
            ffffc802c0ac01c0  QueueObject

        THREAD ffffc802c1aeb080  Cid 1944.0870  Teb: 000000e54e73d000 Win32Thread: 0000000000000000 WAIT: (WrQueue) UserMode Alertable
            ffffc802c0ab7300  QueueObject

        THREAD ffffc802c1046080  Cid 1944.0880  Teb: 000000e54e73f000 Win32Thread: 0000000000000000 WAIT: (UserRequest) UserMode Non-Alertable
            ffffc802c0dd79e0  SynchronizationEvent
            ffffc802c0dd7fe0  SynchronizationEvent
            ffffc802c0dd7460  SynchronizationEvent
            ffffc802c0dd7660  SynchronizationEvent
            ffffc802c0dd80e0  SynchronizationEvent
            ffffc802c0dd6de0  SynchronizationEvent
```

Figure 81: Listar los hilos de un proceso

6.3.3. MEMORIA

6.3.3.1. OPERACIONES DE LECTURA

Para realizar operaciones de lectura sobre la memoria se utiliza el comando d* (Display Memory), que muestra el contenido de una dirección o rango de memoria específico, donde el asterisco hace referencia a sus múltiples variantes: d, da, db, dc, dd, dD, df, dp, dq, du, dw, dda, ddp, ddu, dpa, dpp, dpu, dqa, dqp, dqu, dds, dps, dqs.

De entre todas estas posibilidades, nos centraremos en las siguientes:

- **da (display ASCII)**: Se utiliza para mostrar el contenido de un bloque de memoria como caracteres ASCII.
- **db (display bytes)**: Se utiliza para mostrar el contenido de un bloque de memoria como bytes en notación hexadecimal. Su uso es igual que el comando anterior y ambos son útiles para examinar el contenido de la memoria y las cadenas de caracteres.
- **dd (display doubleword)**: Se utiliza para mostrar los valores hexadecimales de los bytes de memoria, junto con los caracteres ASCII correspondientes.

Como se ve en la imagen utilizando estos tres comandos y a partir de la dirección base de un proceso, sumando el desplazamiento correspondiente, se puede imprimir el nombre de este.

```
kd> da ffffc802c1ae70c0+5a8
ffffc802`c1ae7668  "mspaint.exe"
kd> db ffffc802c1ae70c0+5a8
ffffc802`c1ae7668  6d 73 70 61 69 6e 74 2e-65 78 65 00 00 00 00 02  mspaint.exe.....
ffffc802`c1ae7678  00 00 00 00 00 00 00 00-c0 d6 be c0 02 c8 ff ff  ................
ffffc802`c1ae7688  08 e3 60 c0 02 c8 ff ff-08 e3 60 c0 02 c8 ff ff  ..`.......`.....
ffffc802`c1ae7698  00 00 ff ff ff 7f 00 00-b8 75 05 c1 02 c8 ff ff  .........u......
ffffc802`c1ae76a8  b8 b5 2a c1 02 c8 ff ff-15 00 00 00 31 45 07 7b  ..*.........1E.{
ffffc802`c1ae76b8  00 80 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
ffffc802`c1ae76c8  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
ffffc802`c1ae76d8  00 00 00 00 00 00 00 00-55 00 00 00 00 00 00 00  ........U.......
kd> dd ffffc802c1ae70c0+5a8
ffffc802`c1ae7668  6170736d 2e746e69 00657865 02000000
ffffc802`c1ae7678  00000000 00000000 c0bed6c0 ffffc802
ffffc802`c1ae7688  c060e308 ffffc802 c060e308 ffffc802
ffffc802`c1ae7698  ffff0000 00007fff c10575b8 ffffc802
ffffc802`c1ae76a8  c12ab5b8 ffffc802 00000015 7b074531
ffffc802`c1ae76b8  00008000 00000000 00000000 00000000
ffffc802`c1ae76c8  00000000 00000000 00000000 00000000
ffffc802`c1ae76d8  00000000 00000000 00000055 00000000
```

Figure 82: Ejecución de los comandos da, db y dd

- **dp (display pointer)**: Se utiliza para mostrar el contenido de un puntero que apunta a una determinada dirección de memoria.
- **dps (Display pointer symbols)**: Se utiliza para mostrar los símbolos que están asociados a las direcciones de memoria que se encuentran en un rango específico.

Para ver un ejemplo práctico del uso de estos dos comandos se podría inspeccionar las correspondientes entradas de la tabla de descriptores de servicios (SSDT), que se introduce en detalle a posteriori.

```
kd> dp nt!KeServiceDescriptorTable
fffff806`1ea018c0  fffff806`1dccfb30 00000000`00000000
fffff806`1ea018d0  00000000`000001e6 fffff806`1dcd02cc
fffff806`1ea018e0  00000000`00000000 00000000`00000000
fffff806`1ea018f0  00000000`00000000 00000000`00000000
fffff806`1ea01900  00000000`00000000 00000000`00000000
fffff806`1ea01910  fffff806`1e6f42c0 fffff806`1e6f4340
fffff806`1ea01920  fffff806`1e6f4d40 fffff806`1e6f4dc0
fffff806`1ea01930  fffff806`1e6f4e40 fffff806`1e6f61c0
kd> dps nt!KeServiceDescriptorTable
fffff806`1ea018c0  fffff806`1dccfb30 nt!KiServiceTable
fffff806`1ea018c8  00000000`00000000
fffff806`1ea018d0  00000000`000001e6
fffff806`1ea018d8  fffff806`1dcd02cc nt!KiArgumentTable
fffff806`1ea018e0  00000000`00000000
fffff806`1ea018e8  00000000`00000000
fffff806`1ea018f0  00000000`00000000
fffff806`1ea018f8  00000000`00000000
fffff806`1ea01900  00000000`00000000
fffff806`1ea01908  00000000`00000000
fffff806`1ea01910  fffff806`1e6f42c0 nt!KiBreakpointTrapShadow
fffff806`1ea01918  fffff806`1e6f4340 nt!KiOverflowTrapShadow
fffff806`1ea01920  fffff806`1e6f4d40 nt!KiRaiseSecurityCheckFailureShadow
fffff806`1ea01928  fffff806`1e6f4dc0 nt!KiRaiseAssertionShadow
fffff806`1ea01930  fffff806`1e6f4e40 nt!KiDebugServiceTrapShadow
fffff806`1ea01938  fffff806`1e6f61c0 nt!KiSystemCall64Shadow
kd> dd /c1 KiServiceTable L3
fffff806`1dccfb30  01b60504
fffff806`1dccfb34  02813c00
fffff806`1dccfb38  0711fc02
```

Figure 83: Ejecución de los comandos dp y dps

6.3.3.2. OPERACIONES DE ESCRITURA

De igual forma, también se pueden utilizar comandos para sobrescribir zonas de la memoria, haciendo posible modificar valores de estructuras del kernel, aunque hay que tener en cuenta que esto puede tener consecuencias no deseadas en el equipo y debe hacerse con precaución. Estas operaciones de escritura se llevan a cabo mediante los comandos e* (Enter Values), entre los cuáles podemos encontrar e, ea, eb, ed, eD, ef, ep, eq, eu, ew, eza.

De entre todas estas posibilidades, nos centraremos en las siguientes:

- **ea (enter ASCII)**: Se utiliza para insertar datos en formato ASCII en la memoria.
- **eq (enter quad-word)**: Se utiliza para escribir un valor de 8 bytes en la memoria.

```
kd> da ffff94826130e080+5a8
ffff9482`6130e628  "Notepad.exe"
kd> ea ffff94826130e080+5a8 "Notepad.exe"
kd> eq ffff94826130e080 ffff94826130e080
```

Figure 84: Ejecución de los comandos ea y eq

Como se ve en la imagen anterior, estos dos comandos se utilizan para escribir valores en una dirección de memoria específica, aunque en el ejemplo no se ha realizado una modificación en sí para no provocar un fallo en el sistema.

6.3.4. FLUJO

De entre los distintos tipos de comandos que nos aporta WinDbg, hay que destacar aquellos que permiten gestionar el control de la máquina a analizar en una sesión de depuración remota: parar la ejecución, continuar la ejecución, realizar una ejecución paso a paso, etc.

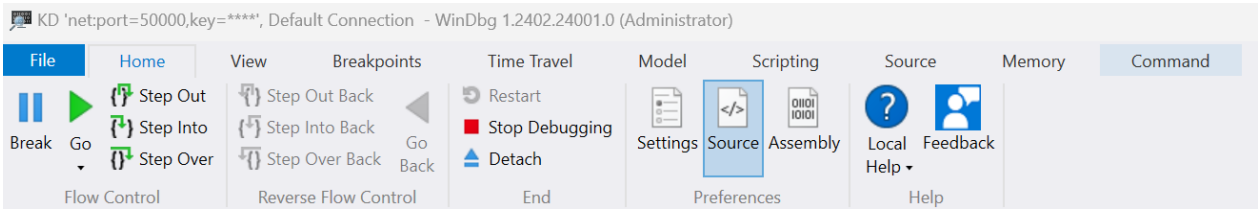


Figure 85: Comandos de flujo en WinDbg

Para controlar el flujo de la ejecución se pueden utilizar, en concreto, los siguientes comandos:

- **bp (Breakpoint)**: Son puntos de interrupción que detienen la ejecución del programa en un punto específico, permitiendo analizar y manipular el estado del sistema en ese momento.
- **bl (Breakpoint list)**: Se muestra una lista de todos los puntos de interrupción activos en la sesión de depuración.
- **bc (Breakpoint clear)**: Se eliminan los puntos de interrupción de acuerdo a sus números de identificación correspondientes.

```
kd> bp nt!NtCreateUserProcess
kd> bl
     0 e Disable Clear  fffff800`57db0a20     0001 (0001) nt!NtCreateUserProcess

kd> bc 0
kd> bl
```

Figure 86: Ejecución de los comandos bp, bl y bc

- **p (Step over)**: Se avanza la ejecución del programa a la siguiente instrucción, sin entrar en las subrutinas.
- **t (Step into)**: Se avanza la ejecución del programa a la siguiente instrucción, entrando en las subrutinas si es necesario.
- **gu (Step out)**: Se avanza la ejecución del programa hasta que la subrutina actual se completa.

```
kd> p
kdnic!TXSendCompleteDpc+0x184:
fffff800`5da63074 5b              pop     rbx
kd> t
kdnic!TXSendCompleteDpc+0x185:
fffff800`5da63075 c3              ret
kd> gu
nt!KiProcessExpiredTimerList+0x1eb:
fffff800`578e5f7b 448b7583        mov     r14d,dword ptr [rbp-7Dh]
```

Figure 87: Ejecución de los comandos p, t y gu

- **r (Register)**: Se muestra el contenido actual de los registros del procesador.

```
kd> r
rax=0000000000000001 rbx=0000000000000002 rcx=0000000000000001
rdx=000005e900000000 rsi=ffff958f600acb40 rdi=fffff80052304180
rip=fffff800578e5f7b rsp=fffff80053674bd0 rbp=fffff80053674c99
 r8=0000000000000000  r9=ffff958f5e1f4000 r10=fffff80057a22360
r11=0000000000000000 r12=0000000000000001 r13=fffff80053674e60
r14=fffff80053674e68 r15=0000000000000080
iopl=0         nv up ei ng nz na po nc
cs=0010  ss=0018  ds=002b  es=002b  fs=0053  gs=002b             efl=00000286
nt!KiProcessExpiredTimerList+0x1eb:
fffff800`578e5f7b 448b7583        mov     r14d,dword ptr [rbp-7Dh] ss:0018:fffff800`53674c1c=0000ffff
```

Figure 88: Ejecución del comando r

- **k (Stack)**: Se muestra el contenido actual de la pila del programa.

```
kd> k
 # Child-SP          RetAddr               Call Site
00 fffff800`53674bd0 fffff800`578410b4     nt!KiProcessExpiredTimerList+0x1eb
01 fffff800`53674d00 fffff800`57a217f5     nt!KiRetireDpcList+0xaf4
02 fffff800`53674fb0 fffff800`57a2179f     nt!KxSwapStacksAndRetireDpcList+0x5
03 ffffa586`8acff3a0 fffff800`57844235     nt!KiPlatformSwapStacksAndCallReturn
04 ffffa586`8acff3b0 fffff800`57a20f2b     nt!KiDispatchInterrupt+0xd5
05 ffffa586`8acff4a0 fffff800`57a1aad1     nt!KiDpcInterruptBypass+0x1b
06 ffffa586`8acff4d0 fffff800`57d90392     nt!KiInterruptDispatchNoLockNoEtw+0xb1
07 ffffa586`8acff660 fffff800`57d901d3     nt!AlpcpReferenceBlob+0x22
08 ffffa586`8acff6a0 fffff800`57d90e2e     nt!AlpcpLookupMessage+0xd3
09 ffffa586`8acff6f0 fffff800`57925647     nt!AlpcpSendMessage+0x53e
0a ffffa586`8acff810 fffff800`5792b851     nt!AlpciSendDeferredMessageBeforeWait+0xe7
0b ffffa586`8acff8a0 fffff800`57a2d505     nt!NtWaitForWorkViaWorkerFactory+0x681
0c ffffa586`8acffa70 00007ffe`756335a4     nt!KiSystemServiceCopyEnd+0x25
0d 000000ab`f597f588 00007ffe`755c537e     ntdll!NtWaitForWorkViaWorkerFactory+0x14
0e 000000ab`f597f590 00007ffe`7437257d     ntdll!TppWorkerThread+0x2ee
0f 000000ab`f597f870 00007ffe`755eaa48     KERNEL32!BaseThreadInitThunk+0x1d
10 000000ab`f597f8a0 00000000`00000000     ntdll!RtlUserThreadStart+0x28
```

Figure 89: Ejecución del comando k

6.3.5. OTROS

Estos comandos, que no caen en las clasificaciones anteriores, también son fundamentales para profundizar y recopilar más información cuando estamos depurando el kernel de Windows.

- **dt**: Se utiliza para mostrar una determinada estructura de datos. Mediante este comando se pueden conocer más en detalle las estructuras opacas del kernel.

```
kd> dt _ETHREAD
ntdll!_ETHREAD
   +0x000 Tcb              : _KTHREAD
   +0x480 CreateTime       : _LARGE_INTEGER
   +0x488 ExitTime         : _LARGE_INTEGER
   +0x488 KeyedWaitChain   : _LIST_ENTRY
   +0x498 PostBlockList    : _LIST_ENTRY
   +0x498 ForwardLinkShadow : Ptr64 Void
   +0x4a0 StartAddress     : Ptr64 Void
   +0x4a8 TerminationPort  : Ptr64 _TERMINATION_PORT
   +0x4a8 ReaperLink       : Ptr64 _ETHREAD
   +0x4a8 KeyedWaitValue   : Ptr64 Void
   +0x4b0 ActiveTimerListLock : Uint8B
   +0x4b8 ActiveTimerListHead : _LIST_ENTRY
   +0x4c8 Cid              : _CLIENT_ID
   +0x4d8 KeyedWaitSemaphore : _KSEMAPHORE
   +0x4d8 AlpcWaitSemaphore : _KSEMAPHORE
   +0x4f8 ClientSecurity   : _PS_CLIENT_SECURITY_CONTEXT
   +0x500 IrpList          : _LIST_ENTRY
   +0x510 TopLevelIrp      : Uint8B
   +0x518 DeviceToVerify   : Ptr64 _DEVICE_OBJECT
   +0x520 Win32StartAddress : Ptr64 Void
   +0x528 ChargeOnlySession : Ptr64 Void
   +0x530 LegacyPowerObject : Ptr64 Void
   +0x538 ThreadListEntry  : _LIST_ENTRY
   +0x548 RundownProtect   : _EX_RUNDOWN_REF
   +0x550 ThreadLock       :  EX PUSH LOCK
```

Figure 90: Ejecución del comando dt

- **x**: Se utiliza para examinar el contenido de la memoria en todas las direcciones que pertenecen a un determinado módulo. Así, por ejemplo, se pueden examinar los contenidos de bibliotecas como "ntdll.dll".

```
kd> x ntdll!*
00007ffe`755da9cc ntdll!RtlpHpVaMgrRangeCreate (void)
00007ffe`7560dd60 ntdll!RtlFindLastBackwardRunClear (void)
00007ffe`755ee270 ntdll!EtwDeliverDataBlock (void)
00007ffe`75616670 ntdll!RtlpInitializeStaticCriticalSection (void)
00007ffe`755dc8a0 ntdll!RtlpTpWorkCallback (void)
00007ffe`75638eeb ntdll!RtlUnicodeStringToOemString$fin$0 (void)
00007ffe`7561aaf0 ntdll!RtlpQueryExtendedInformationHeap (void)
00007ffe`75637d54 ntdll!TppCleanupGroupMemberInitialize$fin$1 (void)
00007ffe`75637d2c ntdll!TppCleanupGroupMemberInitialize$fin$0 (void)
00007ffe`75637dec ntdll!TppCleanupGroupMemberInitialize$fin$3 (void)
00007ffe`75637d71 ntdll!TppCleanupGroupMemberInitialize$fin$2 (void)
00007ffe`755be394 ntdll!SbpDetermineDllContext (void)
00007ffe`755e1610 ntdll!LdrEnumerateLoadedModules (void)
00007ffe`755c5fbc ntdll!TppCallbackCheckThreadAfterCallback (void)
00007ffe`75597c50 ntdll!LdrpVerifyAlternateResourceModuleEx (void)
00007ffe`755c19ac ntdll!LdrpLogDllStateEx2 (void)
00007ffe`755bf0b0 ntdll!RtlGetVersion (void)
00007ffe`7561d500 ntdll!RtlCompareString (void)
00007ffe`755f0fdc ntdll!RtlpCreateWnfUserSubscription (void)
00007ffe`755a1eac ntdll!LdrpConvertLangFallbackListToMultiSz (void)
00007ffe`756b2775 ntdll!RtlpHpSegGetDescriptorValidateSafe$filt$0 (void)
00007ffe`755f7d1c ntdll!RtlWideCharArrayCopyStringWorker (void)
00007ffe`756799c4 ntdll!RtlWideCharArrayCopyStringWorker (void)
00007ffe`755977e0 ntdll!LdrResGetRCConfig (void)
00007ffe`75639ad2 ntdll!TppCritSetThread$fin$3 (void)
00007ffe`755ed880 ntdll!EtwpGetTimeZoneInformation (void)
00007ffe`7560fb28 ntdll!RXactpCommit (void)
00007ffe`75638109 ntdll!TppWorkerThread$filt$11 (void)
```

Figure 91: Ejecución del comando x

- **u**: Se utiliza para desensamblar la memoria en una dirección específica y ver las instrucciones de código ensamblador que se ejecutan, pudiendo comprender mejor las operaciones internas de las funciones.

```
kd> u ntdll!NtCreateFile
ntdll!NtCreateFile:
00007ffe`756303e0 4c8bd1          mov     r10,rcx
00007ffe`756303e3 b855000000      mov     eax,55h
00007ffe`756303e8 f604250803fe7f01 test    byte ptr [SharedUserData+0x308 (00000000`7ffe0308)],1
00007ffe`756303f0 7503            jne     ntdll!NtCreateFile+0x15 (00007ffe`756303f5)
00007ffe`756303f2 0f05            syscall
00007ffe`756303f4 c3              ret
00007ffe`756303f5 cd2e            int     2Eh
00007ffe`756303f7 c3              ret
```

Figure 92: Ejecución del comando u

6.4. SCRIPTING

Existen diferentes formas de mejorar las capacidades de WinDbg al automatizar todas aquellas acciones repetitivas, que se lleven a cabo como parte en el proceso de reversing del sistema operativo. A continuación, se enumeran tres de las que disponemos:

6.4.1. SCRIPTS CLÁSICOS

La forma más obvia de automatización es mediante la escritura de scripts clásicos de WinDbg, que consisten en una serie de comandos básicos de consulta y algunos comandos de flujo de control.

Para desarrollar estos scripts se pueden concatenar, en una única línea, los comandos necesarios dentro de la consola de WinDbg usando el carácter ";" como separador, o ponerlos en un archivo externo con extensión ".wds" y cargarlo.

Como ejemplo, en la siguiente imagen se puede ver la estructura básica de un script de este tipo, aunque existen más ejemplos en los recursos adicionales del módulo.

```
kd> $$ Hello World script; .block { .printf "Hello World!\n" }
Hello World!
kd> $$><C:\Malware\Hello_World_WinDbg_Classic.wds
Hello World!
```

Figure 93: Script clásico de WinDbg

Cabe mencionar, que la sintaxis de scripting de WinDbg puede ser complicada al intentar realizar acciones más avanzadas, además de que no se dispone de variables, por lo que se suelen preferir otras opciones.

6.4.2. JAVASCRIPT

En las versiones más recientes de WinDbg, se integra un motor de scripting JavaScript, lo que ofrece la posibilidad de automatizar tareas de depuración y análisis de datos de una manera más eficiente al utilizar este lenguaje, así como su correspondiente estructura de clases.

JavaScript Copy

```
// WinDbg JavaScript sample
// Says Hello World!

// Code at root will be run with .scriptrun and .scriptload
host.diagnostics.debugLog("***> Hello World! \n");

function sayHi()
{
   //Say Hi
   host.diagnostics.debugLog("Hi from JavaScript! \n");
}
```

Figure 94: Extensión JavaScript para WinDbg

Una vez se ha creado el archivo de javascript .js, se puede hacer uso de los comandos .scriptload, .scriptunload y .scriptrun, entre otros, para gestionar las extensiones que se desarrollen.

```
kd> .scriptload C:\Malware\Hello_World_WinDbg_JavaScript.js
***> Hello World!
JavaScript script successfully loaded from 'C:\Malware\Hello_World_WinDbg_JavaScript.js'
kd> dx Debugger.State.Scripts.Hello_World_WinDbg_JavaScript.Contents.sayHi()
Hi from JavaScript!
Debugger.State.Scripts.Hello_World_WinDbg_JavaScript.Contents.sayHi()
kd> .scriptrun C:\Malware\Hello_World_WinDbg_JavaScript.js
***> Hello World!
JavaScript script successfully loaded from 'C:\Malware\Hello_World_WinDbg_JavaScript.js'
kd> .scriptunload C:\Malware\Hello_World_WinDbg_JavaScript.js
JavaScript script unloaded from 'C:\Malware\Hello_World_WinDbg_JavaScript.js'
```

Figure 95: Ejecución de una extensión de JavaScript para WinDbg

6.4.3. PYTHON

Finalmente, se tendría otra posibilidad, el uso del lenguaje Python y en concreto de la librería PyKd, una herramienta que sirve como un wrapper entre WinDbg y Python.

El valor de esta herramienta es que, aunque la documentación es difícil de entender al no estar traducida, el proyecto está mantenido y por su sintaxis es una opción más idónea para escribir scripts avanzados, pudiendo aprovechar la flexibilidad y capacidad del lenguaje.

PyKd proporciona una interfaz de programación de aplicaciones (API) que permite, a quien está depurando el sistema operativo, interactuar con los comandos y funciones de depuración de WinDbg.

```
kd> !pip install C:\Malware\pykd_ext_2.0.0.25\pykd\pykd-0.3.4.15-cp39-none-win_amd64.whl
Processing c:\malware\pykd_ext_2.0.0.25\pykd\pykd-0.3.4.15-cp39-none-win_amd64.whl
Installing collected packages: pykd
Successfully installed pykd-0.3.4.15
WARNING: You are using pip version 20.2.3; however, version 24.0 is available.
You should consider upgrading via the 'C:\Program Files\WindowsApps\Microsoft.WinDbg_1.2402.24001.0_x64__8wekyb3d8bbwe\amd64\Eng
kd> .load C:\Malware\pykd_ext_2.0.0.25\x64\pykd.dll
kd> !py
Python 3.9.0 (tags/v3.9.0:9cf6752, Oct  5 2020, 15:34:40) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> import pykd
>>> print(pykd.dbgCommand("version"))
Windows 10 Kernel Version 22621 MP (1 procs) Free x64
Product: WinNt, suite: TerminalServer SingleUserTS Personal
Edition build lab: 22621.1.amd64fre.ni_release.220506-1250
Kernel base = 0xfffff800`57600000 PsLoadedModuleList = 0xfffff800`58213130
Debug session time: Tue Jun  4 23:00:28.507 2024 (UTC + 1:00)
System Uptime: 0 days 0:17:35.323
Remote KD: KdSrv:Server=@{<Local>},Trans=@{NET:Port=50000,Key=2541m38eqo4p3.3tg0q6u40lm3c.25lnddtktl2gf.1xk8jkgw0y5ls,Target}

Microsoft (R) Windows Debugger Version 10.0.27553.1004 AMD64
Copyright (c) Microsoft Corporation. All rights reserved.
```

Figure 96: Configuración de la librería PyKd

```
kd> !py C:\Malware\Hello_World_WinDbg_Pykd.py
Hola Mundo
**** NT ACTIVE PROCESS DUMP ****
PROCESS ffff958f5e0a2040
    SessionId: none  Cid: 0004    Peb: 00000000  ParentCid: 0000
    DirBase: 001ae002  ObjectTable: ffffb98b4006eec0  HandleCount: 2979.
    Image: System

PROCESS ffff958f5e1d0080
    SessionId: none  Cid: 0040    Peb: 00000000  ParentCid: 0004
    DirBase: 10004e002  ObjectTable: ffffb98b4008b8c0  HandleCount:   0.
    Image: Registry

PROCESS ffff958f6013d040
    SessionId: none  Cid: 0160    Peb: a9fc375000  ParentCid: 0004
    DirBase: 101ee4002  ObjectTable: ffffb98b409c3880  HandleCount:  58.
    Image: smss.exe

PROCESS ffff958f6021b140
    SessionId: 0  Cid: 01f8    Peb: 946a421000  ParentCid: 01dc
    DirBase: 01bb8002  ObjectTable: ffffb98b40b78bc0  HandleCount: 516.
    Image: csrss.exe
```

Figure 97: Ejecución de un script de PyKd

7. ROOTKITS

Una vez que ya se ha cargado Windows, irán ejecutándose, uno a uno, los drivers existentes en nuestro equipo. Entre estos, pudieran existir drivers cuya funcionalidad fuese maliciosa, drivers desarrollados por nosotros mismos para incluir nuevas funcionalidades en el sistema, drivers de dispositivos de red, etc.

De los distintos tipos de drivers que existen tenemos que mencionar aquí los drivers de modo kernel, también conocidos como controladores de bajo nivel, que son programas que se ejecutan en el núcleo del sistema operativo (Ring 0), lo que les permite acceder directamente al kernel y a los recursos hardware, pudiendo modificar cualquier estructura y comportamiento original de Windows.

Esto último, que es crítico, de lo cual se aprovecha el malware, es lo que nos lleva a introducir este tema y a mencionar que nos encontramos con el elemento concreto que debemos asociar a un Rootkit, un Driver de Modo Kernel, al igual que en su momento asociamos una Aplicación UEFI a un Bootkit moderno.

7.1. INTRODUCCIÓN A LOS ROOTKITS

Los rootkits, como sus análogos más complejos, los bootkits, son un tipo de malware que está diseñado para permanecer oculto, aunque el principal objetivo de estos no es infectar el arranque del sistema operativo, sino el kernel. Este malware hace uso de diversas técnicas para mantener el control de un sistema de forma encubierta, ocultar la presencia de procesos que indiquen una actividad sospechosa, archivos maliciosos, conexiones de red a servidores externos, y otras operaciones/capacidades que adquiere a través de obtener acceso al kernel de nuestro sistema, y en definitiva al poder hacer uso de recursos fundamentales operando en el nivel más bajo que tiene un equipo.

Históricamente, los rootkits tuvieron un auge significativo entre mediados de los 2000s y mediados de los 2010s, un periodo que se asocia a "la era dorada de los rootkits". Durante este tiempo, varios malware, entre ellos Rustock, TDSS, ZeroAccess y Sinowal infectaron miles de sistemas en todo el mundo sin ser detectados, explotando la falta de soluciones y mecanismos de seguridad en aquellas versiones de Windows (XP y 7). Sin embargo, en la actualidad el desarrollo de rootkits para Windows modernos (10 y 11), el desarrollo de drivers de modo kernel para Windows, ha disminuido considerablemente debido a la implementación de medidas de seguridad avanzadas como PatchGuard y DSE, ya comentadas en el módulo, aunque esto no ha frenado a los actores maliciosos que siguen desarrollando nuevas versiones de este tipo de malware.

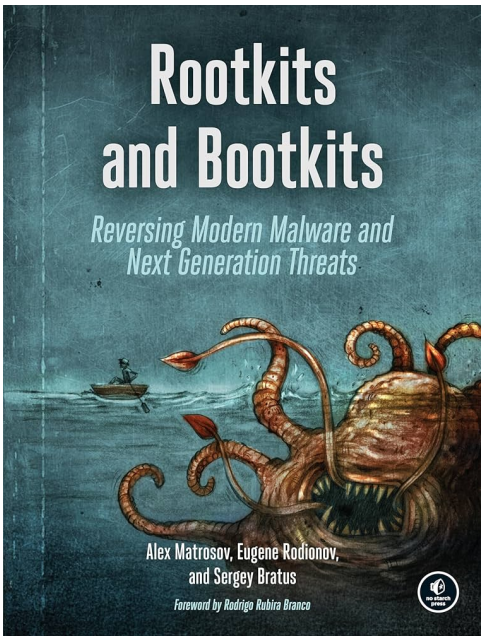


Figure 98: Volumen de referencia sobre Rootkits y amenazas avanzadas

7.2. DESARROLLO DE MALWARE (BASE)

7.2.1. ENTORNO DE DESARROLLO (WDK)

En primer lugar, para llevar a cabo el desarrollo sin dificultades, hay que crear un entorno de trabajo, y la mejor opción es optar por Visual Studio community, un IDE cuya configuración es sencilla y el cual aporta muchas librerías necesarias.

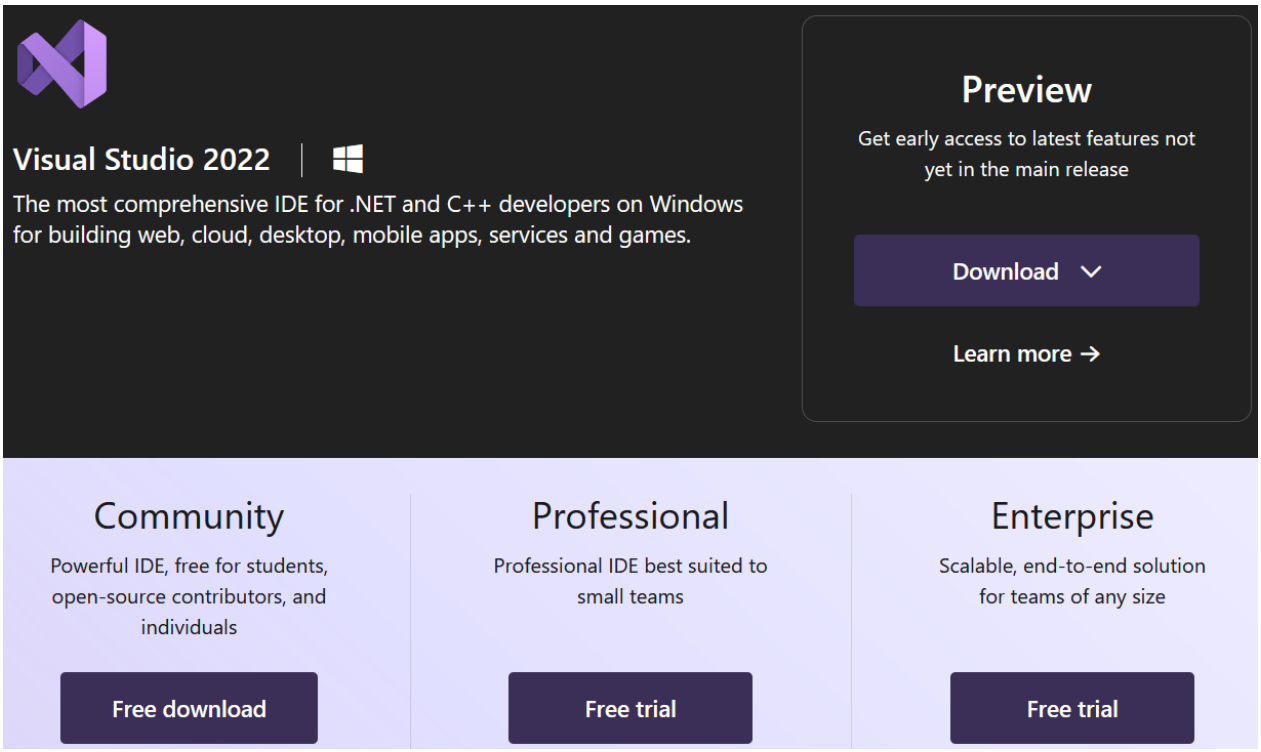


Figure 99: Descarga de Visual Studio

Al ejecutar el instalador aparece una pantalla para seleccionar los elementos que se desean instalar, y en referencia al desarrollo de este malware contemplamos los siguientes:

- Elemento "Desarrollo para el escritorio con C++".

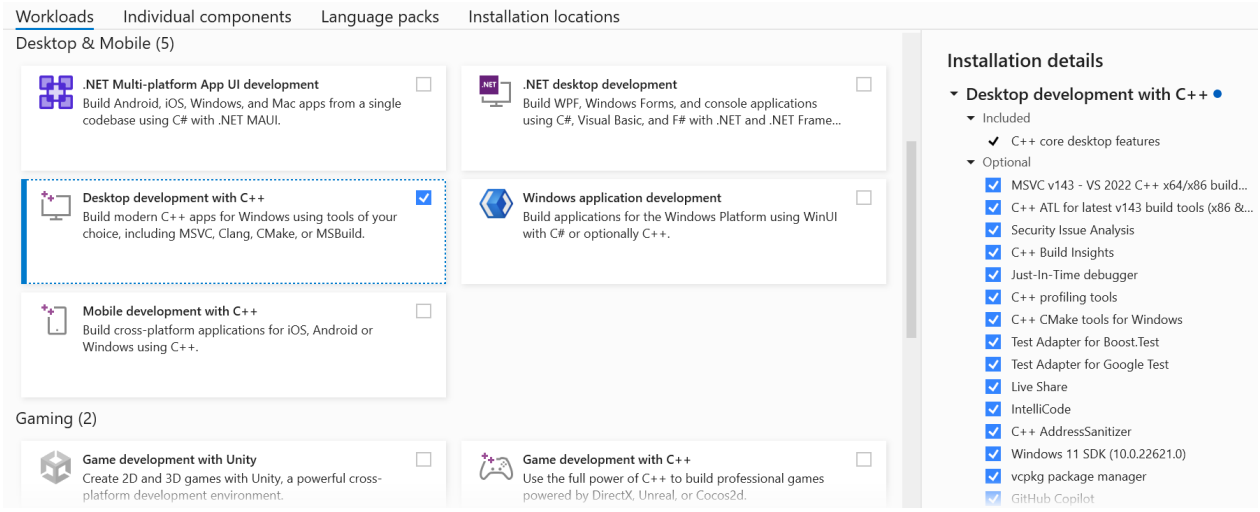


Figure 100: Desarrollo para el escritorio con C++

- Componente individual "Bibliotecas con mitigaciones de Spectre (más recientes)".

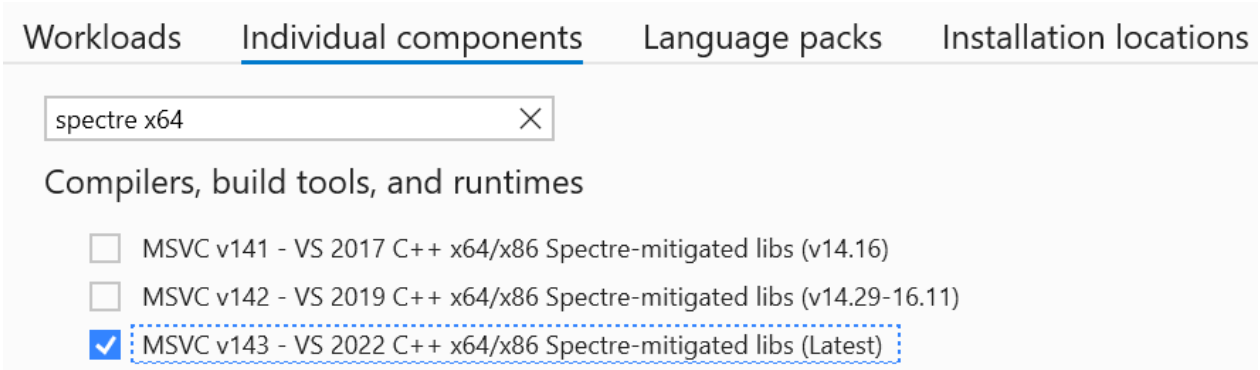


Figure 101: Componente Mitigaciones de Spectre

Después de seleccionar estos elementos, se puede proceder con la instalación de Visual Studio y, una vez termine esta, para finalizar, instalar el kit de desarrollo de software de Microsoft, Windows SDK, y el kit de desarrollo para drivers, Windows Driver Kit (WDK), que en verdad es el componente que utilizaremos para desarrollar el driver de modo kernel (el rootkit) marcando la opción de añadir la extensión de Visual Studio.

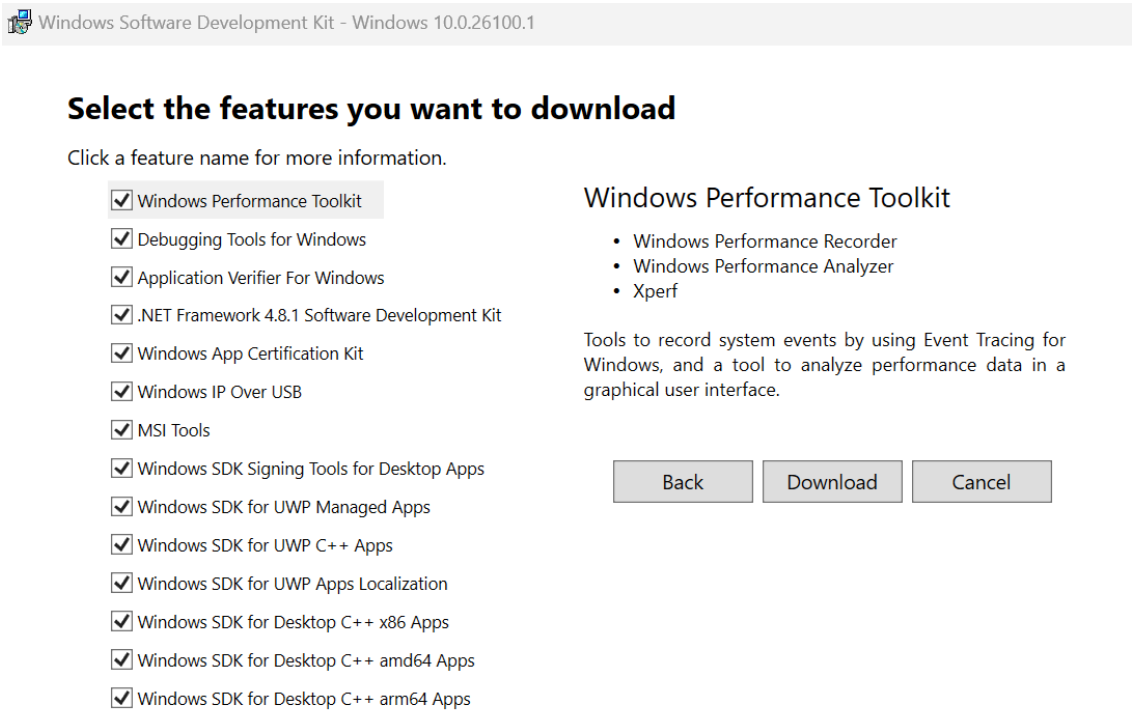


Figure 102: Instalación de Windows SDK

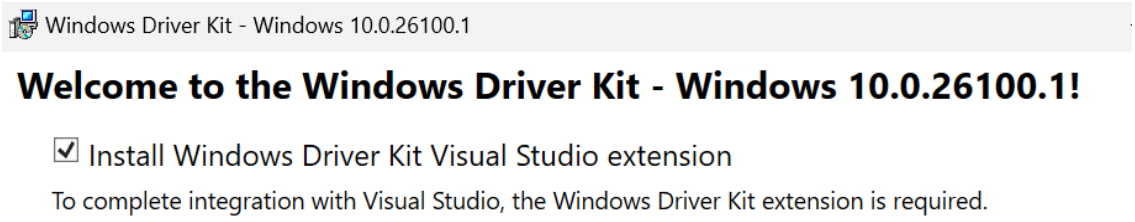


Figure 103: Instalación de Windows Driver Kit y extensión de VS

Una vez hecho esto, habremos preparado el entorno, aunque nos queda una última operación para trabajar con comodidad. Debemos tener en cuenta que existen protecciones de seguridad habilitadas por defecto en nuestra máquina, las cuáles son un impedimento si queremos realizar pruebas con cierta rapidez, entre ellas "Driver Signature Enforcement (DSE)", que como comentamos en un tema pasado podemos deshabilitar sin ningún problema al estar en un entorno controlado.

7.2.2. MI PRIMER DRIVER DE MODO KERNEL

Como punto de partida, y también para comprobar que se ha creado el entorno correctamente, desarrollaremos y compilaremos un driver sencillo. Para hacer esto, una vez abrimos Visual Studio, creamos un proyecto con la plantilla "Kernel Mode Driver, Empty (KMDF)".

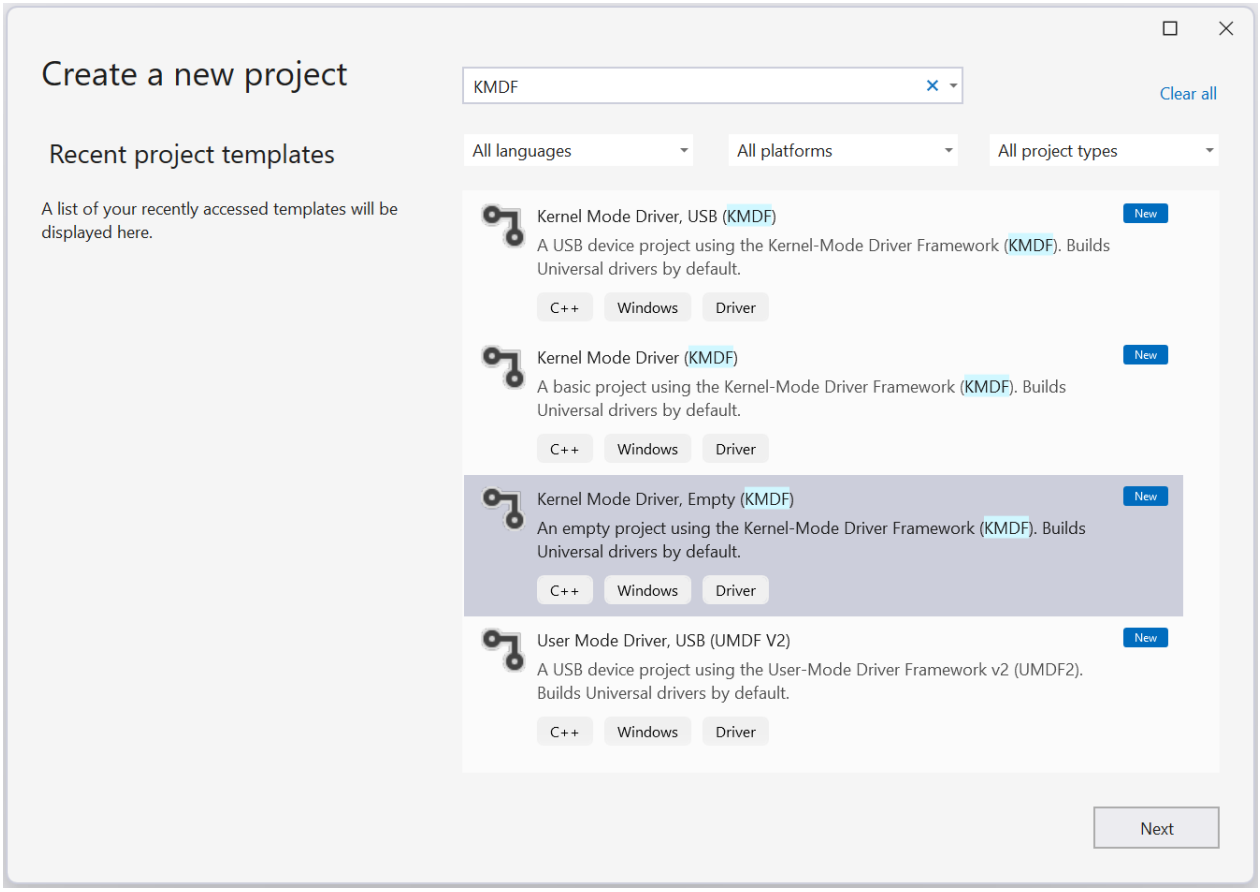


Figure 104: Crear proyecto con plantilla "Kernel Mode Driver, Empty (KMDF)"

Seleccionada esta opción, aparece una nueva pantalla para introducir el nombre del driver. En este caso se llamará "KMDF Driver1 Hola Mundo".

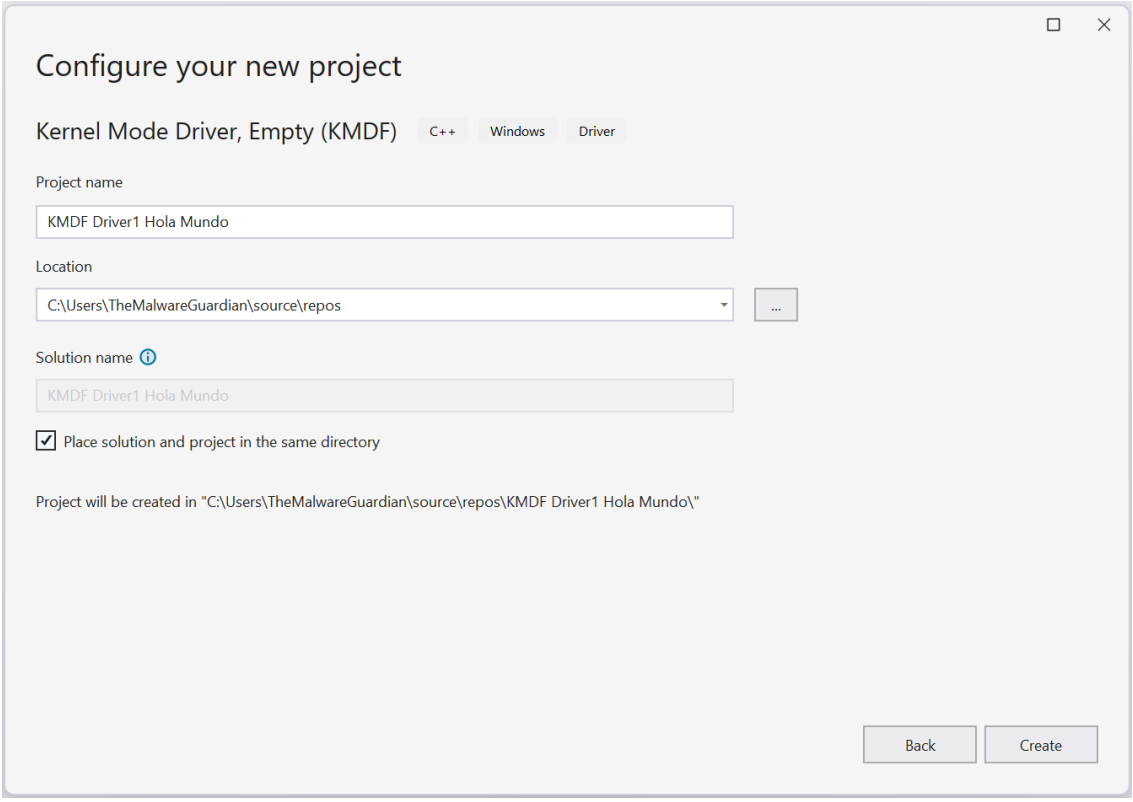


Figure 105: Nombre del proyecto Kernel Mode Driver, Empty (KMDF)

Una vez creado el proyecto se deben añadir los archivos .c y .h correspondientes, los cuales contienen el código fuente.

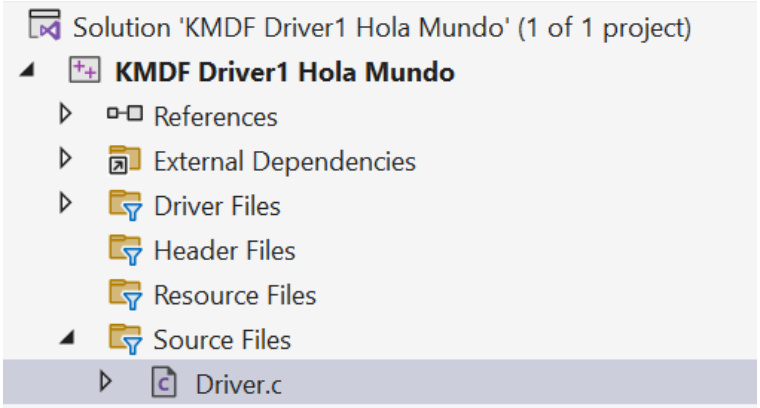


Figure 106: Archivo driver.c del driver Hola Mundo

Para lo que será el primer contacto con el desarrollo de drivers, se puede utilizar el siguiente código, cuyas partes ya se encuentran comentadas, facilitando el entendimiento una vez se copie el contenido en Visual Studio.

Driver.c

KMDF Driver1 Hola Mundo | (Global Scope)

```
1  // ------------------------------------------------------
2  // Nombre: KMDF Driver 1 Hola Mundo
3  // Proyecto en Visual Studio: Plantilla -> Kernel Mode Driver, Empty (KMDF)
4  //
5  // Código Fuente:
6
7  // Librería principal que incluye las definiciones necesarias para el desarrollo de drivers
8  #include <ntddk.h>
9
10 // Función de salida, ejecutada cuando el driver se "descarga" (cuando se pare el servicio)
11 VOID
12 DriverUnload(
13     // Puntero a una estructura DRIVER_OBJECT que representa la propia imagen del driver en
       el kernel del sistema operativo
14     _In_ PDRIVER_OBJECT DriverObject
15 )
16 {
17     // Se evita la advertencia de "parámetro DriverObject no utilizado"
18     UNREFERENCED_PARAMETER(DriverObject);
19     // Se imprime una cadena de texto cuando se para el servicio
20     DbgPrint("Rootkit POC descargando... Se ha parado el servicio");
21 }
22
23 // Punto de entrada de un driver en el modo kernel de Windows, es como un main en c o python
24 NTSTATUS
25 DriverEntry(
26     // Puntero a una estructura DRIVER_OBJECT que representa la propia imagen del driver en
       el kernel del sistema operativo
27     _In_ PDRIVER_OBJECT DriverObject,
28     // Puntero a una estructura UNICODE_STRING, que contiene la ruta del registro del driver
       como una cadena de caracteres unicode, en la que se indica la localización del driver
       en el registro de Windows
29     _In_ PUNICODE_STRING RegistryPath
30 )
31 {
32     // Se evita la advertencia de "parámetro RegistryPath no utilizado"
33     UNREFERENCED_PARAMETER(RegistryPath);
34     // Se establece la rutina de salida
35     DriverObject->DriverUnload = DriverUnload;
36     // Se imprime una cadena de texto para indicar que el driver se ha cargado
37     DbgPrint("Rootkit POC nos saluda: Hola mundo!!!");
38     // Indica que el driver se ha iniciado correctamente
39     return STATUS_SUCCESS;
40 }
41 // ------------------------------------------------------
```

Figure 107: Código fuente del driver Hola Mundo

Una vez copiado este ejemplo al fichero Driver.c, solo queda compilar la solución para la plataforma sobre la que se esté desarrollando y crear el servicio asociado.

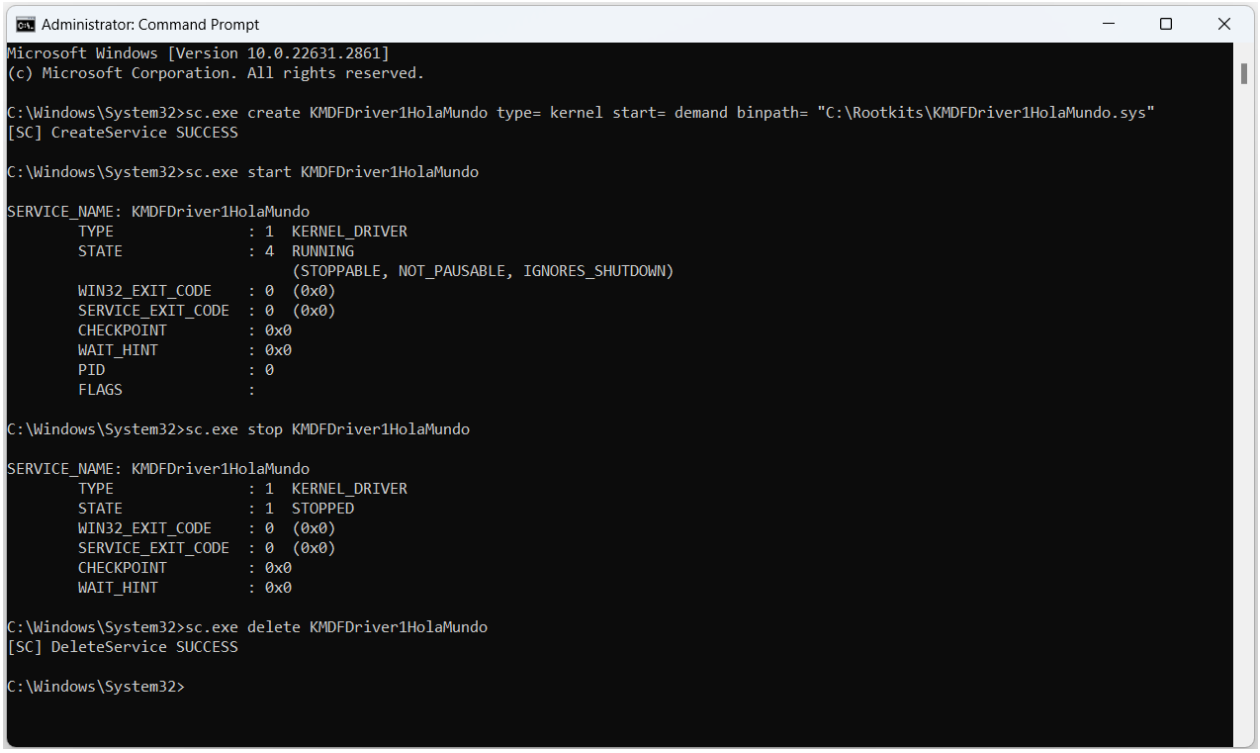


Figure 108: Creación del servicio asociado al driver Hola Mundo

Al iniciar y detener el servicio se pueden observar los mensajes de debugging, a nivel de kernel, mediante alguna herramienta como DebugView de la suite Sysinternals, pudiendo analizarlos para observar el comportamiento del driver según se utiliza el equipo.

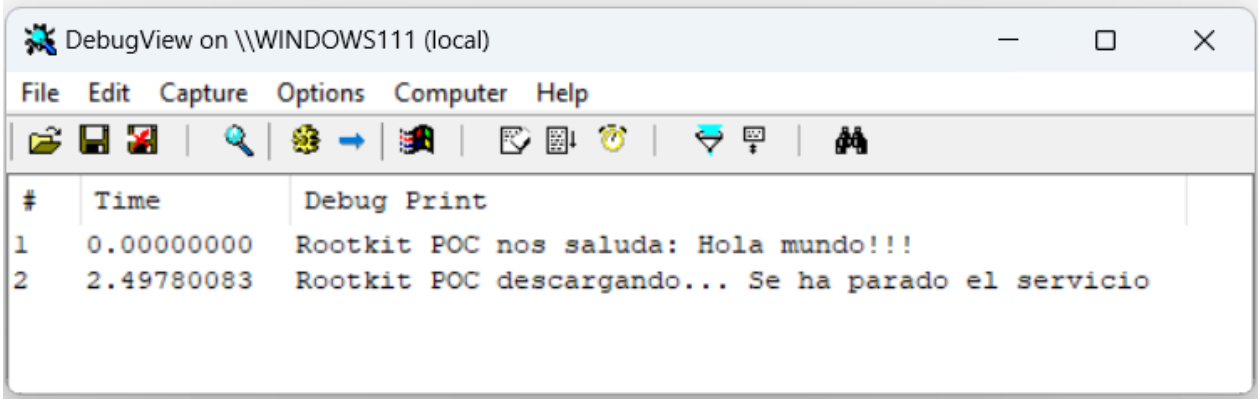


Figure 109: Mensajes de debug del driver Hola Mundo

7.3. TÉCNICAS APLICADAS POR EL MALWARE

Los rootkits, al operar a nivel de kernel, emplean técnicas sofisticadas y/o tradicionales que dificultan enormemente su detección y eliminación por parte de antivirus y otras soluciones de seguridad. Estas técnicas permiten a los rootkits interceptar y manipular las llamadas a funciones críticas del sistema operativo, alterando su comportamiento normal y ocultando sus propias actividades maliciosas. Para contrarrestar estas amenazas, las herramientas orientadas a identificarlas han evolucionado, desarrollándose en respuesta a las técnicas empleadas por el malware para detectar incongruencias y modificaciones a bajo nivel en las estructuras del kernel.

En esta sección, nos centraremos en tres técnicas que fueron principales en la "era dorada", las cuales permiten a los rootkits modificar el núcleo de Windows: Interrupt Descriptor Table (IDT) Hooking, Direct Kernel Object Manipulation (DKOM) y System Service Descriptor Table (SSDT) Hooking. Cada una de estas técnicas tiene su propio conjunto de pasos y aquí los exploraremos de forma manual para obtener una nueva visión de estructuras y elementos no documentados, elementos cuya comprensión también es esencial para desarrollar métodos de detección identifcando incongruencias a bajo nivel que indiquen la modificación del kernel.

7.3.1. INTERRUPT DESCRIPTOR TABLE (IDT) HOOKING

Para entender en qué consiste esta técnica, primero tenemos que introducir la tabla de descriptores de interrupciones o IDT, una tabla que permite al procesador administrar las interrupciones hardware y software, y para la cual se tiene un registro propio, IDTR.

La IDT es una tabla de estructuras de datos que contiene información sobre todas las interrupciones disponibles en el sistema operativo, en concreto cada entrada corresponde con una rutina de servicio de interrupción (Interrupt Service Routine o ISR), que le permite saber a la CPU la rutina del kernel a ejecutar. Algo importante a mencionar es que estas entradas tienen un tamaño de 8 o 16 bytes según la arquitectura.

Cuando se produce una interrupción, el procesador consulta la IDT para obtener información sobre cómo manejar la interrupción y, acto seguido, busca la entrada correspondiente utilizando la dirección de memoria asociada para encontrar la ISR que posteriormente ejecuta.

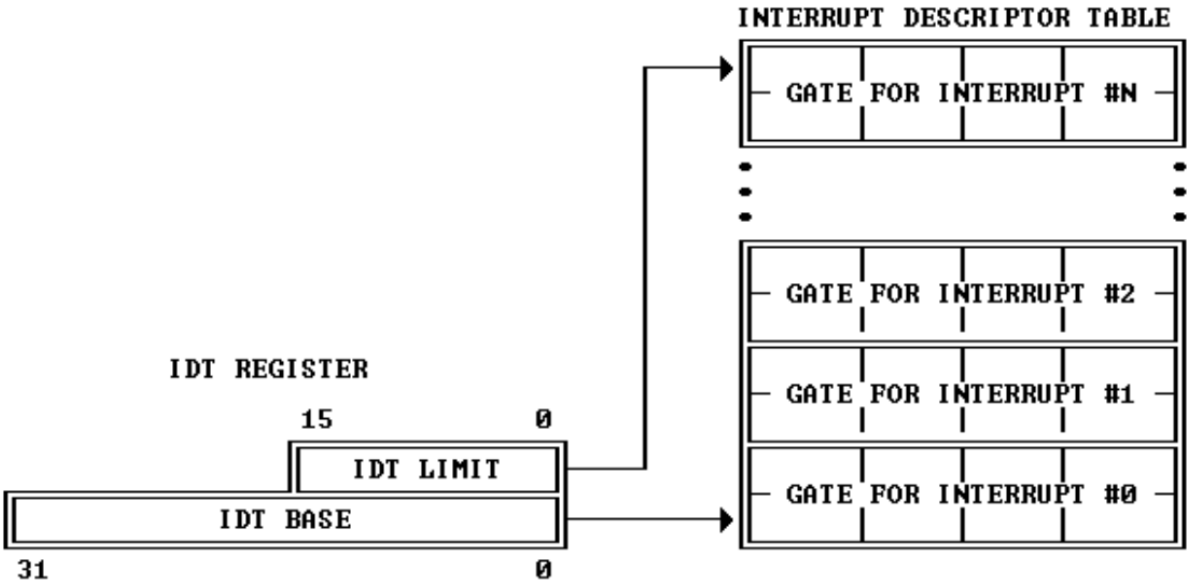


Figure 110: Interrupt Descriptor Table (IDT)

El objetivo de los rootkits al utilizar la técnica de IDT Hooking, es el de modificar la tabla IDT para redirigir las interrupciones a una función de manejo de interrupciones personalizada, en lugar de a la función original. De esta forma, se puede controlar el comportamiento del sistema operativo y, por ejemplo, interceptar las interrupciones al pulsar el teclado o los movimientos del ratón, registrando información confidencial.

Para llevar a cabo esta técnica, y sirviendo como base para el inicio de un keylogger, un programa que grabe las pulsaciones de teclas que realice un usuario en el teclado, primeramente, se tiene que localizar la dirección de la IDT, para posteriormente listar sus entradas y encontrar la función "i8042prt!I8042KeyboardInterruptService".

```
kd> r idtr
idtr=fffff80053661000
kd> !idt

Dumping IDT: fffff80053661000

00: fffff800580f4100 nt!KiDivideErrorFaultShadow
01: fffff800580f4180 nt!KiDebugTrapOrFaultShadow    Stack = 0xFFFFF800536659D0
02: fffff800580f4240 nt!KiNmiInterruptShadow    Stack = 0xFFFFF800536657D0
03: fffff800580f42c0 nt!KiBreakpointTrapShadow
04: fffff800580f4340 nt!KiOverflowTrapShadow
05: fffff800580f43c0 nt!KiBoundFaultShadow
06: fffff800580f4440 nt!KiInvalidOpcodeFaultShadow
07: fffff800580f44c0 nt!KiNpxNotAvailableFaultShadow
08: fffff800580f4540 nt!KiDoubleFaultAbortShadow    Stack = 0xFFFFF800536653D0
09: fffff800580f45c0 nt!KiNpxSegmentOverrunAbortShadow
0a: fffff800580f4640 nt!KiInvalidTssFaultShadow
0b: fffff800580f46c0 nt!KiSegmentNotPresentFaultShadow
0c: fffff800580f4740 nt!KiStackFaultShadow
0d: fffff800580f47c0 nt!KiGeneralProtectionFaultShadow
0e: fffff800580f4840 nt!KiPageFaultShadow
10: fffff800580f48c0 nt!KiFloatingErrorFaultShadow
11: fffff800580f4940 nt!KiAlignmentFaultShadow
12: fffff800580f49c0 nt!KiMcheckAbortShadow Stack = 0xFFFFF800536655D0
13: fffff800580f4ac0 nt!KiXmmExceptionShadow
14: fffff800580f4b40 nt!KiVirtualizationExceptionShadow
15: fffff800580f4bc0 nt!KiControlProtectionFaultShadow
1f: fffff800580f4c40 nt!KiApcInterruptShadow
```

Figure 111: Entradas de la tabla IDT

```
70: fffff800580f5640 i8042prt!I8042MouseInterruptService (KINTERRUPT ffffce80c0d14780)

71: fffff800580f5648 pci!ExpressRootPortMessageRoutine (KINTERRUPT ffffce80c0dca640)

72: fffff800580f5650 pci!ExpressRootPortMessageRoutine (KINTERRUPT ffffce80c0b88c80)

73: fffff800580f5658 pci!ExpressRootPortMessageRoutine (KINTERRUPT ffffce80c0b883c0)

74: fffff800580f5660 pci!ExpressRootPortMessageRoutine (KINTERRUPT ffffce80c0b89a00)

75: fffff800580f5668 vmci!DllInitialize+0x1c10 (KINTERRUPT ffffce80c0b89280)

76: fffff800580f5670 USBPORT!USBPORT_InterruptService (KINTERRUPT ffffce80c0d14500)

                     dxgkrnl!DpiFdoLineInterruptRoutine (KINTERRUPT ffffce80c0d14000)

80: fffff800580f56c0 i8042prt!I8042KeyboardInterruptService (KINTERRUPT ffffce80c0d148c0)
```

Figure 112: Interrupción i8042prt!I8042KeyboardInterruptService

Una vez se localiza la función responsable de manejar las interrupciones del teclado, se puede acceder y examinar el contenido de esa entrada de la tabla, que corresponde con una estructura del kernel de tipo _KIDTENTRY64. La ubicación de dicha entrada se obtiene sumando a la dirección de memoria de la IDT, el índice de interrupción multiplicado por 0x10 ya que cada una ocupa 16 bytes.

```
kd> dt _kidtentry64 (idtr + (0x80+0x10))
ntdll!_KIDTENTRY64
   +0x000 OffsetLow        : 0x45c0
   +0x002 Selector         : 0x10
   +0x004 IstIndex         : 0y000
   +0x004 Reserved0        : 0y00000 (0)
   +0x004 Type             : 0y01110 (0xe)
   +0x004 Dpl              : 0y00
   +0x004 Present          : 0y1
   +0x006 OffsetMiddle     : 0x580f
   +0x008 OffsetHigh       : 0xfffff800
   +0x00c Reserved1        : 0
   +0x000 Alignment        : 0x580f8e00`001045c0
```

Figure 113: Estructura _KIDTENTRY64 correspondiente a la entrada 0x80 de la IDT

Como se puede ver en la siguiente imagen, cuando establecemos un breakpoint en un contexto ordinario y teniendo en cuenta los datos de la estructura, si se pulsa una tecla en la máquina se llama a la interrupción del teclado. No obstante, modificando los datos, lo cual se puede llevar a cabo desarrollando un driver para dicho cometido, se podría obtener el control de este tipo de interrupciones, implementando así un keylogger funcional para el sistema Windows.

```
kd> bp i8042prt!I8042KeyboardInterruptService
kd> gu
nt!KiCallInterruptServiceRoutine+0x16d:
fffff800`57867d9d f605225dab0001  test    byte ptr [nt!PerfGlobalGroupMask+0x6 (fffff800`5831dac6)],1
kd> gu
nt!KiInterruptSubDispatch+0x11f:
fffff800`57a19f0f f685f3000000ff  test    byte ptr [rbp+0F3h],0FFh
kd> gu
nt!KiInterruptDispatch+0x37:
fffff800`57a1a1d7 488b642410      mov     rsp,qword ptr [rsp+10h]
kd> gu
Breakpoint 0 hit
i8042prt!I8042KeyboardInterruptService:
fffff800`5da92100 4889542410      mov     qword ptr [rsp+10h],rdx
```

Figure 114: Llamada a i8042prt!I8042KeyboardInterruptService

7.3.2. DIRECT KERNEL OBJECT MANIPULATION (DKOM)

Esta es una de las técnicas principales que los rootkits utilizan para ocultar todo tipo de acciones que estén ocurriendo en el equipo, procesos relacionados con malware, archivos, conexiones de red, …, lo cual implica la manipulación de estructuras del kernel en memoria, aprovechándose, en definitiva, de la capacidad de los sistemas operativos para permitir que un código, que corra en modo kernel, acceda directamente a los objetos pudiendo realizar modificaciones.

Como ejemplo, podemos comentar el uso de esta técnica para ocultar un proceso, manipulando la lista que utiliza el kernel para mostrarle al usuario los que están en ejecución.

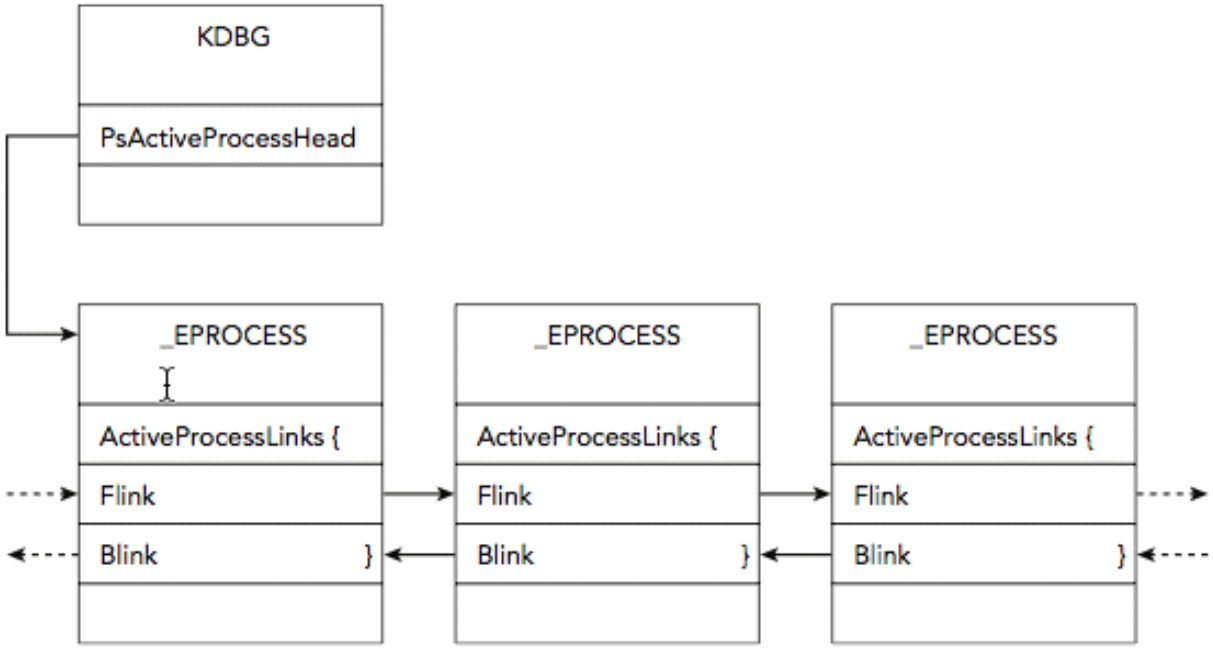


Figure 115: Lista enlazada de procesos activos

Como se observa en la imagen anterior, cada elemento de dicha lista se corresponde con una estructura de tipo EPROCESS, que contiene la información de cada proceso, además de una referencia a los elementos anterior y posterior.

Para ocultar un elemento de esta lista se debe aplicar una modificación al objeto, donde finalmente los elementos anterior y posterior se hagan referencia entre sí, dejando ese proceso en ejecución, pero fuera de cualquier consulta a la lista de procesos.

Primeramente, se tiene que encontrar el proceso a ocultar y consultar su estructura EPROCESS una vez se localiza la dirección en la que se encuentra.

```
kd> !process 0 0 mspaint.exe
PROCESS ffff958f6271b080
    SessionId: 1  Cid: 0af0    Peb: 7cd60f3000  ParentCid: 09ec
    DirBase: 1564d6002  ObjectTable: ffffb98b461d3540  HandleCount: 649.
    Image: mspaint.exe

kd> !process 0xaf0 0
Searching for Process with Cid == af0
PROCESS ffff958f6271b080
    SessionId: 1  Cid: 0af0    Peb: 7cd60f3000  ParentCid: 09ec
    DirBase: 1564d6002  ObjectTable: ffffb98b461d3540  HandleCount: 649.
    Image: mspaint.exe
```

Figure 116: Consulta del proceso mspaint.exe a partir de su PID

```
kd> dt nt!_EPROCESS ffff958f6271b080
   +0x000 Pcb              : _KPROCESS
   +0x438 ProcessLock      : _EX_PUSH_LOCK
   +0x440 UniqueProcessId  : 0x00000000`00000af0 Void
   +0x448 ActiveProcessLinks : _LIST_ENTRY [ 0xffff958f`62f0d4c8 - 0xffff958f`630d94c8 ]
   +0x458 RundownProtect   : _EX_RUNDOWN_REF
   +0x460 Flags2           : 0xd094
   +0x460 JobNotReallyActive : 0y0
   +0x460 AccountingFolded : 0y0
   +0x460 NewProcessReported : 0y1
   +0x460 ExitProcessReported : 0y0
   +0x460 ReportCommitChanges : 0y1
   +0x460 LastReportMemory : 0y0
   +0x460 ForceWakeCharge  : 0y0
   +0x460 CrossSessionCreate : 0y1
   +0x460 NeedsHandleRundown : 0y0
   +0x460 RefTraceEnabled  : 0y0
   +0x460 PicoCreated      : 0y0
   +0x460 EmptyJobEvaluated : 0y0
   +0x460 DefaultPagePriority : 0y101
   +0x460 PrimaryTokenFrozen : 0y1
   +0x460 ProcessVerifierTarget : 0y0
   +0x460 RestrictSetThreadContext : 0y0
   +0x460 AffinityPermanent : 0y0
   +0x460 AffinityUpdateEnable : 0y0
   +0x460 PropagateNode    : 0y0
```

Figure 117: Información de la estructura EPROCESS para el proceso mspaint.exe

Hecha la consulta a esa estructura, como se ve en la siguiente imagen, se tiene que localizar la dirección de la lista de procesos activos "ActiveProcessLinks" teniendo en cuenta el offset, que puede cambiar entre diferentes versiones del sistema operativo.

```
kd> dd ffff958f6271b080+448 L3
ffff958f`6271b4c8  62f0d4c8 ffff958f 630d94c8
```

Figure 118: Dirección de la lista de procesos activos

Posteriormente, siguiendo el mismo procedimiento de consulta para los elementos anterior y siguiente del proceso que se quiere ocultar, a partir de los punteros Flink y Blink se recaba la información restante necesaria. Esto se puede llevar a cabo, recuperando el PID de dichos procesos al restar el offset de la lista y sumar el equivalente a dicho PID.

```
kd> dd ffff958f630d94c8-448+440 L1
ffff958f`630d94c0  00000534
kd> dd ffff958f62f0d4c8-448+440 L1
ffff958f`62f0d4c0  00000878
kd> !process 0534 0
Searching for Process with Cid == 534
PROCESS ffff958f630d9080
    SessionId: 1  Cid: 0534    Peb: 821427d000  ParentCid: 09ec
    DirBase: 18d406002  ObjectTable: ffffb98b452f9800  HandleCount: 382.
    Image: vmtoolsd.exe

kd> !process 0878 0
Searching for Process with Cid == 878
PROCESS ffff958f62f0d080
    SessionId: 0  Cid: 0878    Peb: ed961b1000  ParentCid: 02b8
    DirBase: 14c54f002  ObjectTable: ffffb98b452f7540  HandleCount: 195.
    Image: svchost.exe
```

Figure 119: Información de procesos anterior y siguiente

Finalmente, una vez recabados todos los datos necesarios, se efectuaría la desvinculación del proceso de la lista enlazada, bien manualmente modificando las estructuras, o desarrollando un driver que implemente esa operación.

ImageFileName	Cid	EPROCESS	ActiveProcessLinks	Flink	Blink
vmtoolsd.exe	0534	ffff958f630d9080	ffff958f630d94c	ffff958f6271b4c8	ffff958f628ef4c8
mspaint.exe	0af0	ffff958f6271b080	ffff958f6271b4c8	ffff958f62f0d4c8	ffff958f630d94c8
svchost.exe	0878	ffff958f62f0d080	ffff958f62f0d4c8	ffff958f634a6508	ffff958f6271b4c8

Figure 120: Tabla con información para desvincular un proceso

7.3.3. SYSTEM SERVICE DESCRIPTOR TABLE (SSDT) HOOKING

En esta técnica, la idea es modificar la tabla de descriptores de servicios, otra estructura del kernel que contiene las direcciones de las funciones del sistema operativo, con el objetivo de interceptar las llamadas al sistema (syscalls) y redirigirlas a un código malicioso.

De esta forma, cada vez que un programa llama a una función del sistema operativo, la dirección de la función se buscará en la SSDT y se ejecutará otro código que personaliza el atacante, pudiendo controlar totalmente el comportamiento del equipo.

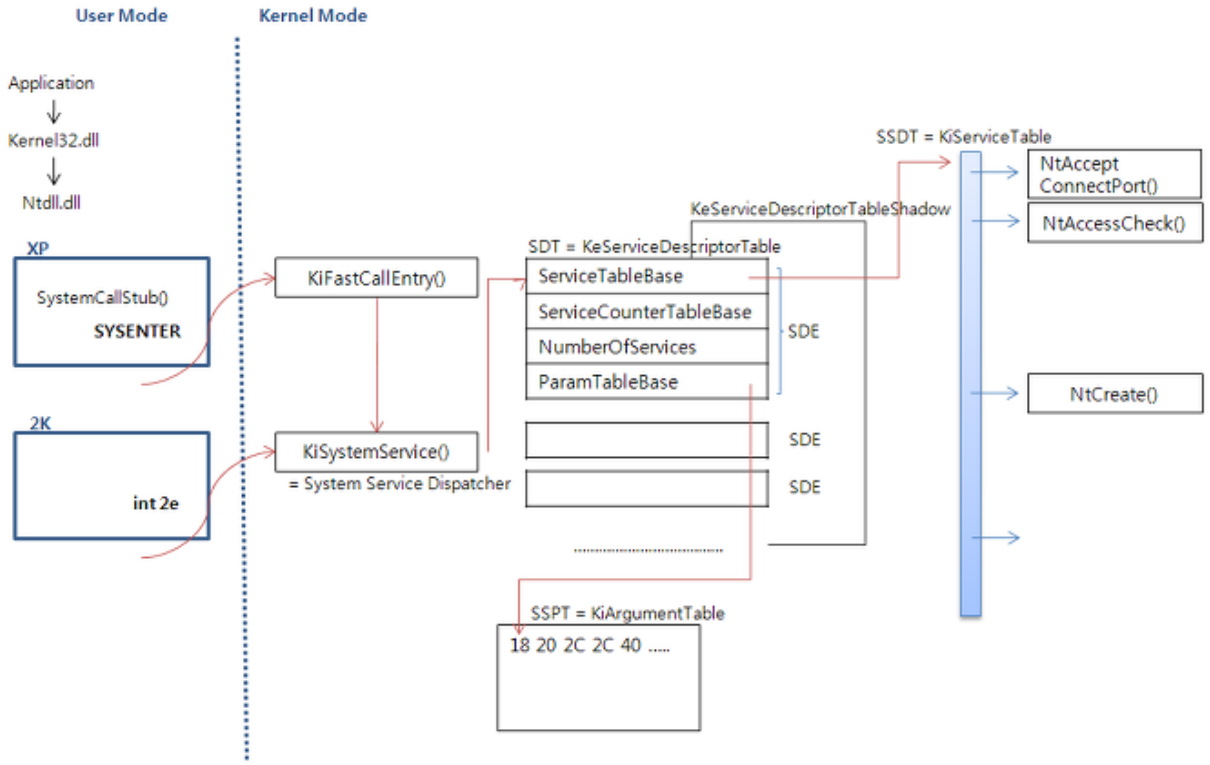


Figure 121: Contexto de la tabla SSDT

Algo importante a mencionar es que esta tabla en los sistemas más modernos de 64 bits contiene direcciones relativas a las rutinas del kernel, y para obtener la dirección absoluta a partir de un determinado desplazamiento, se debe aplicar la siguiente función:

$$RoutineAbsoluteAddress = KiServiceTableAddress + (routineOffset >>> 4)$$

Figure 122: Función para recuperar direcciones absolutas a rutinas del kernel

Como ejemplo, podemos ver la función NTCreateFile utilizada para crear y abrir archivos en el sistema, cuya dirección, en la SSDT, se podría modificar posteriormente para llamar a un código ejecutable personalizado.

```
lkd> .reload /f ntdll.dll
lkd> lm ltn
start             end                 module name
00007ff8`ac1f0000 00007ff8`ac407000   ntdll    ntdll.dll    92B2DF34 (This is a reproducible build file hash, not a timestamp)
fffff801`78600000 fffff801`79647000   nt       ntkrnlmp.exe 8A3A5A16 (This is a reproducible build file hash, not a timestamp)
lkd> u ntdll!ntcreatefile L2
ntdll!NtCreateFile:
00007ff8`ac2903e0 4c8bd1          mov     r10,rcx
00007ff8`ac2903e3 b855000000      mov     eax,55h
```

Figure 123: Función NtCreateFile

Al consultar esta función nos damos cuenta que la llamada al sistema se corresponde con la 0x55 y teniendo en cuenta que los offsets de KiServiceTable tienen un tamaño de 4 bytes, podemos calcular fácilmente el desplazamiento para esa llamada dentro de la tabla de acuerdo a la función introducida anteriormente.

```
lkd> dd /c1 kiservicetable+4*0x55 L1
fffff801`786cfa54  06c45507
lkd> u kiservicetable + (06c45507>>>4) L1
nt!NtCreateFile:
fffff801`78d93e50 4881ec88000000  sub     rsp,88h
```

Figure 124: Desplazamiento de la función NtCreateFile

Siguiendo con la técnica, consultada la estructura de la tabla de descriptores de servicios "KeServiceDescriptorTable" y los valores relativos de las rutinas, bien manualmente o mediante un script que imprima directamente el nombre de las funciones, se busca la entrada correspondiente a NtCreateFile, cuyo offset ahora ya conocemos, para posteriormente modificar esa entrada de la tabla a una función personalizada.

```
lkd> dps nt!keservicedescriptortable L10
fffff801`794018c0  fffff801`786cf900 nt!KiServiceTable
fffff801`794018c8  00000000`00000000
fffff801`794018d0  00000000`000001e6
fffff801`794018d8  fffff801`786d009c nt!KiArgumentTable
fffff801`794018e0  00000000`00000000
fffff801`794018e8  00000000`00000000
fffff801`794018f0  00000000`00000000
fffff801`794018f8  00000000`00000000
fffff801`79401900  00000000`00000000
fffff801`79401908  00000000`00000000
fffff801`79401910  fffff801`790f42c0 nt!KiBreakpointTrapShadow
fffff801`79401918  fffff801`790f4340 nt!KiOverflowTrapShadow
fffff801`79401920  fffff801`790f4d40 nt!KiRaiseSecurityCheckFailureShadow
fffff801`79401928  fffff801`790f4dc0 nt!KiRaiseAssertionShadow
fffff801`79401930  fffff801`790f4e40 nt!KiDebugServiceTrapShadow
fffff801`79401938  fffff801`790f61c0 nt!KiSystemCall64Shadow
lkd> dd /c1 KiServiceTable L10
fffff801`786cf900  01c2d504
fffff801`786cf904  01cec300
fffff801`786cf908  067e4702
fffff801`786cf90c  09716200
fffff801`786cf910  05f20d00
fffff801`786cf914  034d4100
fffff801`786cf918  071cb205
fffff801`786cf91c  05f3d306
fffff801`786cf920  05bce805
fffff801`786cf924  05f1f301
fffff801`786cf928  06110d00
fffff801`786cf92c  069d3200
fffff801`786cf930  0674f000
fffff801`786cf934  06bb4f00
fffff801`786cf938  064aaf00
fffff801`786cf93c  05f2d800
```

Figure 125: Consulta de entradas de la tabla SSDT

```
lkd> .foreach /ps 1 /pS 1 ( offset {dd /c 1 nt!KiServiceTable L poi(nt!KeServiceDescriptorTable+10)}){ r $t0 = ( offset >>> 4) + nt!KiServiceTable; .printf "%p - %y\n", $t0, $t0 }
fffff80178892650 - nt!NtAccessCheck (fffff801`78892650)
fffff8017889e530 - nt!NtWorkerFactoryWorkerReady (fffff801`7889e530)
fffff80178d4dd70 - nt!NtAcceptConnectPort (fffff801`78d4dd70)
fffff80179040f20 - nt!NtMapUserPhysicalPagesScatter (fffff801`79040f20)
fffff80178cc19d0 - nt!NtWaitForSingleObject (fffff801`78cc19d0)
fffff80178a1cd10 - nt!NtCallbackReturn (fffff801`78a1cd10)
fffff80178dec420 - nt!NtReadFile (fffff801`78dec420)
fffff80178cc3630 - nt!NtDeviceIoControlFile (fffff801`78cc3630)
fffff80178c8c780 - nt!NtWriteFile (fffff801`78c8c780)
fffff80178cc1830 - nt!NtRemoveIoCompletion (fffff801`78cc1830)
fffff80178ce09d0 - nt!NtReleaseSemaphore (fffff801`78ce09d0)
fffff80178d6cc20 - nt!NtReplyWaitReceivePort (fffff801`78d6cc20)
fffff80178d44800 - nt!NtReplyPort (fffff801`78d44800)
fffff80178d8adf0 - nt!NtSetInformationThread (fffff801`78d8adf0)
fffff80178d1a3f0 - nt!NtSetEvent (fffff801`78d1a3f0)
fffff80178cc2680 - nt!NtClose (fffff801`78cc2680)
fffff80178d66ed0 - nt!NtQueryObject (fffff801`78d66ed0)
fffff80178d9f2e0 - nt!NtQueryInformationFile (fffff801`78d9f2e0)
fffff80178d19620 - nt!NtOpenKey (fffff801`78d19620)
fffff80178dade50 - nt!NtEnumerateValueKey (fffff801`78dade50)
fffff80178d64730 - nt!NtFindAtom (fffff801`78d64730)
fffff80178de4f90 - nt!NtQueryDefaultLocale (fffff801`78de4f90)
fffff80178cb47c0 - nt!NtQueryKey (fffff801`78cb47c0)
fffff80178cc0fe0 - nt!NtQueryValueKey (fffff801`78cc0fe0)
fffff80178d18ff0 - nt!NtAllocateVirtualMemory (fffff801`78d18ff0)
fffff80178cd9990 - nt!NtQueryInformationProcess (fffff801`78cd9990)
fffff80178d35da0 - nt!NtWaitForMultipleObjects32 (fffff801`78d35da0)
fffff80178d558b0 - nt!NtWriteFileGather (fffff801`78d558b0)
fffff80178cd2320 - nt!NtSetInformationProcess (fffff801`78cd2320)
fffff80178dafa00 - nt!NtCreateKey (fffff801`78dafa00)
fffff80178da0b20 - nt!NtFreeVirtualMemory (fffff801`78da0b20)
fffff80178f76980 - nt!NtImpersonateClientOfPort (fffff801`78f76980)
fffff80178d1bd70 - nt!NtReleaseMutant (fffff801`78d1bd70)
fffff80178d95780 - nt!NtQueryInformationToken (fffff801`78d95780)
```

Figure 126: Ejecución de script para consulta de entradas de la tabla SSDT

7.4. DESARROLLO DE MALWARE (AVANZADO)

7.4.1. PRUEBAS DE CONCEPTO MALWARE

En esta sección, y como parte adicional a la explicación de las técnicas utilizadas por los rootkits, implementamos varias pruebas de concepto de malware, como drivers de modo kernel, haciendo énfasis en las partes más complejas de estos. Durante este punto, meramente práctico, implementaremos operaciones de rootkits modernos como Scranos, Moriya y DirtyMoe, aprendiendo a manipular y ocultar procesos, archivos y conexiones de red asegurando la persistencia del malware en el sistema comprometido.

Estas POCs, que se os generarán con el script del módulo, las cuáles completaremos durante varias clases interactivas, no se materializan en un único fichero, ya que la programación que debemos efectuar es más avanzada, y el código fuente que se incluye, debidamente documentado, lo podemos asociar al inicio de una suite/herramienta/kit que comprenda, desde una visión ofensiva y defensiva, la aplicación y a su vez detección de técnicas de abuso ejecutadas por un rootkit.

```
=== Options ===
1.  Requirements -> Visual Studio 2019 C + Git + Python + NASM + ASL
2.  Set Up Environment -> EDK2
3.  Requirements -> Visual Studio 2022 C + SDK + WDK
4.  Set Up Environment -> Debugging and Signing Mode
5.  Tools -> Microsoft Sysinternals Suite + WinDbg + OSR Driver Loader
6.  Scripts -> WinDbg Scripting + JavaScript + PyKd
7.  Kernel-Mode Driver -> Hello World
8.  Kernel-Mode Driver -> Communication
9.  Kernel-Mode Driver -> DKOM
10. Kernel-Mode Driver -> Keylogger
Q.  Exit
============
```

Figure 127: Script del módulo "Reversing en Sistemas Operativos Windows"

7.4.2. GESTIÓN DE ESTRUCTURAS

La gestión de las estructuras del kernel, cuando se desarrolla un driver, es una tarea crítica ya que cualquier modificación puede derivar en un error fatal, además de que, como ya se comentó, muchas estructuras no están documentadas, teniendo que realizar un trabajo previo de depuración del sistema para saber cómo acceder a esos datos de forma manual, y posteriormente desarrollar un driver que implemente la funcionalidad.

En concreto, en la primera prueba de concepto, donde se maneja la estructura _KIDTENTRY64, que corresponde a una entrada de la tabla IDT, se tiene que buscar información adicional, para tratar de conocer el número, tipos y desplazamientos de sus campos, convirtiendo las fases iniciales del desarrollo en un proceso de prueba y error hasta finalmente conseguir un driver funcional, que puede no serlo para otras versiones del sistema operativo si las estructuras cambian.

Figure 128: Información de la estructura _KIDTENTRY64 en vergiliusproject.com

7.4.3. COMUNICACIÓN ENTRE MODO USUARIO Y MODO KERNEL

7.4.3.1. INPUT/OUTPUT CONTROL CODE

Como se puede ver en la última prueba de concepto, no se realiza una llamada expresa a la función de interés, sino que se define lo que denominamos un código de control de entrada/salida (IO_CTL_CODE o IOCTL), que puede ser utilizado por un programa que corra en modo usuario para realizar llamadas a efectuar esas operaciones, que sí necesitan ser ejecutadas desde un driver de modo kernel.

Estos códigos de control se definen mediante la macro CTL_CODE, que consta de cuatro campos, utilizados para construir un valor entero de 32 bits que representa el propio código de control:

- **Tipo de dispositivo**: Indica el tipo de dispositivo en el que se utilizará el código de control. En el desarrollo de drivers de modo kernel y como se puede observar en la POC, normalmente se indica un dispositivo de E/S genérico, mediante la cadena "FILE_DEVICE_UNKNOWN".
- **Código de control**: Es un valor único que se utiliza para identificar una operación de E/S específica, que el driver tiene que manejar. La recomendación de Microsoft es que estos códigos empiecen desde el valor 0x800.
- **Método de transferencia**: Indica el método de transferencia de datos que se utilizará para la operación de E/S, en el ejemplo, los datos se transferirán mediante un búfer, "METHOD_BUFFERED".
- **Tipo de acceso**: Indica el tipo de acceso que tiene el IO_CTL_CODE, en este caso, se permite el acceso al dispositivo desde cualquier proceso porque se ha inicializado este campo con la cadena "FILE_ANY_ACCESS".

Los códigos que define cada driver se suelen inicializar al principio, cuando se incluyen las librerías, y son utilizados posteriormente en la función DeviceIoControl(), donde el driver de acuerdo al valor dado realiza una determinada operación u otra.

7.4.3.2. DISPATCH ROUTINES

Las dispatch routines, parte fundamental de esta comunicación entre el modo usuario y el modo kernel, son funciones que se registran en una tabla (dispatch table) del objeto del driver (DRIVER_OBJECT), utilizadas para procesar los paquetes de solicitud de entrada/salida o IRP que son enviados al mismo.

Cuando un driver recibe un paquete IRP este es procesado en una función llamada dispatch routine, registrada previamente en el método DriverEntry para manejar un código de función mayor (IRP_MJ_*), el cual indica el tipo de solicitud de E/S a realizar.

En el contexto de las DispatchRoutines, las funciones mayores (IRP_MJ_*) están relacionadas con los códigos IOCTL, ya que cada función mayor tiene asociados un conjunto de ellos, utilizados para realizar operaciones específicas en el dispositivo. Por ejemplo, la función mayor IRP_MJ_READ está asociada con los códigos IOCTL utilizados para leer datos, por lo que la dispatch routine correspondiente para IRP_MJ_READ manejará todas las solicitudes de lectura y devolverá los datos solicitados que se han requerido.

Además de esta función mayor, se suelen utilizar IRP_MJ_CREATE (para crear un archivo o dispositivo), IRP_MJ_CLOSE (para cerrar un archivo o dispositivo), IRP_MJ_WRITE (para escribir datos en un dispositivo) y IRP_MJ_CONTROL (para realizar operaciones de control), entre otras, que las dispatch routines manejan, procesando adecuadamente las solicitudes de E/S entrantes.

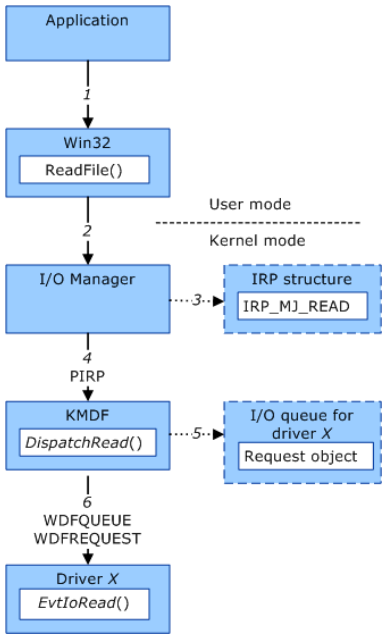


Figure 129: Solicitud para operación de lectura

8. BIBLIOGRAFÍA

- Russinovich, M., Solomon, D., Ionescu, A. & Yosifovich, P. **Windows Internals**. Microsoft Press.
- Marisetty, S., Rothman, M., & Zimmer, V. **Beyond BIOS: Developing with the Unified Extensible Firmware Interface**. Intel Press.
- Matrosov, A., Rodionov, E., & Bratus, S. **Rootkits and Bootkits: Reversing Modern Malware and Next Generation Threats**. No Starch Press.
- Blunden, B. **The Rootkit Arsenal: Escape and Evasion in the Dark Corners of the System**. Jones & Bartlett Learning.
- Hoglund, G., & Butler, J. **Rootkits: Subverting the Windows Kernel**. Addison-Wesley.

9. ANEXOS

9.1. ANEXO A: EXTENSIONES DEL ESTUDIO DEL MALWARE

Una vez comprendidos los fundamentos de los bootkits/rootkits y las técnicas básicas de infección, es importante profundizar en aspectos más avanzados que juegan un papel crucial en la efectividad y sofisticación del malware moderno, como lo son dos áreas que todavía debemos explorar para obtener un conocimiento más profundo de las operaciones relacionadas con la ingeniería inversa en windows: la ofuscación del código en los elementos ya vistos y la explotación de drivers de modo kernel vulnerables.

9.1.1. OFUSCACIÓN DE CÓDIGO FUENTE

La ofuscación de código es una técnica avanzada utilizada para dificultar el análisis y la ingeniería inversa de software malicioso. Tanto en aplicaciones UEFI (bootkits) como en drivers de modo kernel (rootkits), la ofuscación es una estrategia vital para proteger el aplicativo de la detección y el análisis, por lo que los desarrolladores de malware utilizan diversas técnicas que aseguran su código fuente sea difícil de entender, modificar y estudiar por las soluciones de seguridad.

Entre las diversas técnicas de ofuscación podríamos implementar las siguientes o automatizar su adopción haciendo uso de un programa externo diseñado para aplicar esto:

- **Renombrado de símbolos**: Cambiar los nombres de funciones y variables a nombres no significativos, dificultando el seguimiento del flujo del programa.
- **Inserción de código inútil**: Añadir código que no realiza ninguna función útil pero complica el análisis estático y dinámico.
- **Encriptación de cadenas**: Proteger las cadenas de texto utilizadas en el programa, haciéndolas ilegibles hasta que se descifren en tiempo de ejecución.
- **Control de flujo no lineal**: Introducir estructuras de control de flujo como saltos condicionales y bucles, para hacer que el flujo del programa sea difícil de seguir.
- **Polimorfismo**: Modificar el código en cada ejecución para que no haya dos instancias idénticas del mismo software, dificultando la creación de firmas de detección.
- **Codificación de flujo de control**: Alterar el flujo del programa utilizando técnicas como el reemplazo de funciones con punteros de función y el uso de trampolines, que son secuencias de código que saltan a otras ubicaciones de memoria.
- **Técnicas de anti-debugging**: Implementar métodos para detectar y evadir herramientas de depuración, dificultando el análisis dinámico del código malicioso.

9.1.2. EXPLOTACIÓN DE DRIVERS VULNERABLES

Ya que los drivers de modo kernel operan en Ring 0, estos se han convertido en objetivos atractivos para los atacantes que buscan ejecutar código malicioso en ese espacio. Por este motivo, la identificación y posterior explotación de vulnerabilidades existentes en estos controladores, es otra de las extensiones que de forma natural debemos investigar.

A través del abuso de vulnerabilidades existentes en drivers de modo kernel debidamente firmados, los atacantes pueden inyectar rootkits y otros módulos maliciosos que se ejecutarán con los mismos privilegios elevados que el driver comprometido, obteniendo, como ya sabemos ahora, el control total del sistema y de su espacio más sensible.

Entre las vulnerabilidades más comunes que pueden afectar a los drivers de modo kernel, tendríamos el desbordamiento de buffer (Buffer Overflow), donde un programa escribe más datos en un buffer de los que puede manejar, el uso después de liberación (Use After Free), que ocurre cuando un programa accede a la memoria después de haber sido liberada, el doble fetch, una condición de carrera en la que una variable es leída dos veces sin asegurar su integridad, la divulgación de memoria (Memory Disclosure), que expone información sensible, y la escritura arbitraria (Arbitrary Write), que permite escribir datos en ubicaciones de memoria críticas. Es el abuso de estas vulnerabilidades, entre otras, las que permiten a los atacantes desplegar componentes maliciosos en ese nivel.

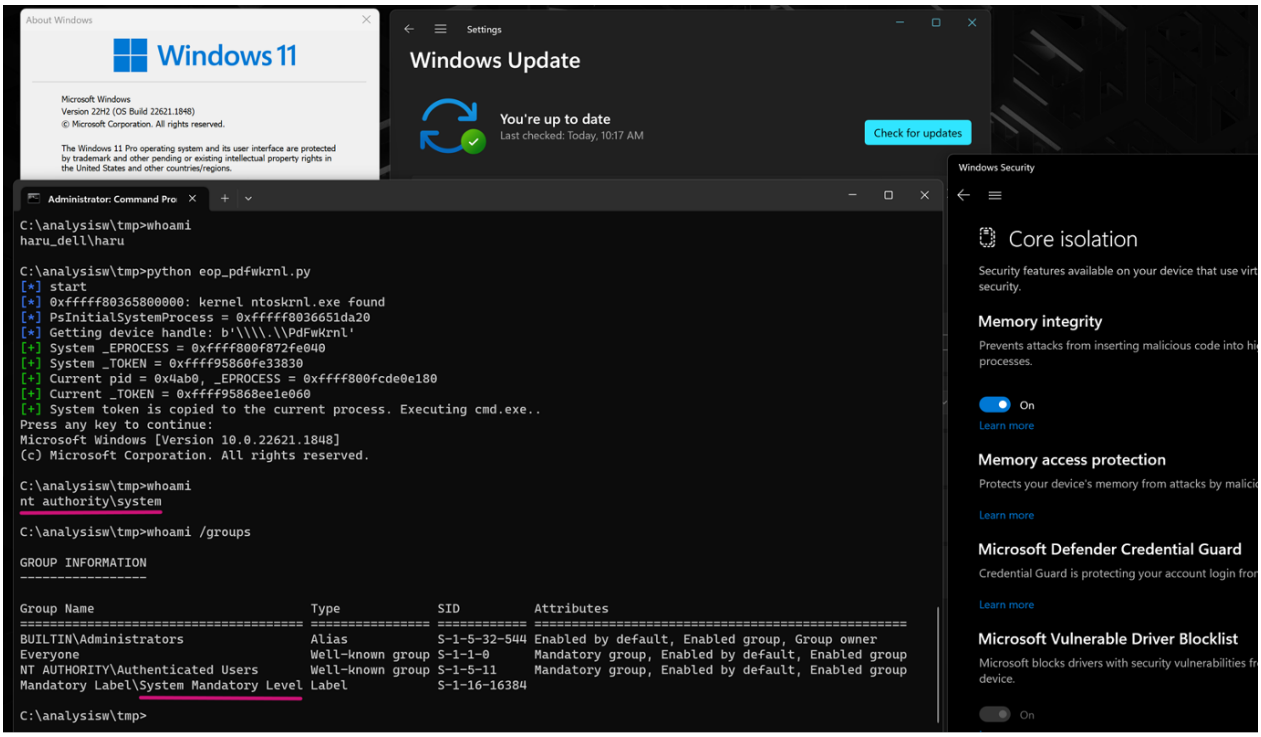


Figure 130: Explotación de Driver de modo kernel vulnerable

9.2. ANEXO B: DOCUMENTOS COMPLEMENTARIOS

- **M10. Reversing de Sistemas Operativos Windows (Recursos Adicionales)**: Documento de recursos adicionales, en el que se enumera y hace referencia a una gran lista de recursos auxiliares (videos formativos, presentaciones, artículos, ilustraciones, …) para profundizar en los temas tratados durante el módulo.
- **M10. Reversing de Sistemas Operativos Windows (Laboratorios)**: Documento auxiliar sobre la entrega de actividades/laboratorios relacionados con el módulo.